

Contents

Base64 Function Reference	6
Function List	6
Examples	6
Error Handling	7
String Operation Function Reference	7
Function List	7
contains	8
starts_with	9
ends_with	9
find	9
substring	9
char_at	9
replace	10
to_upper	10
to_lower	10
trim	10
trim_start	10
trim_end	11
repeat	11
reverse	11
byte_len	11
split	11
join	12
to_int	12
to_float	12
to_str	13
Built-in Function Reference	13
Function List	13
print	17
Some	17
length	18
has_key	18
add	18
remove	18
range	18
exit	19
arguments	19
sleep	19
env	20
append	20
pop	21
reverse (list)	21
slice	21
take	21
tap	22
filter	22
map	22
sort	22
sort!	23
reverse!	23
appended	23

append!	23
first	23
last	24
get (Map)	24
iter	24
next	24
to_list	25
Collection Reference (Tuple, List, Map, Set)	25
Tuple	25
List	26
Copy-on-Write (CoW) Semantics	32
Map	32
Set	34
Iterator	36
Design by Contract (DbC)	37
Preconditions (require)	37
Postconditions (ensure)	38
Combined Example	38
Record Invariants (invariant)	38
Rules	39
Control Flow Reference	39
if / else	39
while	40
for	40
async / await	42
@parallel for	42
break	43
continue	43
... (Ellipsis)	43
when	43
match	44
Scope Rules	47
Directives	48
Syntax	48
Supported Targets	48
Built-in Directives	48
Notes	52
Error Reference	53
Error Format	53
Compile Errors	53
Runtime Errors	54
Filesystem Function Reference	55
Function List	55
Examples	56
Notes	58
Function Reference	58
Function Definition Syntax	58
Parameter and Return Types	58

Recursion	59
Overloading	60
Default Arguments	61
Unit Type Functions	62
Tasks And Async Functions	62
Lambda Functions	63
Closures	63
Function Type	64
Higher-Order Functions	64
Generic Functions	65
UFCS (Uniform Function Call Syntax)	66
Operator Overloading	66
Checked/Saturating Arithmetic	68
GC Reference	68
Overview	68
Import	69
Functions	69
How It Works	69
Static Analysis Optimization	69
Example	69
HTTP Reference	69
Types	69
Functions (from <code>http</code>)	70
Usage Example	71
Behavior	73
Supported Status Codes	74
HTTP Client	75
Error Handling	77
I/O Function Reference	77
Function List	77
Examples	78
Error Handling	79
Notes	79
JSON Function Reference	79
Overview	79
Function List	79
Usage Examples	81
Notes	82
Math Functions (<code>math</code>)	82
Overview	82
Constants	82
Absolute Value	82
Rounding	82
Power and Root	83
Logarithm	83
Exponential	83
Trigonometric	83
Inverse Trigonometric	84
Two-argument Functions	84
Predicates	84

Network (TCP) Reference	84
Types	84
Functions (from <code>net</code>)	85
Built-in Overloaded Functions	85
Usage Example	86
Timeout Configuration	87
Error Handling	88
Byte Conversion	88
Operator Reference	88
Precedence Table	88
Arithmetic Operators	89
Comparison Operators	89
Logical Operators	90
Bitwise Operators	90
Error Propagation Operator (<code>? / !!</code>)	90
<code>when</code> : Conditional Expression	91
Range Operator	91
Null Coalescing Operator (<code>??</code>)	91
Compound Assignment Operators	92
Increment / Decrement Operators	92
Type Rules for Operations	93
Operator Overloading	93
Package Reference	96
Overview	96
Import Syntax	97
Package Resolution	97
Standard Library (<code>std</code>)	98
Search Path Priority	99
<code>RY_PATH</code> Environment Variable	100
Constraints	100
Creating Package Files	100
Path Function Reference	101
Function List	101
Examples	101
Project Management	102
CLI Overview	102
<code>ry init</code> - Project Initialization	103
<code>ry new</code> - Create a New Project	103
<code>ry run</code> - Run Project Scripts	103
<code>ry fmt</code> - Code Formatter	104
<code>ry test</code> - Run Tests	104
<code>ry self-update</code> - Self Update	105
<code>package.toml</code> Configuration File	106
Regular Expression Reference	107
Regex Literal Syntax	107
Function List	107
Supported Pattern Syntax	108
Usage Examples	109
Record Reference	110

Overview	110
Definition Syntax	110
Constructor	110
Field Access	110
Field Assignment	110
Usage as Function Parameters and Return Values	111
Nested Structs	111
Comparison (<code>==</code> / <code>!=</code>)	111
Record Subtyping (Inheritance)	112
Constraints and Errors	113
Enumerations (<code>enum</code>)	113
Testing	115
Running Tests	115
Syntax	116
Output Format	117
Example	117
Mocking	118
Parameterized Tests (<code>@each</code>)	118
Property-Based Tests (<code>@property</code>)	119
Test Coverage	119
Test Outline	119
Limitations	120
Thread Reference	120
Types	120
Import	120
Thread	120
Lock (Mutex)	121
RWLock (Read-Write Lock)	121
Semaphore	121
Barrier	122
AtomicInt	122
AtomicBool	122
Comparison with <code>async/await</code>	123
Type Reference	123
Type List	123
Type Annotation Syntax	125
Available Type Names	125
Type Aliases	126
Literal Types	126
Range Type	127
<code>none</code> Keyword and Option Type Shorthand	127
Weak References (<code>weak T</code>)	128
F-String (String Interpolation)	129
Type Casting (<code>as</code>)	129
Enum with Associated Data (ADT)	130
Generic Enum	131
Error Type	131
Union Type	132
<code>any</code> Type	133
Type Rules (Type Conversion in Operations)	135
Type Safety Constraints	136

Base64 Function Reference

Base64 encoding and decoding. All functions require explicit import from `base64`.

```
from base64 import encode, decode, encode_url_safe, decode_url_safe
```

Function List

Function	Signature	Description
<code>encode</code>	<code>(str) -> str</code>	Encodes a string to standard base64
<code>decode</code>	<code>(str) -> Result<str, Error></code>	Decodes a standard base64 string
<code>encode_url_safe</code>	<code>(str) -> str</code>	Encodes a string to URL-safe base64 (no padding)
<code>decode_url_safe</code>	<code>(str) -> Result<str, Error></code>	Decodes a URL-safe base64 string

Examples

Basic Encoding and Decoding

```
from base64 import encode, decode

encoded = encode("Hello, World!")
print(encoded)  # SGVsbG8sIFdvcmxkIQ==

match decode(encoded):
    case Ok(s):
        print(s)  # Hello, World!
    case Err(e):
        print(e.message)
```

URL-safe Base64

URL-safe base64 uses `-` and `_` instead of `+` and `/`, and omits padding (`=`). Useful for URLs, filenames, and tokens.

```
from base64 import encode_url_safe, decode_url_safe

encoded = encode_url_safe("data with special chars: ?&=")
# No + / or = in the output

match decode_url_safe(encoded):
    case Ok(s):
        print(s)
    case Err(e):
        print(e.message)
```

Working with Byte Data

To encode/decode byte data, combine with `to_bytes` / `bytes_to_str` from `io`.

```
from base64 import encode, decode
from io import to_bytes, bytes_to_str
```

```
bytes = to_bytes("binary data")
encoded = encode(bytes_to_str(bytes)?)
```

Error Handling

`decode` and `decode_url_safe` return `Result<str, Error>`. Decoding fails if the input contains invalid base64 characters.

```
match decode("!!!not-valid!!!"):
  case Ok(s):
    print(s)
  case Err(e):
    print(e.message) # "invalid base64 character at position 0"
```

With the `?` operator:

```
function process(input: str) -> Result<str, Error>:
  decoded = decode(input)?
  return Ok(decoded)
```

[English](#) | [日本語](#) | [繁體中文](#)

String Operation Function Reference

A list of operation functions for strings (`str`). All functions support UFCS notation.

Note: All string operations are UTF-8 aware. `length()`, `char_at()`, `substring()`, `find()`, and `reverse()` operate on Unicode code points, not bytes. Use `byte_len()` if you need the byte length.

Function List

Search and Check

Function	Signature	Description
<code>contains</code>	<code>(str, str, bool = false) -> bool</code>	Returns whether a substring is contained
<code>starts_with</code>	<code>(str, str, bool = false) -> bool</code>	Returns whether it starts with a prefix
<code>ends_with</code>	<code>(str, str, bool = false) -> bool</code>	Returns whether it ends with a suffix
<code>find</code>	<code>(str, str) -> Option<int></code>	Returns the character position of a substring (<code>None</code> if not found)

Extraction and Transformation

Function	Signature	Description
<code>substring</code>	<code>(str, int, int) -> str</code>	Extract a substring (character indices)
<code>char_at</code>	<code>(str, int) -> str</code>	Get the UTF-8 character at a specified position
<code>replace</code>	<code>(str, str, str) -> str</code>	Replace all occurrences of a substring

Case Conversion

Function	Signature	Description
<code>to_upper</code>	<code>str -> str</code>	Convert to ASCII uppercase
<code>to_lower</code>	<code>str -> str</code>	Convert to ASCII lowercase

Whitespace Removal

Function	Signature	Description
<code>trim</code>	<code>str -> str</code>	Remove leading and trailing whitespace
<code>trim_start</code>	<code>str -> str</code>	Remove leading whitespace
<code>trim_end</code>	<code>str -> str</code>	Remove trailing whitespace

Generation and Processing

Function	Signature	Description
<code>repeat</code>	<code>(str, int) -> str</code>	Repeat a string n times
<code>reverse</code>	<code>str -> str</code>	Reverse a string (UTF-8 aware)
<code>byte_len</code>	<code>str -> int</code>	Returns the byte length of a string

Split and Join

Function	Signature	Description
<code>split</code>	<code>(str, str) -> List<str></code>	Split by delimiter
<code>join</code>	<code>(List<str>, str) -> str</code>	Join with separator

Type Conversion

Function	Signature	Description
<code>to_int</code>	<code>str -> Result<int, Error></code>	Convert string to integer
<code>to_float</code>	<code>str -> float</code>	Convert string to floating-point number
<code>to_str</code>	<code>int/float/bool/str/enum/record -> str</code>	Convert value to string

contains

Signature: `contains(string: str, substring: str, ignore_case: bool = false) -> bool`

Returns whether string `string` contains the substring `substring`. When `ignore_case` is `true`, the comparison is case-insensitive (ASCII only).

```
print(contains("hello", "ell"))           # true
print("hello".contains("xyz"))           # false (UFCS)
print(contains("Hello World", "hello", true)) # true (case-insensitive)
```

starts_with

Signature: `starts_with(string: str, prefix: str, ignore_case: bool = false) -> bool`

Returns whether string `string` starts with `prefix`. When `ignore_case` is `true`, the comparison is case-insensitive (ASCII only).

```
print(starts_with("hello", "hel"))           # true
print("hello".starts_with("world"))          # false (UFCS)
print(starts_with("Hello", "hello", true))   # true (case-insensitive)
```

ends_with

Signature: `ends_with(string: str, suffix: str, ignore_case: bool = false) -> bool`

Returns whether string `string` ends with `suffix`. When `ignore_case` is `true`, the comparison is case-insensitive (ASCII only).

```
print(ends_with("hello", "llo"))             # true
print("hello".ends_with("world"))            # false (UFCS)
print(ends_with("Hello World", "WORLD", true)) # true (case-insensitive)
```

find

Signature: `find(string: str, substring: str) -> Option<int>`

Returns the character position of the first occurrence of substring `substring` in string `string`. Returns `None` if not found.

```
print(find("hello world", "world"))          # Some(6)
print(find("hello", "xyz"))                  # None
print("abcdef".find("cd"))                   # Some(2) (UFCS)
```

substring

Signature: `substring(string: str, start: int, end: int) -> str`

Returns the substring of `string` from `start` to `end` (exclusive). Indices are character positions (UTF-8 aware).

Out-of-range indices are clamped to `[0, length]`. If `end < start` after clamping, returns an empty string.

```
print(substring("hello world", 0, 5))        # hello
print(substring("hello world", 6, 11))       # world
print("abcdef".substring(1, 4))              # bcd (UFCS)
print(substring("hello", -1, 100))           # hello (clamped)
```

char_at

Signature: `char_at(string: str, i: int) -> str`

Returns the UTF-8 character at position `i` in string `string` as a string. Raises a runtime error if the index is out of bounds.

Negative indices wrap around from the end (Python-style): `-1` refers to the last character, `-2` to the second-to-last, and so on.

```
print(char_at("hello", 0))    # h
print(char_at("hello", -1))  # o (last character)
print("abc".char_at(2))      # c (UFCS)
```

replace

Signature: `replace(string: str, old: str, new: str) -> str`

Returns a new string with all occurrences of `old` in `string` replaced with `new`.

```
print(replace("hello world", "world", "ry"))  # hello ry
print(replace("aaa", "a", "bb"))              # bbbbbb
print("foo bar foo".replace("foo", "baz"))    # baz bar baz (UFCS)
```

to_upper

Signature: `to_upper(string: str) -> str`

Returns a new string with ASCII lowercase letters (a-z) converted to uppercase.

```
print(to_upper("hello"))      # HELLO
print("Hello World".to_upper()) # HELLO WORLD (UFCS)
```

to_lower

Signature: `to_lower(string: str) -> str`

Returns a new string with ASCII uppercase letters (A-Z) converted to lowercase.

```
print(to_lower("HELLO"))      # hello
print("Hello World".to_lower()) # hello world (UFCS)
```

trim

Signature: `trim(string: str) -> str`

Returns a new string with leading and trailing whitespace characters (spaces, tabs, newlines, carriage returns) removed.

```
print(trim(" hello "))      # hello
print(" hi ".trim())        # hi (UFCS)
```

trim_start

Signature: `trim_start(string: str) -> str`

Returns a new string with leading whitespace characters removed.

```
print(trim_start(" hello ")) # hello
print(" hi".trim_start())    # hi (UFCS)
```

trim_end

Signature: trim_end(string: str) -> str

Returns a new string with trailing whitespace characters removed.

```
print(trim_end(" hello "))    # hello
print("hi ".trim_end())       # hi (UFCS)
```

repeat

Signature: repeat(string: str, count: int) -> str

Returns a new string with **string** repeated **count** times.

```
print(repeat("ab", 3))        # ababab
print("ha".repeat(3))         # hahaha (UFCS)
```

reverse

Signature: reverse(string: str) -> str

Returns a new string with the characters reversed (UTF-8 aware).

```
print(reverse("hello"))       # olleh
print("abc".reverse())        # cba (UFCS)
```

byte_len

Signature: byte_len(string: str) -> int

Returns the byte length of string **string**. Unlike **length()**, which returns the number of UTF-8 characters, **byte_len()** returns the number of bytes.

```
print(byte_len("hello"))      # 5
print(byte_len("あいう"))     # 9
print(length("あいう"))       # 3 (characters)
```

split

Signature: split(string: str, delimiter: str) -> List<str>

Splits string **string** by delimiter **delimiter** and returns a **List<str>**.

When the delimiter is an empty string "", the string is split into individual characters (UTF-8 aware).

```
parts = split("a,b,c", ",")
print(parts[0])    # a
print(parts[1])    # b
print(parts[2])    # c

for word in "hello world".split(" "):
    print(word)
# hello
# world
```

```

# Split into characters
chars = split("hello", "")
print(chars)    # [h, e, l, l, o]

# UTF-8 characters
chars = split("あいう", "")
print(chars)    # [あ, い, う]

```

join

Signature: `join(values: List<str>, sep: str) -> str`

Joins the elements of a string list with separator `sep` and returns a string.

```

parts = ["a", "b", "c"]
print(join(parts, ","))      # a,b,c
print(parts.join("-"))      # a-b-c (UFCS)
print(",".join(parts))      # a,b,c (UFCS, Python-style)

```

to_int

Signature: `to_int(string: str) -> Result<int, Error>`

Converts a string to an integer. Leading whitespace is allowed. Returns `Err` if the string is empty, contains invalid characters, or overflows.

```

match to_int("42"):
  case Ok(v):
    print(v)                # 42
  case Err(e):
    print(e.message)

match "123".to_int():
  case Ok(v):
    print(v)                # 123
  case Err(e):
    print(e.message)

# Invalid input returns Err
print(to_int("abc"))        # Err(Error("to_int: invalid character in 'abc'"))
print(to_int(""))          # Err(Error("to_int: empty string"))

```

to_float

Signature: `to_float(string: str) -> float`

Converts a string to a floating-point number.

```

print(to_float("3.14"))    # 3.14
print("2.5".to_float())    # 2.5 (UFCS)

```

to_str

Signature: `to_str(v: int | float | bool | str | enum | record) -> str`

Converts a value to a string.

Type	Conversion Format
int	%ld
float	%g
bool	"true" / "false"
str	Returned as-is
enum	Variant name (e.g. "Red")
record	TypeName(field1: val1, field2: val2)

Record types automatically generate a `to_str` representation. If a user-defined function `to_str(v: MyRecord) -> str` is provided, it takes precedence over the auto-generated version. This also works with `print()` and f-string interpolation.

```
print(to_str(42))           # 42
print(to_str(3.14))         # 3.14
print(to_str(true))         # true
print(99.to_str())          # 99 (UFCS)
```

```
enum Color:
    Red
    Green
```

```
print(to_str(Color::Red))   # Red
```

```
record Point:
    x: int
    y: int
```

```
p = Point(10, 20)
print(to_str(p))            # Point(x: 10, y: 20)
print(p)                    # Point(x: 10, y: 20)
print(f"pos={p}")           # pos=Point(x: 10, y: 20)
```

[English](#) | [日本語](#) | [繁體中文](#)

Built-in Function Reference

Function List

Core

Function	Description
<code>print()</code> / <code>print(expr1, expr2, ...)</code>	Prints values to standard output (space-separated)
<code>length(value)</code>	Returns the number of elements in a list, map, or set, or the number of UTF-8 characters in a string
<code>range(n)</code> / <code>range(start, end)</code> / <code>range(start, end, step)</code>	Generates a list of integers
<code>exit(code)</code>	Terminates the process with the given exit code
<code>arguments()</code>	Returns command-line arguments as <code>List<str></code>

Function	Description
<code>available_parallelism()</code>	Returns the runtime worker count as <code>int</code>
<code>sleep(duration_ms)</code>	Suspends execution for the specified number of milliseconds
<code>env(key)</code>	Returns the environment variable as <code>Option<str></code>
<code>env(key, default)</code>	Returns the environment variable, or <code>default</code> if not set
<code>send(stream, data)</code>	Sends <code>List<u8></code> through <code>TcpStream</code> or <code>TlsStream</code> , returns <code>Result<int, Error></code>
<code>receive(stream, max)</code>	Receives up to <code>max</code> bytes from <code>TcpStream</code> or <code>TlsStream</code> as <code>Result<List<u8>, Error></code>
<code>close(handle)</code>	Closes a <code>TcpStream</code> , <code>TlsStream</code> , or <code>TcpListener</code>
<code>block_on(task)</code>	Blocks the current thread until a <code>Task<T></code> completes and returns its result
<code>to_str(value)</code>	Converts a value to its string representation (<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code> , <code>record</code> , <code>enum</code> , <code>tuple</code> , <code>List</code> , <code>Map</code> , <code>Set</code> , <code>Result</code> , <code>Option</code>)
<code>fail()</code> / <code>fail(message)</code>	Marks the current test as failed (only available in <code>ry test</code> mode)

Option

Function	Description
<code>Some(expr)</code>	Constructs the value-present variant of an <code>Option</code> type

Result / Error

Function	Description
<code>Ok(value)</code>	Constructs the success variant of a <code>Result<T, Error></code>
<code>Err(error)</code>	Constructs the error variant of a <code>Result<T, Error></code>
<code>Error(message)</code>	Creates an <code>Error</code> value with a message
<code>Error(message, code)</code>	Creates an <code>Error</code> value with a message and error code
<code>result.and_then(closure)</code>	If <code>Ok</code> , calls <code>closure</code> (which returns <code>Result<U, E></code>); if <code>Err</code> , propagates the error
<code>result.map(closure)</code>	If <code>Ok</code> , applies <code>closure</code> to the value and wraps the return in <code>Ok</code> ; if <code>Err</code> , propagates the error

Checked Arithmetic

Function	Description
<code>checked_add(a, b)</code>	Returns <code>Ok(a + b)</code> if no overflow, otherwise <code>Err(Error("arithmetic overflow"))</code>
<code>checked_sub(a, b)</code>	Returns <code>Ok(a - b)</code> if no overflow, otherwise <code>Err(Error("arithmetic overflow"))</code>
<code>checked_mul(a, b)</code>	Returns <code>Ok(a * b)</code> if no overflow, otherwise <code>Err(Error("arithmetic overflow"))</code>

Function	Description
<code>saturating_add(a, b)</code>	Returns <code>a + b</code> , clamped to <code>int</code> range on overflow
<code>saturating_sub(a, b)</code>	Returns <code>a - b</code> , clamped to <code>int</code> range on overflow
<code>saturating_mul(a, b)</code>	Returns <code>a * b</code> , clamped to <code>int</code> range on overflow
<code>wrapping_add(a, b)</code>	Returns <code>a + b</code> with wrapping on overflow
<code>wrapping_sub(a, b)</code>	Returns <code>a - b</code> with wrapping on overflow
<code>wrapping_mul(a, b)</code>	Returns <code>a * b</code> with wrapping on overflow

Collection Operations

Function	Description
<code>has_key(map, key)</code>	Returns whether a key exists in the map
<code>add(set, value)</code>	Adds an element to a set (duplicates are ignored)
<code>remove(set, value)</code>	Removes an element from a set
<code>append(list, value) / append!(list, value)</code>	Adds an element to the end of a list (mutating)
<code>appended(list, value)</code>	Returns a new list with the element added (non-mutating)
<code>pop(list)</code>	Removes and returns the last element as <code>Option<T></code>
<code>reverse(list)</code>	Returns a new reversed list (also works on strings)
<code>reverse!(list)</code>	Reverses a list in place (mutating)
<code>slice(list, start, end)</code>	Returns a new sub-list from start to end
<code>take(list, count)</code>	Returns a new list with the first count elements
<code>tap(list, function)</code>	Calls function on each element for side effects, returns the original list
<code>filter(list, pred)</code>	Returns a new list with elements matching the predicate
<code>map(list, function)</code>	Returns a new list with each element transformed
<code>sort(list) / sort(list, comp)</code>	Returns a new sorted list (default ascending)
<code>sort!(list) / sort!(list, comp)</code>	Sorts a list in place (mutating)
<code>insert(list, i, val)</code>	Inserts an element at index <code>i</code>
<code>remove_at(list, i)</code>	Removes and returns the element at index <code>i</code>
<code>items(map)</code>	Returns a list of (key, value) tuples
<code>remove(map, key)</code>	Removes the entry with the specified key
<code>get(map, key)</code>	Returns the value for key as <code>Option<V></code>
<code>get(map, key, default)</code>	Returns the value for key, or default if not found
<code>union(set, set)</code>	Returns the union of two sets
<code>intersection(set, set)</code>	Returns the intersection of two sets
<code>difference(set, set)</code>	Returns the difference of two sets
<code>symmetric_difference(set, set)</code>	Returns the symmetric difference of two sets
<code>is_subset(set, set)</code>	Returns whether the first set is a subset of the second
<code>is_superset(set, set)</code>	Returns whether the first set is a superset of the second
<code>first(list)</code>	Returns the first element as <code>Option<T></code> , or <code>None</code> if empty
<code>last(list)</code>	Returns the last element as <code>Option<T></code> , or <code>None</code> if empty
<code>remove(list, value)</code>	Removes the first occurrence of value from a list
<code>is_empty(list)</code>	Returns whether the list is empty
<code>distinct(list)</code>	Returns a new list with duplicates removed
<code>flatten(list)</code>	Returns a new list with nested lists flattened
<code>reduce(list, fn)</code>	Reduces a list to a single value using the reducer function

Function	Description
<code>fold(list, init, fn)</code>	Folds a list with an initial accumulator value
<code>any(list, pred)</code>	Returns true if any element matches the predicate
<code>all(list, pred)</code>	Returns true if all elements match the predicate
<code>sum(list)</code>	Returns the sum of all elements
<code>min(list)</code>	Returns the minimum element
<code>max(list)</code>	Returns the maximum element
<code>enumerate(list)</code>	Returns a list of (index , value) tuples
<code>zip(list1, list2)</code>	Returns a list of (a , b) tuples pairing elements from two lists
<code>keys(map)</code>	Returns all keys as a List<K>
<code>values(map)</code>	Returns all values as a List<V>
<code>merge(map1, map2)</code>	Returns a new map containing entries from both maps

Iterator

Function	Description
<code>iter(collection)</code>	Creates a lazy iterator from a List, Set, or Map
<code>next(iter)</code>	Returns the next element as Option<T> , or None if exhausted
<code>to_list(iter)</code>	Collects all remaining iterator elements into a List<T>
<code>filter(iter, pred)</code>	Returns a lazy iterator that yields only elements matching the predicate
<code>map(iter, function)</code>	Returns a lazy iterator that transforms each element
<code>take(iter, count)</code>	Returns a lazy iterator that yields at most count elements

String Operations

Function	Description
<code>contains(string, substring)</code>	Whether a substring is contained
<code>starts_with(string, prefix)</code>	Whether it starts with a prefix
<code>ends_with(string, suffix)</code>	Whether it ends with a suffix
<code>find(string, substring)</code>	Character position of a substring (Option<int>)
<code>byte_len(string)</code>	Returns the byte length of a string
<code>substring(string, start, end)</code>	Extract a substring
<code>char_at(string, i)</code>	Get the character at a specified position
<code>replace(string, old, new)</code>	Replace all occurrences of a substring
<code>to_upper(string) / to_lower(string)</code>	Uppercase / lowercase conversion
<code>trim(string) / trim_start(string) / trim_end(string)</code>	Whitespace removal
<code>repeat(string, count)</code>	Repeat a string n times
<code>reverse(string)</code>	Reverse a string
<code>split(string, delimiter)</code>	Split a string into a list
<code>join(list, sep)</code>	Join list elements with a separator
<code>to_int(s) / to_float(s) / to_str(v)</code>	Type conversion (to_int returns Result<int, Error>)

-> See [String Operation Function Reference](#) for details

print

Signature: `print()` / `print(expr1, expr2, ...)`

Prints one or more values to standard output, separated by spaces. A newline is appended at the end. When called with no arguments, prints only a newline.

Type	Output Format
int	%ld
float	%g
bool	true / false
str	%s
Result (Ok)	Ok(value)
Result (Err)	Err(value)
Option (Some)	Some(value)
Option (None)	None
list	[elem1, elem2, ...]
map	{key1: val1, key2: val2, ...}
set	{elem1, elem2, ...}
tuple	(elem1, elem2, ...)
enum	Variant name (e.g., Red)
record	RecordName(field: val, ...)

```
print(42)           # 42
print(3.14)         # 3.14
print(true)        # true
print("hello")     # hello
print(Ok(42))      # Ok(42)
print(Err(Error("fail"))) # Err(Error: fail (code: 0))
print(Some(1))     # Some(1)
print(None)       # None
print([1, 2, 3])  # [1, 2, 3]
print({"a": 1})   # {a: 1}
print({1, 2, 3})  # {1, 2, 3}
print((1, "hello")) # (1, hello)
```

```
# Multiple arguments (space-separated)
print(1, 2, 3)           # 1 2 3
print("hello", "world")  # hello world
print(1, "hello", true)  # 1 hello true
print()                  # (empty line)
```

Some

Signature: `Some(expr) -> Option<T>`

Constructs the value-present variant of an Option type.

```
x: Option<int> = Some(42)
print(x)      # Some(42)
```

length

Signature: `length(x: List<T> | Map<K, V> | Set<T> | str) -> int`

Returns the number of elements in a list, map, or set, or the number of UTF-8 characters in a string. Use `byte_len()` for the byte length.

```
print(length([1, 2, 3]))           # 3
print(length({"a": 1, "b": 2}))    # 2
print(length({1, 2, 3}))           # 3
print(length("hello"))             # 5
print(length("あい"))              # 3 (UTF-8 characters)
```

has_key

Signature: `has_key(m: Map<K, V>, key: K) -> bool`

Returns whether a specified key exists in the map. UFCS notation is also available.

```
m = {"a": 1, "b": 2}
print(has_key(m, "a"))             # true
print(m.has_key("z"))              # false (UFCS)
```

add

Signature: `add(s: Set<T>, value: T)`

Adds an element to a set. Does nothing if the element already exists. UFCS notation is also available.

```
s = {1, 2, 3}
s.add(4)                            # UFCS
add(s, 5)                          # Normal call
s.add(1)                            # Ignored because it already exists
print(length(s))                   # 5
```

remove

Signature: `remove(s: Set<T>, value: T)`

Removes an element from a set. UFCS notation is also available.

```
s = {1, 2, 3}
s.remove(2)                         # UFCS
print(2 in s)                      # false
```

range

Signature: `range(n: int) -> List<int> / range(start: int, end: int) -> List<int> / range(start: int, end: int, step: int) -> List<int>`

Generates a list of integers.

Form	Generated Values
<code>range(n)</code>	<code>[0, 1, ..., n-1]</code>
<code>range(start, end)</code>	<code>[start, start+1, ..., end-1]</code>
<code>range(start, end, step)</code>	<code>[start, start+step, start+2*step, ...]</code> (up to but not including <code>end</code>)

- When `step > 0`, generates values from `start` ascending toward `end`.
- When `step < 0`, generates values from `start` descending toward `end`.
- When `step == 0`, a runtime error occurs.
- If the range is empty (e.g., `range(0, 10, -1)`), returns an empty list.

```
print(range(3))          # [0, 1, 2]
print(range(2, 5))       # [2, 3, 4]
print(range(0, 10, 2))   # [0, 2, 4, 6, 8]
print(range(10, 0, -3))  # [10, 7, 4, 1]
```

```
for i in range(3):
    print(i)
# 0
# 1
# 2
```

exit

Signature: `exit(code: int)`

Terminates the process immediately with the given exit code. Code after `exit()` is unreachable.

```
exit(0)      # normal termination
exit(1)      # error termination
```

arguments

Signature: `arguments() -> List<str>`

Returns the command-line arguments passed to the script as a list of strings. Does not include the interpreter name or the script filename —only the arguments after the script path.

```
# Run: ry script.ry hello world
a = arguments()
print(length(a))    # 2
print(a[0])         # hello
print(a[1])         # world

for x in arguments():
    print(x)
```

sleep

Signature: `sleep(duration_ms: int) -> Unit`

Suspends execution of the current thread for the specified number of milliseconds. If `duration_ms` is 0 or negative, the function returns immediately.

```
sleep(1000)    # wait 1 second
sleep(0)       # returns immediately
```

env

Signature: `env(key: str) -> Option<str> / env(key: str, default: str) -> str`

Returns the value of an environment variable. The one-argument form returns `Option<str>` (`Some(value)` if set, `None` if not). The two-argument form returns the value or `default` if the variable is not set.

If a `.env` file exists in the project root (the directory containing `package.toml`), its entries are automatically loaded into the process environment at startup. Existing environment variables are not overwritten by `.env` values.

Security note: `.env` files typically contain secrets (API keys, database passwords, tokens, etc.). Do **not** commit `.env` to version control (add it to `.gitignore` or equivalent), and treat its contents as sensitive configuration.

```
# One-argument form: returns Option<str>
path = env("PATH")
match path:
  case Some(v):
    print(v)
  case None:
    print("PATH not set")

# Two-argument form: returns str with default
port = env("PORT", "8080")
print(port)  # "8080" if PORT is not set
```

.env file format

```
# Comments start with #
DATABASE_URL=postgres://localhost/mydb
API_KEY="secret-key-123"
EMPTY_VALUE=
QUOTED='single quoted'
```

Environment-specific .env files

When `RY_ENV` is set, Ry loads environment-specific `.env` files with the following priority:

- `.env.<env>` is loaded first (e.g., `.env.dev` when `RY_ENV=dev`)
- `.env` is loaded second (values already set by `.env.<env>` are not overwritten)
- When `RY_ENV=prod`, no `.env` files are loaded (security)
- When `RY_ENV` is not set, only `.env` is loaded (backward compatible)

See [RY_ENV](#) for details on environment modes.

append

Signature: `append(list: List<T>, value: T)`

Adds an element to the end of a list. This is a mutating operation — the list is modified in place. UFCS notation is also available.

```
xs = [1, 2]
xs.append(3)
print(xs)    # [1, 2, 3]
```

pop

Signature: `pop(list: List<T>) -> Option<T>`

Removes and returns the last element of a list as `Option<T>`. Returns `None` if the list is empty. UFCS notation is also available.

```
xs = [1, 2, 3]
v = xs.pop()
print(v)      # Some(3)
print(xs)     # [1, 2]
```

reverse (list)

Signature: `reverse(list: List<T>) -> List<T>`

Returns a new list with elements in reverse order. The original list is not modified. Also works on strings (see [String Operations](#)). UFCS notation is also available.

```
xs = [1, 2, 3]
ys = reverse(xs)
print(ys)     # [3, 2, 1]
print(xs)     # [1, 2, 3] (unchanged)
```

slice

Signature: `slice(list: List<T>, start: int, end: int) -> List<T>`

Returns a new sub-list from `start` (inclusive) to `end` (exclusive). Indices are clamped to the valid range (0 to `length(list)`). UFCS notation is also available.

```
xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))    # [2, 3]
print(slice(xs, 0, 100))  # [1, 2, 3, 4, 5] (clamped)
```

take

Signature: `take(list: List<T>, count: int) -> List<T>`

Returns a new list with the first `count` elements. If `count` exceeds the list length, returns a copy of the entire list. If `count <= 0`, returns an empty list. The original list is not modified. UFCS notation is also available.

```
xs = [1, 2, 3, 4, 5]
ys = xs.take(3)
print(ys)    # [1, 2, 3]
```

```
print(xs.take(10))    # [1, 2, 3, 4, 5] (clamped)
print(xs.take(0))     # []
```

tap

Signature: `tap(list: List<T>, function: function(T) -> R) -> List<T>`

Calls the given function on each element (ignoring any return value), then returns the original list unchanged. Useful for debugging or inserting side effects in a method chain. UFCS notation is also available.

```
xs = [1, 2, 3]
ys = xs.tap((x: int) => print(x)).map((x: int) => x * 2)
# prints 1, 2, 3, then ys = [2, 4, 6]
```

filter

Signature: `filter(list: List<T>, pred: function(T) -> bool) -> List<T>`

Returns a new list containing only elements for which the predicate returns `true`. The original list is not modified. UFCS notation is also available.

```
xs = [1, 2, 3, 4, 5]
ys = xs.filter((x: int) => x > 3)
print(ys)    # [4, 5]
print(xs)    # [1, 2, 3, 4, 5] (unchanged)
```

map

Signature: `map(list: List<T>, function: function(T) -> U) -> List<U>`

Returns a new list with each element transformed by the given function. The output element type can differ from the input type. The original list is not modified. UFCS notation is also available.

```
xs = [1, 2, 3]
ys = xs.map((x: int) => x * 2)
print(ys)    # [2, 4, 6]
```

sort

Signature: `sort(list: List<T>) -> List<T> / sort(list: List<T>, comp: function(T, T) -> bool) -> List<T>`

Returns a new sorted list. Default is ascending order. An optional comparator function can be provided that returns `true` if the first argument should come before the second. The original list is not modified. The sort is **stable** (equal elements preserve their original order). UFCS notation is also available.

```
xs = [3, 1, 2]
print(xs.sort())    # [1, 2, 3]

# Descending order
desc = xs.sort((a: int, b: int) => a > b)
print(desc)        # [3, 2, 1]
```

sort!

Signature: `sort!(list: List<T>) / sort!(list: List<T>, comp: function(T, T) -> bool)`

Sorts a list in place. Same sorting algorithm as `sort()`, but modifies the original list instead of creating a new one. UFCS notation is also available.

```
xs = [3, 1, 2]
xs.sort!()
print(xs)    # [1, 2, 3]
```

reverse!

Signature: `reverse!(list: List<T>)`

Reverses a list in place. UFCS notation is also available.

```
xs = [1, 2, 3]
xs.reverse!()
print(xs)    # [3, 2, 1]
```

appended

Signature: `appended(list: List<T>, value: T) -> List<T>`

Returns a new list with the element added at the end. The original list is not modified. UFCS notation is also available.

```
xs = [1, 2]
ys = xs.appended(3)
print(xs)    # [1, 2] (unchanged)
print(ys)    # [1, 2, 3]
```

append!

Signature: `append!(list: List<T>, value: T)`

Alias for `append()`. Adds an element to the end of a list in place. Provided for naming consistency with the `!` convention.

first

Signature: `first(list: List<T>) -> Option<T>`

Returns the first element of a list as `Option<T>`. Returns `None` if the list is empty.

```
print(first([10, 20, 30]))    # Some(10)
```

last

Signature: `last(list: List<T>) -> Option<T>`

Returns the last element of a list as `Option<T>`. Returns `None` if the list is empty.

```
print(last([10, 20, 30]))    # Some(30)
```

get (Map)

Signature: `get(map: Map<K, V>, key: K) -> Option<V>` / `get(map: Map<K, V>, key: K, default: V) -> V`

Two-argument form returns the value for key as `Option<V>`. Three-argument form returns the value or the default.

```
m = {"a": 1, "b": 2}
print(get(m, "a"))           # Some(1)
print(get(m, "z"))           # None
print(get(m, "z", 0))        # 0
```

iter

Signature: `iter(collection: List<T> | Set<T>) -> Iterator<T>` / `iter(collection: Map<K, V>) -> Iterator<(K, V)>`

Creates a lazy iterator from a collection. The iterator does not copy data; it references the original collection. UFCS notation is also available.

- For `List<T>` and `Set<T>`, the element type is `T`.
- For `Map<K, V>`, the element type is the tuple `(K, V)`.

```
xs = [1, 2, 3]
it = xs.iter()               # Iterator<int>
ys = it.to_list()            # [1, 2, 3]

m = {"a": 1, "b": 2}
for k, v in m.iter():         # Iterator<(str, int)>
    print(k)
```

next

Signature: `next(iter: Iterator<T>) -> Option<T>`

Returns the next element from the iterator as `Option<T>`. Returns `None` when the iterator is exhausted. The iterator advances its internal state on each call. UFCS notation is also available.

```
it = [10, 20].iter()
print(it.next())             # Some(10)
print(it.next())             # Some(20)
print(it.next())             # None
```

to_list

Signature: `to_list(iter: Iterator<T>) -> List<T>`

Collects all remaining elements from the iterator into a new list. UFCS notation is also available.

```
xs = [1, 2, 3, 4, 5]
ys = xs.iter().filter((x: int) => x > 2).to_list()
print(ys)    # [3, 4, 5]
```

[English](#) | [日本語](#) | [繁體中文](#)

Collection Reference (Tuple, List, Map, Set)

Tuple

Overview

A fixed-length combination of heterogeneous values. Implemented as a stack-allocated value type using LLVM literal StructType.

Syntax

```
t = (42,)                                # single-element tuple (trailing comma required)
t = (1, 3.14)
t: (int, float) = (1, 3.14)
```

Type Annotation

```
single: (int,) = (42,)                  # trailing comma required for single-element
pair: (int, str) = (42, "hello")
triple: (int, float, bool) = (1, 2.0, true)
```

Comparison

Tuples support `==` and `!=` via element-wise comparison.

```
print((1, 2) == (1, 2))    # true
print((1, 2) != (3, 4))    # true
print(("a", 1) == ("b", 1)) # false
```

Element Access

Access elements using numeric indices `.0`, `.1`, etc.

```
t = (10, 3.14)
print(t.0)    # 10
print(t.1)    # 3.14
```

Function Return Values

```
function swap(a: int, b: int) -> (int, int):
    return (b, a)
```

```
result = swap(1, 2)
print(result.0)    # 2
print(result.1)    # 1
```

Constraints and Errors

Constraint	Details
Out-of-range index	Compile error
Comparison operators	Only == and != are supported; <, <=, >, >= are not

List

Overview

A variable-length sequence of elements of the same type. Allocated on the heap.

Syntax

```
xs = [1, 2, 3]
xs: List<int> = [1, 2, 3]
```

Empty List

An empty list requires a type annotation so the element type can be determined:

```
xs: List<int> = []
xs: List<str> = []
```

Concatenation

Lists can be concatenated with + and +=:

```
a = [1, 2, 3]
b = [4, 5, 6]
c = a + b      # [1, 2, 3, 4, 5, 6]
a += [7, 8]    # a is now [1, 2, 3, 7, 8]
```

Both operands must have the same element type.

Supported Element Types

int, float, bool, str

Index Access

```
xs = [1, 2, 3]
print(xs[0])  # 1
print(xs[2])  # 3
```

Negative indices wrap around from the end (Python-style):

```
xs = [10, 20, 30]
print(xs[-1])  # 30 (last element)
print(xs[-2])  # 20
print(xs[-3])  # 10
```

Out-of-bounds access (including negative indices that exceed the list length) raises a runtime error.

Index Assignment

```
xs = [1, 2, 3]
xs[0] = 99
print(xs[0])    # 99
xs[-1] = 42     # assigns to last element
print(xs[2])    # 42
```

length

```
xs = [1, 2, 3]
print(length(xs))    # 3
```

print

```
xs = [1, 2, 3]
print(xs)    # [1, 2, 3]
```

for Iteration

```
xs = [10, 20, 30]
for x in xs:
    print(x)
# 10
# 20
# 30
```

append

Adds an element to the end of the list. This is a mutating operation.

```
xs = [1, 2]
xs.append(3)
print(xs)    # [1, 2, 3]
```

pop

Removes and returns the last element. Returns **None** if the list is empty.

```
xs = [1, 2, 3]
v = xs.pop()
print(v)    # Some(3)
print(xs)    # [1, 2]
```

reverse

Returns a new list with elements in reverse order. The original list is not modified. Also works on strings.

```
xs = [1, 2, 3]
print(reverse(xs))    # [3, 2, 1]
print(xs)    # [1, 2, 3] (unchanged)
```

slice

Returns a new sub-list from **start** (inclusive) to **end** (exclusive). Indices are clamped to the valid range.

```
xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))    # [2, 3]
print(slice(xs, 0, 100))    # [1, 2, 3, 4, 5] (clamped)
```

take

Returns a new list with the first `count` elements. If `count` exceeds the list length, returns a copy of the entire list. If `count` $\leq$ 0, returns an empty list. The original list is not modified.

```
xs = [1, 2, 3, 4, 5]
ys = xs.take(3)
print(ys)      # [1, 2, 3]
print(xs.take(10)) # [1, 2, 3, 4, 5] (clamped)
print(xs.take(0))  # []
```

tap

Calls the given function on each element (ignoring any return value), then returns the original list unchanged. Useful for debugging or inserting side effects in a method chain.

```
xs = [1, 2, 3]
ys = xs.tap((x: int) => print(x)).map((x: int) => x * 2)
# prints 1, 2, 3, then ys = [2, 4, 6]
```

filter

Returns a new list containing only elements that satisfy the predicate. The original list is not modified.

```
xs = [1, 2, 3, 4, 5]
ys = xs.filter((x: int) => x > 3)
print(ys)  # [4, 5]
```

map

Returns a new list with each element transformed by the given function. The output element type can differ from the input. The original list is not modified.

```
xs = [1, 2, 3]
ys = xs.map((x: int) => x * 2)
print(ys)  # [2, 4, 6]
```

sort

Returns a new sorted list. Default is ascending order. A custom comparator can be provided. The original list is not modified. The sort is **stable** (equal elements preserve their original order). Internally uses TimSort.

```
xs = [3, 1, 2]
print(xs.sort())  # [1, 2, 3]

# Descending order with comparator
desc = xs.sort((a: int, b: int) => a > b)
print(desc)  # [3, 2, 1]
```

Chaining filter, map, sort

These functions return new lists, so they can be chained via UFCS.

```
xs = [5, 3, 1, 4, 2]
result = xs.filter((x: int) => x > 1).map((x: int) => x * 10).sort()
print(result)  # [20, 30, 40, 50]
```

reduce

Reduces a list to a single value using an accumulator function, starting with the first element.

```
xs = [1, 2, 3, 4, 5]
total = reduce(xs, (a: int, b: int) => a + b)
print(total)    # 15
```

fold

Folds a list to a single value using an accumulator function and an explicit initial value.

```
xs = [1, 2, 3, 4, 5]
total = fold(xs, 0, (a: int, b: int) => a + b)
print(total)    # 15
```

any

Returns **true** if at least one element satisfies the predicate.

```
xs = [1, 2, 3, 4, 5]
print(any(xs, (x: int) => x > 4))    # true
print(any(xs, (x: int) => x > 9))    # false
```

all

Returns **true** if every element satisfies the predicate.

```
xs = [2, 4, 6]
print(all(xs, (x: int) => x > 0))    # true
print(all(xs, (x: int) => x > 3))    # false
```

sum

Returns the sum of all elements.

```
xs = [1, 2, 3, 4, 5]
print(sum(xs))    # 15
```

min

Returns the minimum element.

```
xs = [3, 1, 4, 1, 5]
print(min(xs))    # 1
```

max

Returns the maximum element.

```
xs = [3, 1, 4, 1, 5]
print(max(xs))    # 5
```

first

Returns the first element. Returns **None** if the list is empty.

```
xs = [10, 20, 30]
print(first(xs))    # Some(10)
```

last

Returns the last element. Returns `None` if the list is empty.

```
xs = [10, 20, 30]
print(last(xs))    # Some(30)
```

is_empty

Returns `true` if the list has no elements.

```
xs = [1, 2, 3]
print(is_empty(xs))    # false
print(is_empty([]))    # true (requires type annotation in practice)
```

enumerate

Returns a list of (index, element) tuples.

```
xs = [10, 20, 30]
pairs = enumerate(xs)
# pairs = [(0, 10), (1, 20), (2, 30)]

# Tuple destructuring in for loop
for i, x in enumerate(xs):
    print(f"{i}: {x}")    # 0: 10, 1: 20, 2: 30
```

zip

Combines two lists into a list of (elem1, elem2) tuples. The result length equals the shorter list.

```
xs = [1, 2, 3]
ys = ["a", "b", "c"]
pairs = zip(xs, ys)
# pairs = [(1, "a"), (2, "b"), (3, "c")]

# Tuple destructuring in for loop
for a, b in zip(xs, ys):
    print(f"{a}: {b}")    # 1: a, 2: b, 3: c
```

insert

Inserts an element at the specified index. Elements at and after the index are shifted right.

```
xs = [1, 2, 3]
insert(xs, 1, 99)
print(xs)    # [1, 99, 2, 3]
```

remove_at

Removes and returns the element at the specified index. Elements after the index are shifted left.

```
xs = [1, 2, 3, 4]
v = remove_at(xs, 1)
print(v)    # 2
print(xs)    # [1, 3, 4]
```

remove

Removes the first occurrence of the specified value from the list. Does nothing if the value is not found. This is a mutating operation.

```
xs = [1, 2, 3, 2, 4]
remove(xs, 2)
print(xs)    # [1, 3, 2, 4]
```

distinct

Returns a new list with duplicate elements removed. The original order is preserved (first occurrence kept). The original list is not modified.

```
xs = [1, 2, 3, 2, 1, 4]
print(distinct(xs))  # [1, 2, 3, 4]
print(xs)            # [1, 2, 3, 2, 1, 4] (unchanged)
```

flatten

Flattens a nested list (list of lists) by one level. Returns a new list. The original list is not modified.

```
xs = [[1, 2], [3, 4]]
print(flatten(xs))  # [1, 2, 3, 4]
print(xs)           # [[1, 2], [3, 4]] (unchanged)
```

Operation Complexity

Operation	Complexity
xs[i] index access	O(1)
append / append!	O(1) amortized
pop	O(1)
first, last	O(1)
insert, remove_at	O(n)
sort / sort!	O(n log n)
take	O(n)
tap	O(n)
filter, map, reduce, fold	O(n)
reverse / reverse!	O(n)
distinct	O(n)
length	O(1)

Constraints and Errors

Constraint	Details
All elements must be the same type	Mixed types cause a compile error
Empty list []	Requires type annotation (e.g., <code>xs: List<int> = []</code>)
Out-of-range access	Runtime error (<code>exit(1)</code>)

Copy-on-Write (CoW) Semantics

All collection types (List, Map, Set) use **Copy-on-Write** semantics when managed by ARC. This means:

- **Assignment shares data:** `b = a` does not copy the collection — both variables reference the same data. The reference count is incremented.
- **Mutation triggers a copy:** When a shared collection is mutated (e.g., `append`, `remove`, index assignment), a deep copy is automatically created before the mutation. Only the mutator pays the cost.
- **Unique owners mutate in-place:** When a collection has only one reference (`strong_count == 1`), mutations are performed in-place with zero copy overhead.

```
a = [1, 2, 3]           # strong_count = 1
b = a                   # strong_count = 2 (shared)
append(b, 4)            # strong_count > 1 → deep copy b, then mutate
                        # a = [1, 2, 3] (strong_count = 1)
                        # b = [1, 2, 3, 4] (strong_count = 1, new allocation)

c = [10, 20]            # strong_count = 1
append(c, 30)           # strong_count == 1 → mutate in-place (no copy)
```

Operations that trigger CoW

Type	Mutating operations
List	<code>append()</code> , <code>pop()</code> , <code>insert()</code> , <code>remove()</code> , <code>remove_at()</code> , <code>sort!()</code> , <code>reverse!()</code> , index assignment (<code>xs[i] = val</code>)
Map	<code>remove()</code> , index assignment (<code>m[key] = val</code>)
Set	<code>add()</code> , <code>remove()</code>

Non-mutating operations (`appended`, `slice`, `take`, `filter`, `map`, `sort`, `reverse`, `get`, `items`, etc.) create new collections and do not trigger CoW.

Map

Overview

A key-value mapping. Allocated on the heap.

Syntax

```
m = {"a": 1, "b": 2}
m: Map<str, int> = {"a": 1, "b": 2}
```

Key Access

```
m = {"a": 1, "b": 2}
print(m["a"])    # 1
```

Insert and Update

```
m = {"a": 1}
m["b"] = 2      # Insert new entry
m["a"] = 99     # Update existing entry
```


length

```
m = {"a": 1, "b": 2, "c": 3}
print(length(m))    # 3
```

print

```
m = {"a": 1, "b": 2}
print(m)            # {a: 1, b: 2}
```

has_key

```
m = {"a": 1, "b": 2}
print(m.has_key("a"))    # true
print(m.has_key("z"))    # false
```

keys

Returns a list of all keys in the map.

```
m = {"a": 1, "b": 2, "c": 3}
print(keys(m))          # ["a", "b", "c"]
```

values

Returns a list of all values in the map.

```
m = {"a": 1, "b": 2, "c": 3}
print(values(m))        # [1, 2, 3]
```

items

Returns a list of (key, value) tuples for all entries in the map.

```
m = {"a": 1, "b": 2}
pairs = items(m)
# pairs = [("a", 1), ("b", 2)]
```

remove (Map)

Removes the entry with the specified key from the map. Does nothing if the key does not exist.

```
m = {"a": 1, "b": 2}
remove(m, "a")
print(m)            # {b: 2}
```

get

Returns the value for the specified key, or a default value if the key does not exist.

```
m = {"a": 1, "b": 2}
print(get(m, "a", 0))    # 1
print(get(m, "z", 0))    # 0
```

merge

Returns a new map that combines all entries from both maps. When keys overlap, values from the second map take precedence. The original maps are not modified.

```

m1 = {"a": 1, "b": 2}
m2 = {"b": 99, "c": 3}
m3 = merge(m1, m2)
print(m3["a"])    # 1
print(m3["b"])    # 99
print(m3["c"])    # 3

```

Constraints and Errors

Constraint	Details
All keys must be the same type	Mixed key types cause a compile error
All values must be the same type	Mixed value types cause a compile error
Empty map	Type annotation is required (e.g., <code>m: Map<str, int> = {"a": 1}</code>)
Accessing a non-existent key	Runtime error (<code>exit(1)</code>)
Key lookup	Hash table ($O(1)$ average)
Capacity overflow	Automatically doubles in size

Set

Overview

A collection that holds elements of the same type without duplicates. Allocated on the heap.

Syntax

```

s = {1, 2, 3}
s: Set<int> = {1, 2, 3}

```

Supported Element Types

int, float, bool, str

in Operator (Membership Check)

```

s = {1, 2, 3}
print(2 in s)    # true
print(5 in s)    # false

```

length

```

s = {1, 2, 3}
print(length(s)) # 3

```

print

```

s = {1, 2, 3}
print(s)        # {1, 2, 3}

```

add (Add Element)

Duplicate elements are ignored when added.

```
s = {1, 2, 3}
s.add(4)           # Add
s.add(1)           # Ignored because it already exists
print(length(s))   # 4
```

remove (Remove Element)

```
s = {1, 2, 3}
s.remove(2)
print(2 in s)      # false
```

for Iteration

```
s = {10, 20, 30}
for x in s:
    print(x)
```

Empty Set

An empty set requires a type annotation.

```
s: Set<int> = {}
```

Function Parameters

```
function has_value(s: Set<int>, v: int) -> bool:
    return v in s
```

union

Returns a new set containing all elements from both sets.

```
a = {1, 2, 3}
b = {3, 4, 5}
print(union(a, b))    # {1, 2, 3, 4, 5}
```

intersection

Returns a new set containing only elements present in both sets.

```
a = {1, 2, 3}
b = {2, 3, 4}
print(intersection(a, b)) # {2, 3}
```

difference

Returns a new set containing elements in the first set but not in the second.

```
a = {1, 2, 3}
b = {2, 3, 4}
print(difference(a, b))   # {1}
```

symmetric_difference

Returns a new set containing elements that are in either set but not in both.

```
a = {1, 2, 3}
b = {2, 3, 4}
print(symmetric_difference(a, b)) # {1, 4}
```

is_subset

Returns **true** if all elements of the first set are contained in the second set.

```
a = {1, 2}
b = {1, 2, 3}
print(is_subset(a, b))    # true
print(is_subset(b, a))    # false
```

is_superset

Returns **true** if the first set contains all elements of the second set.

```
a = {1, 2, 3}
b = {1, 2}
print(is_superset(a, b))  # true
print(is_superset(b, a))  # false
```

Constraints and Errors

Constraint	Details
All elements must be the same type	Mixed types cause a compile error
Empty set {}	Type annotation is required
Element lookup	Hash table (O(1) average)
Capacity overflow	Automatically doubles in size

Iterator

Overview

A lazy iterator abstraction that enables efficient data transformation pipelines. Iterators do not copy or materialize intermediate results —each element is processed on demand.

Creating Iterators

Use `iter()` to create an iterator from any collection:

```
xs = [1, 2, 3, 4, 5]
it = xs.iter()           # Iterator<int>

s = {10, 20, 30}
sit = s.iter()           # Iterator<int>

m = {"a": 1, "b": 2}
mit = m.iter()           # Iterator<(str, int)>
```

Lazy Method Chaining

Iterator methods return new iterators, forming a pipeline that is only evaluated when consumed:

Method	Description
<code>.filter(function)</code>	Yields only elements where the predicate returns true
<code>.map(function)</code>	Transforms each element using the given function
<code>.take(count)</code>	Yields at most count elements

```

result = [1, 2, 3, 4, 5]
    .iter()
    .filter((x: int) => x > 2)
    .map((x: int) => x * 2)
    .take(2)
    .to_list()    # [6, 8]

```

Consuming Iterators

Method	Description
<code>.to_list()</code>	Collects all elements into a <code>List<T></code>
<code>.next()</code>	Returns the next element as <code>Option<T></code>

```

it = [10, 20].iter()
print(it.next())    # Some(10)
print(it.next())    # Some(20)
print(it.next())    # None

```

For Loop Support

Iterators can be used directly in for loops:

```

for x in [1, 2, 3].iter().filter((x: int) => x > 1):
    print(x)
# 2
# 3

```

```

for k, v in {"a": 1, "b": 2}.iter():
    print(k)

```

[English](#) | [日本語](#) | [繁體中文](#)

Design by Contract (DbC)

Ry supports Eiffel-style Design by Contract with preconditions (**require**), postconditions (**ensure**), and record invariants (**invariant**). Contract violations terminate the process with `exit(1)`.

Preconditions (**require**)

Preconditions are checked at function entry. They specify what must be true for the function to be called correctly.

```

function deposit(amount: int, balance: int) -> int:
    require:
        amount > 0
        balance >= 0
    new_balance: int = balance + amount
    return new_balance

```

If any precondition fails, the program terminates with:

Contract violation: require failed in deposit()

Postconditions (ensure)

Postconditions are checked before every `return`. They specify what the function guarantees about its result.

Variable binding

`ensure` requires a variable name that binds the return value. This variable can be used in the postcondition expressions.

```
function abs(x: int) -> int:
  ensure v:
    v >= 0
  if x < 0:
    return -x
  return x
```

Since function arguments are immutable in Ry, you can reference them directly in `ensure` blocks to compare with entry values:

```
function increment(x: int) -> int:
  ensure v:
    v == x + 1
  return x + 1
```

Tuple destructuring

For functions that return tuples, multiple variable names can be specified, separated by commas:

```
function divide(a: int, b: int) -> (int, int):
  ensure q, r:
    q >= 0
    r >= 0
  return (a // b, a % b)
```

The number of binding variables must match the number of tuple elements.

Combined Example

```
function deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  ensure v:
    v >= 0
    v == balance + amount
  new_balance: int = balance + amount
  return new_balance
```

Record Invariants (invariant)

Invariants are conditions that must always hold for a record instance. They are checked: - After construction
- After every field assignment

```
record BankAccount:
  balance: int
```

```

    min_balance: int
    invariant:
        balance >= min_balance

a = BankAccount(100, 0)    # OK: 100 >= 0
a.balance = -1             # Contract violation: invariant failed

```

Rules

- `require` and `ensure` blocks are optional and appear before the function body.
- `require` must come before `ensure` when both are present.
- `ensure` requires a variable binding (e.g., `ensure v:`) to name the return value.
- For tuple returns, multiple bindings can be specified (e.g., `ensure q, r:`).
- `invariant` appears at the end of a `record` definition, after all field declarations.
- All contract violations terminate with `exit(1)` and print a diagnostic message.

[English](#) | [日本語](#) | [繁體中文](#)

Control Flow Reference

if / else

Syntax

```

if condition:
    # then block
else:
    # else block (optional)

```

Condition Types

Type	Falsy Value	Truthy Value
bool	false	true
int	0	non-zero

float and `str` cannot be used directly as conditions.

Example

```

x = 10

if x > 5:
    print("big")
else:
    print("small or equal")

```

Scope Rules

- Each `if` / `else` block has its own independent block scope.
- Variables declared inside a block are not accessible outside the block.

```

if true:
    y = 42
# y is not accessible here

```

while

Syntax

```
while condition:
    # loop body
```

Repeats the loop body while the condition is `true`.

Example

```
i = 0
while i < 5:
    print(i)
    i += 1
```

Combining with break / continue

```
i = 0
while true:
    if i >= 3:
        break
    i += 1
```

for

Syntax

```
# List / set iteration
for x in iterable_expr:
    # x is assigned each element

# range (starting from 0)
for i in range(n):
    # i = 0, 1, ..., n-1

# range (with start and end)
for i in range(start, end):
    # i = start, start+1, ..., end-1

# range (with step)
for i in range(start, end, step):
    # i = start, start+step, start+2*step, ...
```

Map Key-Value Iteration

```
for k, v in map_expr:
    # k is the key, v is the value for each entry
```

Tuple Destructuring

When iterating over a list of tuples, you can destructure into N variables matching the tuple's element count. Use `_` to discard a value.


```

xs = [10, 20, 30]

for i, x in enumerate(xs):
    print(f"{i}: {x}")      # 0: 10, 1: 20, 2: 30

for a, b in zip([1, 2], [10, 20]):
    print(a + b)            # 11, 22

for _, x in enumerate(xs):
    print(x)                # index discarded

# N-element destructuring (3+ variables)
triples = [(1, 2, 3), (4, 5, 6)]
for a, b, c in triples:
    print(a + b + c)        # 6, 15

for a, _, c in triples:
    print(a + c)            # 4, 10 (middle element discarded)

```

Range Operator (..)

The .. operator creates an inclusive integer range. 1 .. 5 produces [1, 2, 3, 4, 5].

```

for i in 1 .. 5:
    print(i)                # 1 2 3 4 5

```

Example

```

xs = [10, 20, 30]
for x in xs:
    print(x)

s = {1, 2, 3}
for x in s:
    print(x)

for i in range(5):
    print(i)                # 0 1 2 3 4

for i in range(2, 6):
    print(i)                # 2 3 4 5

for i in range(0, 10, 2):
    print(i)                # 0 2 4 6 8

for i in range(10, 0, -3):
    print(i)                # 10 7 4 1

# Map iteration
m = {"a": 1, "b": 2}
for k, v in m:
    print(k)
    print(v)

# Range operator

```

```
for i in 1 .. 3:
    print(i)      # 1 2 3
```

async / await

`async` function declares a function that runs concurrently. Calling an `async` function returns `Task<T>`. Use `await` inside another `async` function or `block_on()` from synchronous context to wait for the result.

```
async function add(a: int, b: int) -> int:
    return a + b
```

```
# From synchronous context, use block_on()
t: Task<int> = add(20, 22)
print(block_on(t))           # 42
print(block_on(add(1, 2)))   # 3
```

```
# Inside async function, use await
async function double_add(a: int, b: int) -> int:
    result = await add(a, b)
    return result * 2
```

Rules

- `async function name(...) -> T`: is declared with the awaited result type `T`.
 - Calling an `async` function immediately returns `Task<T>`.
 - `await expr` requires `expr` to be `Task<T>` and produces `T`.
 - `await` can only be used inside an `async` function. Use `block_on(task)` from synchronous context.
 - `block_on(task)` blocks the current thread until the task completes and returns the result.
 - `async function ... -> Unit` is supported; `block_on(task)` is the primary way to wait when no value is produced.
 - Tasks run on the runtime worker pool; they are not implemented as one OS thread per task.
 - `async` lambdas and `async @native function` are not supported in v1.
-

@parallel for

`@parallel` can be attached only to counted `for` loops over `range(...)` or integer `..` ranges. The loop body runs in parallel chunks on the runtime worker pool.

```
@parallel
for i in range(8):
    print(i)
```

Constraints

- Only `range(...)` and integer `..` loops are supported.
- Destructuring iteration is not supported.
- Assigning to outer mutable bindings is rejected.
- `break` and `continue` are rejected.
- Indexed assignment and field assignment inside the loop body are rejected in v1.

Use `available_parallelism()` to inspect the runtime worker count.

break

- Immediately exits the innermost loop (**while** or **for**).
- Using it outside a loop causes a compile error.

```
for i in range(10):
    if i == 5:
        break      # Exits when i == 5
    print(i)        # 0 1 2 3 4
```

Error Example

```
# break outside a loop is a compile error
break      # Error: break outside loop
```

continue

- Ends the current iteration of the innermost loop and skips to the next iteration.
- Using it outside a loop causes a compile error.

```
for i in range(5):
    if i == 2:
        continue   # Skip i == 2
    print(i)        # 0 1 3 4
```

... (Ellipsis)

- A no-op statement that does nothing. Used as a placeholder for empty blocks.
- Can be used in any block: function body, **if/else**, **while**, **for**, **match** arm, etc.

```
function not_yet():
    ...

if true:
    ...
else:
    ...
```

when

when: provides multi-branch conditional flow without a subject value.

Syntax

```
when:
    condition:
        # body
    condition:
        # body
    else:
        # fallback body
```

Example

```
x = 0
```

```
when:
  x > 0:
    print("positive")
  x < 0:
    print("negative")
  else:
    print("zero")
```

The conditional **when:** statement evaluates arms from top to bottom and executes only the first arm whose condition is truthy. The **else:** arm is optional for statements.

For the expression form of **when:**, see [Operator Reference](#).

match

Syntax

```
match expression:
  case pattern:
    # body
  case pattern if guard_condition:
    # guarded body
  case _:
    # wildcard (matches anything)
```

Pattern Types

Pattern	Example	Description
Wildcard	<code>_</code>	Matches anything
Literal	<code>0, "hello", true</code>	Equality comparison
Variable binding	<code>n</code>	Matches anything and binds to a variable
enum variant	<code>Color::Red</code>	Compares enum tag (simple enum)
ADT enum variant	<code>Shape::Circle(r)</code>	Matches an enum variant with associated data and binds it
<code>Some(x)</code>	<code>Some(v)</code>	When Option has a value, binds the inner value
<code>None</code>	<code>None</code>	When Option has no value
<code>Ok(x)</code>	<code>Ok(v)</code>	When Result is Ok, binds the inner value
<code>Err(x)</code>	<code>Err(e)</code>	When Result is Err, binds the error value
OR pattern	<code>1 \ 2 \ 3</code>	Matches if any alternative matches

Guard Clause

A guard condition can be specified in the form `case pattern if condition:`. The arm is executed only when the pattern matches and the guard condition is true.

OR Pattern

Multiple patterns can be combined with `|` to match any of them. Variable bindings (`n`, `Some(x)`, `Ok(v)`, `Err(e)`) are not allowed in OR patterns.

```
match x:
    case 1 | 2 | 3:
        print("small")
    case _:
        print("other")

# Enum OR pattern
match color:
    case Color::Red | Color::Blue:
        print("warm or cool")
    case Color::Green:
        print("green")
```

Exhaustiveness Checking

- enum types: Must cover all variants or include `_`. OR patterns count each alternative individually.
- Option types: Must cover both `Some` and `None` or include `_`.
- bool type: Must cover both `true` and `false` or include `_`.
- int / float / str literals: `_` is required.
- Guarded arms do not count toward exhaustiveness.

Example

```
# enum pattern match
enum Color:
    Red
    Green
    Blue

match color:
    case Color::Red:
        print("red")
    case Color::Green:
        print("green")
    case Color::Blue:
        print("blue")

# Option pattern match
x: Option<int> = Some(42)
match x:
    case Some(v):
        print(v)
    case None:
        print("nothing")

# Result pattern match
```

```

function divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:
        return Err(Error("division by zero"))
    return Ok(a // b)

match divide(10, 2):
    case Ok(v):
        print(v)           # 5
    case Err(e):
        print(e.message)

# Literal pattern match
match x:
    case 0:
        print("zero")
    case 1:
        print("one")
    case _:
        print("other")

# Guard clause
match x:
    case n if n > 0:
        print("positive")
    case n if n < 0:
        print("negative")
    case _:
        print("zero")

```

ADT Enum Pattern Matching

When an enum variant carries associated data, use a binding pattern to extract the value(s).

```

enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point

s = Shape::Circle(3.14)
match s:
    case Shape::Circle(r):
        print(r)           # 3.14
    case Shape::Rectangle(w, h):
        print(w)
        print(h)
    case Shape::Point:
        print("point")

```

Multi-field variants bind each field to a separate name in declaration order.

Match Expressions

`match` can be used as an expression by replacing `:` with `=>` in each arm. Each arm provides a single expression whose value becomes the result.

Syntax

```

result = match expression:
    case pattern => value_expression
    case pattern if guard => value_expression
    case _ => default_value

```

All patterns supported in match statements are also supported in match expressions: literals, variable bindings, enums, ADT enums, `Some/None`, `Ok/Err`, OR patterns, guards, and wildcards.

Match expressions must be exhaustive (same rules as match statements).

Examples

```

# Option
value = match opt:
    case Some(v) => v
    case None    => 0

# Enum
label = match direction:
    case Direction::North => "N"
    case Direction::South => "S"
    case Direction::East  => "E"
    case Direction::West  => "W"

# Guard
grade = match score:
    case n if n >= 90 => "A"
    case n if n >= 80 => "B"
    case _             => "F"

# OR pattern
kind = match x:
    case 1 | 2 | 3 => "small"
    case _         => "large"

# ADT enum
area = match shape:
    case Shape::Circle(r)  => 3.14 * r * r
    case Shape::Rect(w, h) => w * h
    case Shape::Point      => 0.0

```

Scope Rules

- Each `case` arm has its own block scope.
- Variables bound by variable binding patterns (`n`), `Some(x)`, `Ok(v)`, or `Err(e)` are only valid within that arm.

Scope Rules

Block Scope

- Each block of `if` / `else` / `while` / `for` / `when` has a block scope.
- Variables declared inside a block go out of scope when the block ends.

```

for i in range(3):
    tmp = i * 2
# tmp is not accessible here

if true:
    a = 1
# a is not accessible here

```

Inner Scope Reassignment

- Assigning to a variable inside an inner scope modifies the outer variable (Python-style scoping).
- There is no shadowing — the inner assignment changes the same variable.

```

x = 10
if true:
    x = 99 # Modifies the outer x
    print(x) # 99
print(x) # 99

```

[English](#) | [日本語](#) | [繁體中文](#)

Directives

Directives are compile-time metadata annotations that can be attached to declarations. They use the `@name` syntax, similar to Java annotations.

Syntax

```

@name
@name(key=value, ...)

```

Directives are placed before the target declaration. Multiple directives can be stacked.

Supported Targets

Directives can be applied to the following declarations:

- **function** - Function definitions
- **record** - Struct definitions
- Variable declarations (with or without `@const`)
- Fields within a **record** definition
- **for** - Counted loops only for `@parallel`
- **it** - Test case definitions (for `@each` and `@property` only)

Built-in Directives

`@deprecated`

Marks a declaration as deprecated. When a deprecated entity is used (called, referenced, or accessed), a compile-time warning is emitted.

On functions:

```

@deprecated
function old_function() -> int:
    return 42

print(old_function()) # warning: 'old_function' is deprecated

```


On types:

```
@deprecated
record OldPoint:
  x: int
  y: int
```

```
@const
p = OldPoint(1, 2) # warning: 'OldPoint' is deprecated
```

On variables:

```
@deprecated
@const
old_value = 99
```

```
print(old_value) # warning: 'old_value' is deprecated
```

On fields:

```
record Config:
  @deprecated
  old_setting: int
  new_setting: int
```

```
@const
c = Config(1, 2)
print(c.old_setting) # warning: 'Config.old_setting' is deprecated
print(c.new_setting) # no warning
```

@const

Marks a variable as immutable. Variables declared with `@const` cannot be reassigned after initialization. Without `@const`, variables are mutable by default.

```
@const
x = 42
# x = 10 # Error: cannot reassign @const variable
```

With type annotation:

```
@const
name: str = "hello"
```

Tuple destructuring:

```
@const
a, b = (1, 2)
```

@native

Declares a function whose implementation is provided by the runtime (built-in). The function must not have a body.

Basic syntax:

```
@native
function contains(string: str, substring: str) -> bool

print(contains("hello world", "world")) # true
```

Operator overloads:

```
@native
function operator+(a: str, b: str) -> str

print("hello" + " world")  # hello world
```

UFCS-compatible:

```
@native
function to_upper(string: str) -> str

print("hello".to_upper())  # HELLO
```

Argument count validation:

When a `@native` declaration includes a type signature, the compiler validates the number of arguments at call sites. Overloaded functions (e.g., `range` with 1, 2, or 3 arguments) are supported — any matching overload passes validation.

```
@native
function range(n: int) -> List<int>
@native
function range(start: int, end: int) -> List<int>

print(length(range(5)))      # OK: matches 1-arg overload
print(length(range(1, 10)))  # OK: matches 2-arg overload
print(length(range()))       # Error: expects 1 or 2 argument(s), but got 0
```

Standard library declarations (`core/`):

The `core/` directory contains `@native` declarations for all built-in functions, organized by category:

File	Contents
<code>core/builtins.ry</code>	<code>print</code> , <code>length</code> , <code>range</code> , <code>enumerate</code> , <code>zip</code> , <code>exit</code> , <code>args</code> , <code>available_parallelism</code> , <code>sleep</code>
<code>core/str.ry</code>	<code>contains</code> , <code>starts_with</code> , <code>ends_with</code> , <code>find</code> , <code>substring</code> , <code>char_at</code> , <code>replace</code> , <code>to_upper</code> , <code>to_lower</code> , <code>trim</code> , <code>trim_start</code> , <code>trim_end</code> , <code>repeat</code> , <code>reverse</code> , <code>split</code> , <code>join</code>
<code>core/convert.ry</code>	<code>to_int</code> , <code>to_float</code> , <code>to_str</code>
<code>core/list.ry</code>	<code>append</code> , <code>pop</code> , <code>insert</code> , <code>remove_at</code> , <code>slice</code> , <code>distinct</code> , <code>flatten</code> , <code>sort</code> , <code>first</code> , <code>last</code> , <code>is_empty</code>
<code>core/map.ry</code>	<code>keys</code> , <code>values</code> , <code>items</code> , <code>has_key</code> , <code>get</code> , <code>merge</code>
<code>core/set.ry</code>	<code>add</code> , <code>remove</code> , <code>union</code> , <code>intersection</code> , <code>difference</code> , <code>symmetric_difference</code> , <code>is_subset</code> , <code>is_superset</code>
<code>core/higher_order.ry</code>	<code>filter</code> , <code>map</code> , <code>reduce</code> , <code>fold</code> , <code>any</code> , <code>all</code> , <code>sum</code> , <code>min</code> , <code>max</code>

These files are automatically loaded as a prelude when the `core/` directory is found relative to the `ry` executable. The prelude enables argument count validation for built-in function calls.

Constraints: - `@native` functions must not have a body (no `:` after the signature). - Providing a body causes a parse error: `@native function must not have a body`. - The declared function must correspond to an existing built-in; otherwise the call will fail at compile time.

Future extensions: - `@native("libfoo.so")` — FFI binding to external shared libraries.

@parallel

Marks a counted **for** loop for parallel execution.

```
@parallel
for i in range(8):
    work(i)
```

Supported target:

- **for** statements only

Constraints:

- Only a single **@parallel** directive is allowed on a **for** statement.
- The iterable must be **range(...)** or an integer **.. range**.
- Destructuring iteration is not supported.
- Assigning to outer mutable variables is rejected.
- **break**, **continue**, indexed assignment, and field assignment inside the loop body are rejected in v1.

@each

Enables parameterized testing by running an **it** block multiple times with different parameters.

Syntax:

```
@each([(arg1, arg2, ...), ...])
it("description with {0} and {1}", (param1: type, param2: type):
    # test body
)
```

Supported target: it calls only

Constraints: - The argument must be a list of tuples - Tuple arity must match the lambda parameter count
- Placeholders {0}, {1}, ... in the description string are replaced with stringified values

@property

Enables property-based testing by generating random inputs for an **it** block.

Syntax:

```
@property(count=100)
it("property name", (a: int, b: int):
    # test body with random values
)
```

Supported target: it calls only

Parameters:

Parameter	Type	Default	Description
count	int	100	Number of random trials

Supported parameter types:

Type	Range
int	-1000 to 1000
float	-1000.0 to 1000.0
bool	true or false

Type	Range
str	Random ASCII, 0-20 characters

On failure, the counterexample (parameter values that caused the failure) is printed.

@inline

Provides inlining hints to the LLVM optimizer. By default, marks the function for aggressive inlining.

Basic usage (always inline):

```
@inline
function add(a: int, b: int) -> int:
    return a + b
```

With mode parameter:

```
@inline(mode="always")
function hot_path(x: int) -> int:
    return x * 2 + 1
```

```
@inline(mode="hint")
function medium_path(x: int) -> int:
    return x + 1
```

```
@inline(mode="never")
function cold_error_handler(msg: str):
    print("ERROR: " + msg)
```

Modes:

Mode	LLVM Attribute	Description
always (default)	AlwaysInline	Always inline this function
hint	InlineHint	Suggest inlining to the optimizer
never	NoInline	Never inline this function

Constraints: - @inline cannot be used with @native (native functions have no body to inline). - An unknown mode value causes a compile error.

Parameters (future extension)

Directives support an optional parameter syntax for future extensions:

```
@deprecated(reason="use new_api instead")
function old_api() -> int:
    return 0
```

Currently, parameters are parsed but not used by the @deprecated directive.

Notes

- Deprecated entities still function normally; only a warning is emitted.
- Warnings are emitted at the point of use, not at the definition.
- Defining a deprecated entity without using it produces no warnings.
- Unknown directive names cause a parse error.

- Directives on unsupported targets (e.g., `if`, `while`) cause a parse error. `@parallel` is the only directive supported on `for`.

[English](#) | [日本語](#) | [繁體中文](#)

Error Reference

Error Format

ry displays compile errors in a Rust-inspired rich format that shows the exact location of the error with source context:

```
error: cannot reassign @const variable: x
--> main.ry:5:1
|
5 | x = 10
|   ^ cannot reassign @const variable: x
```

Each error message includes: - **Error level and message** (error: ...) - **File location** (--> file:line:col) - **Source line** with the relevant code - **Caret indicator** (^) pointing to the exact column

Compile Errors

Error	Cause	Example
Use of undeclared variable	Referenced a variable that has not been declared	<code>print(x)</code> (x is undeclared)
Reassignment to @const variable	Reassigned to a variable declared with <code>@const</code>	<code>@const x = 1 -> x = 2</code>
Redeclaration of same-named variable	Redeclared a variable with the same name in the same scope	<code>x = 1 -> another declaration of x</code>
Type-changing reassignment	Assigned a value of a different type to a variable	<code>x = 1 -> x = 3.14</code>
Type annotation mismatch	Declared type and assigned value type differ	<code>x: int = 3.14</code>
Overload return type conflict	Defined overloads with the same parameter types but different return types only	Two functions with parameters (<code>int</code> , <code>int</code>) returning <code>int</code> and <code>float</code>
Overload resolution failure	No overload matches the argument types	<code>function add(a: int, b: int) called with add(1.0, 2.0)</code>
Float in bitwise operation	Passed <code>float</code> type to <code>&</code> , <code>\ </code> , <code>^</code> , <code>~</code> , <code><<</code> , <code>>></code>	<code>3.14 & 1</code>
Empty list	Cannot infer type from empty list literal <code>[]</code>	<code>xs = []</code>
Empty map	Cannot infer type from empty map literal <code>{}</code>	<code>m = {}</code>
Tuple out-of-range index	Accessed a non-existent index on a tuple	<code>t = (1, 2) -> t.2</code>
break/continue outside loop	Used <code>break</code> or <code>continue</code> outside a <code>for/while</code> loop	<code>break</code> at function top level

Error	Cause	Example
Module import inside block	Used from statement inside a function or conditional block	from math inside a function
Circular import	Modules import each other	a.ry imports b.ry and b.ry imports a.ry
Duplicate field name	Defined the same field name twice in a struct	Defining x twice in type T: x: int
Non-exhaustive match	match does not cover all patterns	Some enum variants uncovered, missing None for Option , missing Ok/Err for Result , no _ for literals
? on non-Result type	Applied ? to an expression that is not a Result type	x = 42 -> x?
? in non-Result function	Used ? in a function that does not return Result	function foo() -> int: -> bar()?

Compile Error Examples

```

# Reassignment to @const variable
@const
x = 10
x = 20    # Error

# Type-changing reassignment
n = 1
n = "hello"    # Error: assigning str to int variable

# Empty list
xs = []    # Error: type cannot be inferred

# break outside loop
break    # Error: outside loop

# Import inside block
function foo():
    from math    # Error: top level only

# Duplicate field name
record Bad:
    x: int
    x: float    # Error: x is duplicated

```

Runtime Errors

Error	Cause	Example
List out-of-range access	List index exceeds bounds	xs = [1, 2, 3] -> xs[5]
Map non-existent key access	Referenced a key that does not exist in the map	m = {"a": 1} -> m["b"]

Error	Cause	Example
Contract violation	A <code>require</code> , <code>ensure</code> , or <code>invariant</code> condition evaluated to false	See Design by Contract

All runtime errors terminate the process with `exit(1)`.

Runtime Error Examples

```
# List out-of-range access
xs = [1, 2, 3]
print(xs[10])    # Runtime error: exit(1)
```

```
# Map non-existent key access
m = {"a": 1}
print(m["z"])    # Runtime error: exit(1)
```

[English](#) | [日本語](#) | [繁體中文](#)

Filesystem Function Reference

File and directory manipulation. All functions require explicit import from `filesystem`.

The `filesystem` package handles operations on files and directories themselves (copy, move, remove, etc.), while the `io` package handles reading and writing file contents.

```
from filesystem import list_dir, walk, glob_files, copy, move, remove, remove_all
from filesystem import make_dir, make_dir_all, file_size, is_file, is_dir, is_symlink
from filesystem import chmod, symlink, read_link
```

Function List

Function	Signature	Description
<code>list_dir</code>	<code>(str) -> Result<List<str>, Error></code>	Lists entries in a directory (non-recursive)
<code>walk</code>	<code>(str) -> Result<List<str>, Error></code>	Recursively lists all files and directories
<code>glob_files</code>	<code>(str) -> Result<List<str>, Error></code>	Finds files matching a glob pattern
<code>copy</code>	<code>(str, str) -> Result<Unit, Error></code>	Copies a file
<code>move</code>	<code>(str, str) -> Result<Unit, Error></code>	Moves or renames a file or directory
<code>remove</code>	<code>(str) -> Result<Unit, Error></code>	Removes a file or empty directory
<code>remove_all</code>	<code>(str) -> Result<Unit, Error></code>	Removes a file or directory tree recursively
<code>make_dir</code>	<code>(str) -> Result<Unit, Error></code>	Creates a single directory
<code>make_dir_all</code>	<code>(str) -> Result<Unit, Error></code>	Creates a directory and all missing parents
<code>file_size</code>	<code>(str) -> Result<int, Error></code>	Returns file size in bytes
<code>is_file</code>	<code>(str) -> bool</code>	Checks if path is a regular file

Function	Signature	Description
<code>is_dir</code>	<code>(str) -> bool</code>	Checks if path is a directory
<code>is_symlink</code>	<code>(str) -> bool</code>	Checks if path is a symbolic link
<code>chmod</code>	<code>(str, int) -> Result<Unit, Error></code>	Changes file permissions (POSIX mode)
<code>symlink</code>	<code>(str, str) -> Result<Unit, Error></code>	Creates a symbolic link
<code>read_link</code>	<code>(str) -> Result<str, Error></code>	Reads the target of a symbolic link

Examples

Directory Operations

```
from filesystem import make_dir, make_dir_all, list_dir, remove_all

# Create a single directory
match make_dir("/tmp/myapp"):
    case Ok(_):
        print("created")
    case Err(e):
        print("error: " + e.message)

# Create nested directories (like mkdir -p)
make_dir_all("/tmp/myapp/data/logs")

# List directory contents
match list_dir("/tmp/myapp"):
    case Ok(entries):
        for entry in entries:
            print(entry)
    case Err(e):
        print("error: " + e.message)

# Remove a directory tree (like rm -rf)
remove_all("/tmp/myapp")
```

File Operations

```
from filesystem import copy, move, remove, file_size
from io import write_text

write_text("/tmp/hello.txt", "Hello, World!")

# Copy a file
copy("/tmp/hello.txt", "/tmp/hello_copy.txt")

# Get file size
match file_size("/tmp/hello.txt"):
    case Ok(sz):
        print("size: " + to_str(sz))
    case Err(e):
        print("error: " + e.message)

# Move / rename a file
```



```

move("/tmp/hello_copy.txt", "/tmp/renamed.txt")

# Remove a file
remove("/tmp/renamed.txt")

```

Recursive Traversal

```

from filesystem import walk, glob_files

# Walk a directory tree (like find)
match walk("/var/log"):
    case Ok(files):
        for f in files:
            print(f)
    case Err(e):
        print("error: " + e.message)

# Glob pattern matching
match glob_files("/var/log/*.log"):
    case Ok(matches):
        for m in matches:
            print(m)
    case Err(e):
        print("error: " + e.message)

```

Path Type Checks

```

from filesystem import is_file, is_dir, is_symlink

if is_file("/etc/hosts"):
    print("regular file")

if is_dir("/tmp"):
    print("directory")

if is_symlink("/usr/local/bin/python"):
    print("symbolic link")

```

Symbolic Links

```

from filesystem import symlink, read_link, is_symlink

# Create a symlink
symlink("/usr/local/bin/ry", "/tmp/ry_link")

# Check and read symlink
if is_symlink("/tmp/ry_link"):
    match read_link("/tmp/ry_link"):
        case Ok(target):
            print("points to: " + target)
        case Err(e):
            print("error: " + e.message)

```

Permissions

```
from filesystem import chmod

# chmod 755 (rwxr-xr-x) — use decimal value: 0o755 = 493
chmod("/tmp/script.sh", 493)

# chmod 644 (rw-r--r--) — 0o644 = 420
chmod("/tmp/data.txt", 420)
```

Notes

- `is_file`, `is_dir`, and `is_symlink` return `false` on error (e.g., path does not exist)
- `is_file` and `is_dir` follow symlinks; `is_symlink` uses `lstat` to detect links
- `list_dir` returns entry names only (not full paths)
- `walk` returns full paths for all entries (both files and directories)
- `glob_files` returns an empty list (not an error) when no files match the pattern
- `remove` fails on non-empty directories; use `remove_all` for recursive deletion

[English](#) | [日本語](#) | [繁體中文](#)

Function Reference

Function Definition Syntax

```
function function_name(param_name: type, ...) -> return_type:
```

```
    # body
    return value
```

- Parameter types are optional. When omitted, the parameter is treated as `any` type.
- Return type is optional. When omitted, the return type is **inferred from the body** (both named functions and lambdas). If the function has no `return` statement, the return type is inferred as `Unit`. Use `-> any` explicitly for functions that should accept any return type.
- The body is an indented block.
- Functions with an explicit return type (other than `Unit` or `any`) must have a `return` statement on all control flow paths. The compiler reports an error if any path is missing a return.
- Functions can have `require` (precondition) and `ensure` (postcondition) clauses. See [Design by Contract](#).

Naming convention: Function names and parameter names must use `snake_case` (e.g., `add`, `get_value`, `map_list`). The compiler enforces this convention.

```
function add(a: int, b: int) -> int:
    return a + b
```

```
function greet(name: str) -> Unit:
    print("Hello, " + name)    # Return type is Unit (explicit)
```

Parameter and Return Types

Item	Description
Parameter type	Optional. Defaults to <code>any</code> when the <code>: type</code> annotation is omitted
Return type	Optional. Inferred from the body when omitted (inferred as <code>Unit</code> if no <code>return</code> statement)

Item	Description
Unit	Return type for functions that return no value

Note: Function parameters are **immutable**. You cannot reassign a parameter inside the function body. This ensures that parameter values at entry are always available for postcondition checks (see [Design by Contract](#)).

```
function no_return(x: int) -> Unit:  # Return type Unit (explicit)
    print(x)
```

```
function get_value() -> int:        # Return type int
    return 42
```

```
function identity(x) -> any:       # Parameter type any (omitted)
    return x
```

Type Omission and any

When a parameter type annotation is omitted, the parameter is treated as **any** — a dynamic type that accepts any primitive value at runtime. This is similar to Python's untyped parameters.

```
# All parameters default to any
function add(a, b):
    return a + b

add(1, 2)                # 3 (int + int)
add("hello", " world")  # "hello world" (str + str)
add(1, 2.0)              # 3.0 (int + float)
```

You can also use **any** explicitly in type annotations:

```
function identity(x: any) -> any:
    return x
```

Return Type Inference

When the return type is omitted, it is inferred from the **return** statements in the body:

```
function double(x: int):          # return type inferred as int
    return x * 2

function greet(name: str):        # return type inferred as Unit (no return)
    print("Hello, " + name)
```

To explicitly allow any return type, use **-> any**:

```
function flexible(x: any) -> any:
    return x    # can return int, float, str, etc.
```

Recursion

Functions can call themselves.

```
function factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

Mutual Recursion

Functions can call each other regardless of definition order. The compiler forward-declares top-level functions with explicit return types before processing function bodies, provided all referenced types are already known (primitive types are always available; record/enum types must be defined earlier in the file).

```
function is_even(n: int) -> bool:
  if n == 0:
    return true
  return is_odd(n - 1)      # calls is_odd defined below

function is_odd(n: int) -> bool:
  if n == 0:
    return false
  return is_even(n - 1)    # calls is_even defined above
```

Requirements for forward references:

- The function must have an **explicit return type** annotation (`-> type`). Functions with inferred return types cannot be forward-referenced.
- The function must be defined at the **top level** (not nested inside another function).
- All parameter and return types must be resolvable at the point of forward declaration (e.g., record types must be defined before the functions that use them).

Tail Call Optimization

The compiler automatically detects self-recursive tail calls — where the last action in a function is a call to itself — and applies LLVM's `musttail` optimization. This guarantees that tail-recursive functions use constant stack space, preventing stack overflow for deep recursion.

```
function sum_to(n: int, acc: int) -> int:
  if n <= 0:
    return acc
  return sum_to(n - 1, acc + n)    # tail call -> optimized

sum_to(1000000, 0)    # works without stack overflow
```

Conditions for TCO:

- The function calls itself directly in a **return** statement (`return f(args)`)
- The call result is returned without any further computation (`return n * f(n-1)` is NOT a tail call)
- The function has no **ensure** (postcondition) clauses

Mutual recursion (A calls B, B calls A) is not currently optimized for tail calls.

Overloading

Multiple functions with the same name can be defined if they differ in the number or types of parameters.

Rules

- Functions with the same name can be defined if the number or types of parameters differ.
- The appropriate function is selected at the call site based on the argument types and count.
- Overloading by return type alone is not allowed.

```
function area(side: int) -> int:
  return side * side
```

```
function area(w: int, h: int) -> int:
    return w * h

a = area(5)          # 25
b = area(3, 4)       # 12
```

Resolution Priority

When multiple overloads match a call, the compiler selects the most specific one using the following priority (highest first):

1. **Exact type match** — argument type matches parameter type exactly
2. **Implicit widening** — safe widening conversion (`u8` $\rightarrow$ `int`, `u8` $\rightarrow$ `float`, `int` $\rightarrow$ `float`)
3. **Union type match** — argument type is a member of a union parameter type
4. **any type match** — parameter type is `any` (accepts anything)

The overload with the most exact matches wins. If two or more overloads have equal specificity, the compiler reports an ambiguity error.

Low-level numeric types (`i8`, `i16`, `i32`, `i64`, `u8` – `u64`, `f32`) do **not** participate in implicit widening — they require explicit `as` casts.

```
function process(x: int) -> str:
    return "int"

function process(x) -> str:          # x: any
    return "any"

process(42)          # "int" — exact match (int) beats any
process("hello")     # "any" — no exact match for str, falls back to any

function double(x: float) -> float:
    return x * 2.0

double(5)            # OK — int is implicitly widened to float, returns 10.0
```

Default Arguments

Parameters can have default values, allowing callers to omit trailing arguments.

Syntax

```
function connect(host: str, port: int = 8080, timeout: int = 30):
    # ...

connect("localhost")          # port=8080, timeout=30
connect("localhost", 3000)    # port=3000, timeout=30
connect("localhost", 3000, 10000) # port=3000, timeout=10000
```

Rules

- Default parameters must come after all non-default parameters.
- A parameter with a default value **must** have an explicit type annotation (e.g., `x: int = 10`; `x = 10` is a compile error).
- Default values must be compile-time constant expressions (literals and `@const` variables).

- If a function with default arguments creates an ambiguous overload (overlapping arity with matching types), the compiler reports an error.

```
# Error: ambiguous overload
function calc(x: int, y: int = 0) -> int:
    return x + y
function calc(x: int) -> int:           # conflicts with calc(int) from above
    return x * 2
```

Limitations

- Default arguments are not supported in **generic functions** or **lambda expressions**.
-

Unit Type Functions

Functions without a return value return `Unit`. The return type can be omitted (inferred as `Unit`) or explicitly specified with `-> Unit`.

```
function log(msg: str) -> Unit:
    print(msg)
```

Tasks And Async Functions

`Task<T>` is the built-in handle type for concurrent work. `async function` returns `Task<T>`, `await` extracts `T` inside another `async function`, and `block_on(task)` blocks from synchronous context until the task completes.

```
async function add(a: int, b: int) -> int:
    return a + b

# From synchronous context, use block_on()
t: Task<int> = add(20, 22)
print(block_on(t))           # 42
block_on(add(1, 2))          # waits and discards the result

# Inside async function, use await
async function double_add(a: int, b: int) -> int:
    return (await add(a, b)) * 2
```

Rules

- `async function name(...) -> T`: is declared with the awaited result type `T`.
 - Calling an `async function` immediately returns `Task<T>`.
 - `await expr` requires `expr` to be `Task<T>` and produces `T`.
 - `await` can only be used inside an `async function`. Use `block_on(task)` from synchronous context.
 - `block_on(task)` blocks the current thread until the task completes and returns the result.
 - `async function ... -> Unit` is supported; `block_on(task)` is the primary way to wait when no value is produced.
 - Tasks run on the runtime worker pool; they are not implemented as one OS thread per task.
 - `async lambdas` and `async @native function` are not supported in v1.
-

Lambda Functions

Anonymous functions can be defined inline.

Syntax

```
# Single expression (return type inferred from expression)
(param_name: type, ...) => expression

# Parameter type can be omitted (defaults to any)
(param_name, ...) => expression

# Multi-line block
(param_name: type, ...):
    # multiple statements
    return value

# With explicit return type (optional)
(param_name: type, ...) -> return_type => expression
```

Example

```
double = (x: int) => x * 2
result = double(5)    # 10

add = (a: int, b: int) => a + b
sum = add(3, 4)       # 7

# Multi-line lambda
abs = (x: int):
    if x < 0:
        return -x
    return x
```

Closures

Lambda functions **capture by value** the variables from the outer scope at the time of definition. This means the closure works with its own independent copy — changes in either direction (outer → closure or closure → outer) are not visible to the other side.

Outer changes do not affect the closure

```
base = 10
add_base = (x: int) => x + base    # Captures base by value

base = 99                          # Does not affect the captured value
r = add_base(5)                    # 15 (uses base = 10 from capture time)
```

Closure mutations do not affect the outer scope

```
counter = 0
items = [1, 2, 3, 4, 5]
items.map((x: int):
    counter += x    # Modifies the closure's local copy only
```

```

    return x
)
print(counter)      # 0 (outer variable unchanged)

```

Capture Rules

Item	Details
Capture method	Capture by value (copy)
Capture timing	At lambda definition time
Effect of outer variable changes	None (the closure holds its own copy)
Effect of mutations inside the closure	None (does not propagate to the outer scope)

Note for Python/JavaScript users: In JavaScript, closures capture variables by reference, so changes to a captured variable are reflected outside the closure. In Python, closures can access outer variables, and mutations of captured objects (for example, appending to a list) are visible outside, but rebinding an outer name (such as `counter += x`) requires declaring it `nonlocal`. In Ry, closures always capture by value. This is intentional — it ensures safety and predictability, especially in concurrent or higher-order contexts.

Function Type

A type for treating functions as values.

Syntax

```
function(param_type1, param_type2, ...) -> return_type
```

Example

```

f: function(int) -> int = (x: int) => x * 2
g: function(int, int) -> int = (a: int, b: int) => a + b

function apply(func: function(int) -> int, x: int) -> int:
    return func(x)

result = apply(f, 5)    # 10

```

Higher-Order Functions

Functions can accept functions as arguments or return them as values.

```

function map_list(xs: List<int>, f: function(int) -> int) -> List<int>:
    result: List<int> = []
    for x in xs:
        result += [f(x)]
    return result

doubled = map_list([1, 2, 3], (x: int) => x * 2)
# [2, 4, 6]

```


Generic Functions

Functions can have type parameters, enabling type-safe reuse without code duplication.

Syntax

```
function name<T, U>(param1: T, param2: U) -> T:  
    # body using T, U as types
```

Example

```
function identity<T>(x: T) -> T:  
    return x  
  
# Explicit type argument  
result = identity[int](42)           # 42  
result = identity[str]("hello")    # "hello"  
  
# Type inference (type argument deduced from actual argument)  
result = identity(42)                # T = int, result = 42  
result = identity("hello")           # T = str, result = "hello"
```

Multiple Type Parameters

```
function pick_first<T, U>(a: T, b: U) -> T:  
    return a  
  
result = pick_first(1, "x")           # T = int, U = str, result = 1  
result = pick_first("hello", 42)      # T = str, U = int, result = "hello"
```

Type Constraints (Bounds)

Type parameters can be constrained with record types using `: RecordName` syntax. The concrete type must be the bound type itself or a subtype of it.

```
record Animal:  
    name: str  
    legs: int  
  
record Dog < Animal:  
    breed: str  
  
function get_name<T: Animal>(a: T) -> str:  
    return a.name  
  
get_name(Dog("Rex", 4, "Lab"))      # OK — Dog is a subtype of Animal  
get_name(Animal("Cat", 4))           # OK — exact type match
```

Bounded and unbounded type parameters can be mixed:

```
function pair_name<T: Animal, U>(a: T, x: U) -> str:  
    return a.name
```

How It Works

Generic functions use **monomorphization**: a specialized version of the function is generated for each unique combination of type arguments. The same instantiation is cached and reused across multiple calls. When type constraints are present, they are validated at instantiation time.

UFCS (Uniform Function Call Syntax)

`a.f(b)` can be used to call `f(a, b)`. Useful for method chaining.

Syntax

```
# Normal call
f(a, b)

# UFCS call (equivalent)
a.f(b)
```

Chaining

```
function double(x: int) -> int:
    return x * 2

function add_one(x: int) -> int:
    return x + 1

result = 5.double().add_one()    # double(5) -> 10, add_one(10) -> 11
```

Mixing with Field Access

Field access (`.field`) and UFCS (`.method()`) use the same dot notation but are distinguished by the presence of arguments.

```
p = Point(3, 4)
length = p.x.to_float()    # Field access + UFCS
```

Operator Overloading

You can define operator behavior for user-defined types.

Syntax

```
# Binary operator (2 parameters)
function operator<op>(a: type, b: type) -> return_type:
    ...

# Unary operator (1 parameter)
function operator<op>(a: type) -> return_type:
    ...
```

Overloadable Operators

Category	Operators
Arithmetic (binary)	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> <code>**</code> <code>//</code>
Comparison (binary)	<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>
Bitwise (binary)	<code>&</code> <code> </code> <code>^</code> <code><<</code> <code>>></code>
Logical (binary)	<code>and</code> <code>or</code>
Membership	<code>in</code>

Category	Operators
Subscript	[] (read), []= (write)
Call	()
Cast	as
Unary	- ~ not

Return Type Constraints

Comparison, logical, and membership operators must return bool:

Category	Operators	Required Return Type
Comparison	== != < <= > >=	bool
Logical	and or not	bool
Membership	in	bool
Cast	as	Required (target type)

OK

```
function operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y
```

Error: comparison operator '==' must return 'bool', but returns 'int'

```
function operator==(a: Vec2, b: Vec2) -> int:
    return 42
```

Arithmetic and bitwise operators have no return type constraint.

Distinguishing Binary and Unary

Distinguished by the number of parameters.

```
record Vec2:
    x: float
    y: float
```

Binary +

```
function operator+(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)
```

Unary -

```
function operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

Comparison

```
function operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y
```

```
v1 = Vec2(1.0, 2.0)
v2 = Vec2(3.0, 4.0)
v3 = v1 + v2      # Vec2(4.0, 6.0)
v4 = -v1          # Vec2(-1.0, -2.0)
```

Checked/Saturating Arithmetic

Built-in functions for explicit overflow control on low-level integer types (`i8`, `i16`, `i32`, `i64`, `u8`, `u16`, `u32`, `u64`). Both arguments must be the same type.

Function	Returns	Behavior
<code>checked_add(a, b)</code>	<code>Result<T, Error></code>	Returns <code>Err</code> on overflow
<code>checked_sub(a, b)</code>	<code>Result<T, Error></code>	Returns <code>Err</code> on underflow
<code>checked_mul(a, b)</code>	<code>Result<T, Error></code>	Returns <code>Err</code> on overflow
<code>saturating_add(a, b)</code>	<code>T</code>	Clamps to type min/max on overflow
<code>saturating_sub(a, b)</code>	<code>T</code>	Clamps to type min/max on underflow
<code>saturating_mul(a, b)</code>	<code>T</code>	Clamps to type min/max on overflow
<code>wrapping_add(a, b)</code>	<code>T</code>	Explicit wrapping (same as <code>+</code>)
<code>wrapping_sub(a, b)</code>	<code>T</code>	Explicit wrapping (same as <code>-</code>)
<code>wrapping_mul(a, b)</code>	<code>T</code>	Explicit wrapping (same as <code>*</code>)

```
# Checked: returns Result, use match or ? to handle
r = checked_add(2147483647i32, 1i32)
match r:
  case Ok(v):
    print(v)
  case Err(e):
    print("overflow!")    # prints "overflow!"

# Saturating: clamps to bounds
v = saturating_add(2147483647i32, 100i32)
print(v as int)    # 2147483647

# Wrapping: self-documenting wrapping behavior
v = wrapping_add(2147483647i32, 1i32)
print(v as int)    # -2147483648
```

Note: These functions do not support floating-point types (`f32`) or the high-level `int` type. The default `+`, `-`, `*` operators on low-level integers use wrapping behavior (two's complement for signed, modular for unsigned).

[English](#)

GC Reference

The `gc` package provides control over the cycle collector, a safety net that works alongside ARC (Automatic Reference Counting) to detect and reclaim circular reference chains.

Overview

Ry uses ARC for memory management, which handles most deallocations immediately and deterministically. However, ARC alone cannot reclaim circular references (e.g., $A \rightarrow B \rightarrow A$). The cycle collector uses a CPython-style trial deletion algorithm to periodically find and free such unreachable cycles.

Using `weak` references is still the recommended way to avoid cycles, but the cycle collector catches cases the user misses.

Import

```
from gc import collect, enable, disable, set_threshold
```

Functions

Function	Signature	Description
<code>collect</code>	<code>() -> int</code>	Runs a full collection cycle. Returns the number of objects collected.
<code>enable</code>	<code>() -> Unit</code>	Enables automatic collection (enabled by default).
<code>disable</code>	<code>() -> Unit</code>	Disables automatic collection. Useful for performance-critical sections.
<code>set_threshold</code>	<code>(n: int) -> Unit</code>	Sets the candidate count threshold for automatic collection (default: 700).

How It Works

1. **Candidate tracking:** When `arc_release` decrements an object's reference count but it remains above zero, the object is added to a candidate set (only for types that can potentially form cycles).
2. **Trial deletion:** The collector tentatively decrements reference counts of all objects referenced by candidates. If an object's count reaches zero through trial deletion alone, it is only reachable within the candidate set — it is part of a cycle.
3. **Resurrection check:** Objects still reachable from outside the candidate set have their counts restored.
4. **Collection:** Remaining unreachable objects are deallocated.

Static Analysis Optimization

The compiler performs static analysis at compile time to identify which types can potentially form reference cycles (e.g., recursive ADT enums). Only these types participate in GC candidate tracking. Non-cyclic types have zero GC overhead.

Example

```
from gc import collect, disable, enable

# Disable automatic collection for a performance-critical section
disable()

# ... performance-critical code ...

# Re-enable and force a collection
enable()
collected = collect()
print(f"Collected {collected} cyclic objects")
```

[English](#)

HTTP Reference

Types

Type	Description
<code>HttpRequest</code>	Opaque handle for an incoming HTTP request
<code>HttpResponse</code>	Opaque handle for an outgoing HTTP response
<code>HttpClientResponse</code>	Opaque handle for an HTTP client response

`HttpRequest` is provided by the server framework. `HttpResponse` is created via `response()`. `HttpClientResponse` is returned by client functions (`http_get`, `http_post`, `http_request`).

Functions (from `http`)

These functions require explicit import:

```
from http import listen, method, path, header, body, body_bytes, query, query_all, cookie, cookies, form
```

Server

Function	Signature	Description
<code>listen</code>	<code>(host: str, port: int, handler: function(HttpRequest) -> HttpResponse) -> Unit</code>	Starts an HTTP server on the given address. Blocks in an accept loop, calling <code>handler</code> for each request.
<code>listen</code>	<code>(host: str, port: int, handler: function(HttpRequest) -> HttpResponse, max_requests: int) -> Unit</code>	Starts an HTTP server that stops after processing <code>max_requests</code> requests. Enables <code>async function + block_on()</code> lifecycle management.
<code>listen</code>	<code>(host: str, port: int, handler: function(HttpRequest) -> HttpResponse, max_requests: int, port_callback: function(int) -> Unit) -> Unit</code>	Same as above, but calls <code>port_callback</code> with the actual bound port after <code>bind + listen</code> succeeds. Use with port 0 for OS-assigned ephemeral ports.

Request Accessors

Function	Signature	Description
<code>method</code>	<code>(req: HttpRequest) -> str</code>	Returns the HTTP method (e.g., "GET", "POST").
<code>path</code>	<code>(req: HttpRequest) -> str</code>	Returns the request path without the query string (e.g., <code>"/search"</code> for <code>"/search?q=hello"</code>).
<code>header</code>	<code>(req: HttpRequest, key: str) -> Option<str></code>	Returns the value of a request header (case-insensitive lookup). Returns <code>None</code> if not found.
<code>body</code>	<code>(req: HttpRequest) -> str</code>	Returns the request body as a string. Truncated at the first NUL byte; use <code>body_bytes</code> for binary data.
<code>body_bytes</code>	<code>(req: HttpRequest) -> List<u8></code>	Returns the request body as a byte list. Binary-safe — preserves all bytes including NUL.

Function	Signature	Description
<code>query</code>	<code>(req: HttpRequest, key: str) -> Option<str></code>	Returns the value of a query parameter. Returns <code>None</code> if not found. Values are automatically URL-decoded.
<code>query_all</code>	<code>(req: HttpRequest) -> Map<str, str></code>	Returns all query parameters as a map. Keys and values are automatically URL-decoded.
<code>cookie</code>	<code>(req: HttpRequest, name: str) -> Option<str></code>	Returns the value of a cookie by name. Returns <code>None</code> if not found.
<code>cookies</code>	<code>(req: HttpRequest) -> Map<str, str></code>	Returns all cookies as a map.
<code>form_field</code>	<code>(req: HttpRequest, name: str) -> Option<str></code>	Returns the value of a multipart form text field. Returns <code>None</code> if not found.
<code>form_file</code>	<code>(req: HttpRequest, name: str) -> Option<Map<str, str>></code>	Returns file upload info as an <code>Option</code> . <code>Some(map)</code> contains keys <code>"filename"</code> , <code>"content_type"</code> , <code>"data"</code> . Returns <code>None</code> if not found.
<code>form_fields</code>	<code>(req: HttpRequest) -> Map<str, str></code>	Returns all multipart form text fields as a map.

Response Builder

Function	Signature	Description
<code>response</code>	<code>(status: int, headers: Map<str, str>, body: str) -> HttpResponse</code>	Creates an HTTP response with the given status code, headers, and body.

Usage Example

Basic HTTP Server

```
from http import listen, method, path, header, body, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    m = method(req)
    p = path(req)
    if p == "/hello":
        return response(200, {"Content-Type": "text/plain"}, "Hello, World!")
    if p == "/echo":
        b = body(req)
        return response(200, {"Content-Type": "text/plain"}, b)
    return response(404, {"Content-Type": "text/plain"}, "Not Found")
)
```

Non-blocking Server with async function

```
from http import listen, path, response

async function start_server(port: int) -> str:
    listen("127.0.0.1", port, (req: HttpRequest) -> HttpResponse:
        p = path(req)
        if p == "/api/health":
```

```

        return response(200, {"Content-Type": "application/json"}, "{\"status\": \"ok\"}")
    return response(404, {"Content-Type": "text/plain"}, "Not Found")
)
return "done"

t = start_server(8080)
# Server runs in background task

Server with Request Limit (max_requests)

from http import listen, path, response, http_get, status, body

port_holder = [0]
function on_port(p: int) -> Unit:
    port_holder[0] = p

async function start_server() -> str:
    listen("127.0.0.1", 0, (req: HttpRequest) -> HttpResponse:
        return response(200, {"Content-Type": "text/plain"}, "Hello!")
    , 1, on_port) # Stop after 1 request; call on_port with bound port
    return "done"

t = start_server()
sleep(100) # Wait for server to start
port = port_holder[0]

match http_get("http://127.0.0.1:" + to_str(port) + "/"):
    case Ok(resp):
        print(body(resp)) # "Hello!"
    case Err(e):
        print("error")

result = block_on(t) # Server exits after 1 request; block_on completes

Reading Query Parameters

from http import listen, path, query, query_all, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    p = path(req)
    if p == "/search":
        match query(req, "q"):
            case Some(q):
                return response(200, {"Content-Type": "text/plain"}, "Search: " + q)
            case None:
                return response(400, {"Content-Type": "text/plain"}, "Missing query parameter: q")
    return response(404, {"Content-Type": "text/plain"}, "Not Found")
)

Reading Headers

from http import listen, header, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    match header(req, "Authorization"):

```



```

        case Some(token):
            return response(200, {"Content-Type": "text/plain"}, "Authenticated: " + token)
        case None:
            return response(401, {"Content-Type": "text/plain"}, "Unauthorized")
    )

```

Handling Form Submissions

```

from http import listen, form_field, form_file, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    match form_field(req, "username"):
        case Some(name):
            match form_file(req, "avatar"):
                case Some(file_info):
                    filename = file_info["filename"]
                    return response(200, {"Content-Type": "text/plain"}, "Hello " + name + ", file: " +
                        filename)
                case None:
                    return response(400, {"Content-Type": "text/plain"}, "No file uploaded")
        case None:
            return response(400, {"Content-Type": "text/plain"}, "Missing username")
)

```

Reading Cookies

```

from http import listen, cookie, cookies, response

listen("127.0.0.1", 8080, (req: HttpRequest) -> HttpResponse:
    match cookie(req, "session_id"):
        case Some(sid):
            return response(200, {"Content-Type": "text/plain"}, "Session: " + sid)
        case None:
            return response(401, {"Content-Type": "text/plain"}, "No session")
)

```

Behavior

- `listen()` binds to the address, starts listening, and enters an accept loop.
- When called with 3 arguments, the accept loop runs indefinitely.
- When called with 4 arguments (`max_requests`), the server stops after processing the specified number of requests. `max_requests` must be a positive integer. This enables `async function + block_on()` lifecycle management. Malformed requests (silently skipped) do not count toward the limit.
- When called with 5 arguments (`max_requests`, `port_callback`), `port_callback` is called synchronously with the actual bound port after `bind + listen` succeeds. This allows safe use of port 0 (OS-assigned ephemeral port) to avoid port conflicts in parallel tests.
- The server supports HTTP/1.1 keep-alive by default. Multiple requests can be processed on a single connection. The server checks the `Connection` header on each request: if `Connection: close` is sent, the connection is closed after the response; otherwise the connection stays open for subsequent requests. Idle connections are closed after a 5-second timeout.
- `Content-Length` is automatically added to the response if not provided in the headers map.
- The server supports HTTP/1.1 with `Content-Length`-based body reading and `Transfer-Encoding: chunked` decoding.
- When `Transfer-Encoding: chunked` is present in a request, the body is automatically decoded and concatenated — Ry code receives the full body transparently.

- If both **Transfer-Encoding: chunked** and **Content-Length** are present, the request is rejected as malformed (per RFC 9112).
- To send a chunked response, include **"Transfer-Encoding": "chunked"** in the headers map passed to **response()**. The body will be encoded in chunked format automatically.
- Header lookup via **header()** is case-insensitive.
- **path()** returns the path without the query string. Query parameters are accessed separately via **query()** or **query_all()**.
- Query parameter values are automatically URL-decoded (**%20** $\rightarrow$ space, **+** $\rightarrow$ space).
- For duplicate query parameter keys, the first value is returned.
- **cookie()** and **cookies()** parse the **Cookie** header by splitting on **;**, then splitting each pair on the first **=**. Leading and trailing whitespace is trimmed from names and values.
- For duplicate cookie names, the first value is returned.
- Cookie values may contain **=** characters (only the first **=** separates the name from the value).
- **form_field()**, **form_file()**, and **form_fields()** parse **multipart/form-data** request bodies. Parsing is lazy — the body is parsed on the first call and cached.
- The **boundary** parameter is extracted from the **Content-Type** header and supports both quoted and unquoted values.
- Parts with a **filename** in **Content-Disposition** are treated as file uploads; parts without are treated as text fields.
- For duplicate field/file names, the first value is returned.
- **form_file()** returns **Some(map)** with keys **"filename"**, **"content_type"**, and **"data"**, or **None** if the field is not found. If no **Content-Type** is specified for the part, it defaults to **"application/octet-stream"**.
- For non-multipart requests, form functions return **None** (for **form_field**, **form_file**) or an empty map (for **form_fields**).

Supported Status Codes

The following status codes have standard reason phrases (RFC 9110):

Code	Reason
100	Continue
101	Switching Protocols
200	OK
201	Created
202	Accepted
203	Non-Authoritative Information
204	No Content
205	Reset Content
206	Partial Content
300	Multiple Choices
301	Moved Permanently
302	Found
303	See Other
304	Not Modified
307	Temporary Redirect
308	Permanent Redirect
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
405	Method Not Allowed
406	Not Acceptable
408	Request Timeout

Code	Reason
409	Conflict
410	Gone
411	Length Required
413	Content Too Large
414	URI Too Long
415	Unsupported Media Type
416	Range Not Satisfiable
417	Expectation Failed
422	Unprocessable Content
426	Upgrade Required
429	Too Many Requests
500	Internal Server Error
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout
505	HTTP Version Not Supported

Other status codes use "Unknown" as the reason phrase.

HTTP Client

Client Functions

Function	Signature	Description
<code>http_get</code>	<code>(url: str) -> Result<HttpClientResponse, Error></code>	Sends an HTTP GET request to the given URL.
<code>http_post</code>	<code>(url: str, body: str, headers: Map<str, str>) -> Result<HttpClientResponse, Error></code>	Sends an HTTP POST request with body and headers.
<code>http_request</code>	<code>(method: str, url: str, headers: Map<str, str>, body: str) -> Result<HttpClientResponse, Error></code>	Sends an HTTP request with a custom method.

Response Accessors

Function	Signature	Description
<code>status</code>	<code>(resp: HttpClientResponse) -> int</code>	Returns the HTTP status code.
<code>body</code>	<code>(resp: HttpClientResponse) -> str</code>	Returns the response body as a string. Truncated at the first NUL byte; use <code>body_bytes</code> for binary data.
<code>body_bytes</code>	<code>(resp: HttpClientResponse) -> List<u8></code>	Returns the response body as a byte list. Binary-safe — preserves all bytes including NUL.

Function	Signature	Description
<code>header</code>	<code>(resp: HttpClientResponse, key: str) -> Option<str></code>	Returns the value of a response header (case-insensitive). Returns <code>None</code> if not found.
<code>http_client_response_free</code>	<code>(resp: HttpClientResponse) -> Unit</code>	Frees the response and its associated memory. Call when done with the response.

Client Usage Example

```
from http import http_get, http_post, status, body, header

# Simple GET request
match http_get("http://example.com/api/data"):
    case Ok(resp):
        s = status(resp)
        b = body(resp)
        print(to_str(s) + ": " + b)
    case Err(e):
        print("Request failed")

# POST request with body and headers
headers: Map<str, str> = {"Content-Type": "application/json"}
match http_post("http://example.com/api/data", "{\"key\": \"value\"}", headers):
    case Ok(resp):
        print(body(resp))
    case Err(e):
        print("Request failed")
```

Client Behavior

- Both `http://` and `https://` URLs are supported. HTTPS uses TLS with system CA bundle certificate validation.
- The `Host` header is automatically added based on the URL.
- `Connection: close` is always sent; each request uses a separate TCP connection.
- `Content-Length` is always added automatically (including 0 for empty bodies). User-provided `Content-Length` headers are overridden with the correct value.
- Response body reading supports `Content-Length`, `Transfer-Encoding: chunked`, or `read-until-close`.
- Response header lookup is case-insensitive.
- Connection timeout is 5 seconds; receive timeout is 30 seconds.
- Returns `Err` on connection failure, invalid URL, or malformed response.
- `HttpClientResponse` owns allocated memory (headers, body). Call `http_client_response_free()` when done to avoid memory leaks.

Redirect Behavior

HTTP client functions automatically follow redirect responses (3xx with `Location` header).

- **Supported status codes:** 301, 302, 303, 307, 308
- **Maximum redirects:** 10 (returns `Err` if exceeded)
- **Method conversion** (per RFC 9110):
 - 301, 302: POST is changed to GET (body is dropped); other methods are preserved
 - 303: Method is always changed to GET (body is dropped)
 - 307, 308: Method and body are preserved

- **URL resolution:** Absolute URLs, protocol-relative (`//...`), absolute paths (`/...`), and relative paths in `Location` headers are supported
- User-provided headers are re-sent on each redirect hop, except sensitive headers (`Authorization`, `Proxy-Authorization`, `Cookie`) which are stripped on cross-origin redirects (different host or port)
- If the `Location` header is missing or empty, the redirect response is returned as-is
- The caller receives only the final response after all redirects are followed

Error Handling

- `listen()` raises a runtime error if `bind()` fails (e.g., port already in use).
- Malformed requests or idle timeouts on a keep-alive connection cause the connection to be closed. The server then resumes accepting new connections.
- The handler function must always return an `HttpResponse` —there is no default response.
- Client functions return `Result<HttpClientResponse, Error>` —use `match` to handle success and failure.

[English](#) | [日本語](#) | [繁體中文](#)

I/O Function Reference

Standard I/O and file operations. All functions require explicit import from `io`.

```
from io import read_text, write_text, exists
```

Function List

Standard Input

Function	Signature	Description
<code>read_line</code>	<code>() -> str</code>	Reads one line from stdin (trailing newline removed)
<code>read_all</code>	<code>() -> str</code>	Reads all of stdin until EOF

File I/O

Function	Signature	Description
<code>read_text</code>	<code>(str) -> Result<str, Error></code>	Reads entire file as a string
<code>write_text</code>	<code>(str, str) -> Result<Unit, Error></code>	Writes a string to a file (overwrites)
<code>append_text</code>	<code>(str, str) -> Result<Unit, Error></code>	Appends a string to the end of a file
<code>exists</code>	<code>(str) -> bool</code>	Checks if a file exists
<code>delete_file</code>	<code>(str) -> Result<Unit, Error></code>	Deletes a file
<code>read_bytes</code>	<code>(str) -> Result<List<u8>, Error></code>	Reads a file as a byte list
<code>write_bytes</code>	<code>(str, List<u8>) -> Result<Unit, Error></code>	Writes a byte list to a file

Byte Conversions

Function	Signature	Description
<code>to_bytes</code>	<code>(str) -> List<u8></code>	Converts a string to UTF-8 bytes
<code>bytes_to_str</code>	<code>(List<u8>) -> Result<str, Error></code>	Converts a byte list to a string

Examples

Reading and Writing Files

```
from io import read_text, write_text, append_text, exists, delete_file
```

```
match write_text("hello.txt", "Hello, World!"):
    case Ok(_):
        match read_text("hello.txt"):
            case Ok(content):
                print(content)    # Hello, World!
            case Err(e):
                print(e.message)
    case Err(e):
        print(e.message)
```

```
print(exists("hello.txt"))    # true
```

```
match delete_file("hello.txt"):
    case Ok(_):
        print(exists("hello.txt"))    # false
    case Err(e):
        print(e.message)
```

Byte Operations

```
from io import to_bytes, bytes_to_str, write_bytes, read_bytes
```

```
bs = to_bytes("ABC")
print(length(bs))    # 3
```

```
match write_bytes("data.bin", bs):
    case Ok(_):
        match read_bytes("data.bin"):
            case Ok(rb):
                match bytes_to_str(rb):
                    case Ok(s):
                        print(s)    # ABC
                    case Err(e):
                        print(e.message)
            case Err(e):
                print(e.message)
    case Err(e):
        print(e.message)
```

Reading from Standard Input

```
from io import read_line
```

```
name = read_line()
print(f"Hello, {name}!")
```

Error Handling

File operations return `Result<T, Error>` instead of terminating on failure. Use `match` with `Ok/Err` patterns to handle errors:

```
match read_text("missing.txt"):
    case Ok(content):
        print(content)
    case Err(e):
        print(e.message)    # cannot open file 'missing.txt' for reading
```

Operation	Error Condition
<code>read_text</code> / <code>read_bytes</code>	File does not exist or cannot be opened
<code>write_text</code> / <code>write_bytes</code> / <code>append_text</code>	File cannot be opened for writing
<code>delete_file</code>	File cannot be deleted
<code>bytes_to_str</code>	Input contains NUL byte

Notes

- `List<u8>` is used as the buffer type. Standard list operations (`length()`, `append()`, `slice()`, index access) all work with byte lists.
- File paths are relative to the current working directory unless absolute paths are specified.
- `write_text` and `write_bytes` overwrite existing files. Use `append_text` to add content to existing files.

[English](#) | [日本語](#) | [繁體中文](#)

JSON Function Reference

JSON parsing and serialization. All functions require explicit import from `json`.

```
from json import parse, stringify, kind, get, at, to_str, to_int, to_float, to_bool, length, keys, json
```

Overview

The `json` package provides functions to parse JSON text into an opaque `JsonValue` type, access its contents via accessor functions, and serialize it back to text. Since JSON values can be heterogeneous (objects can contain strings, numbers, booleans, arrays, and nested objects), the package uses an opaque pointer type with typed accessor functions.

Function List

Parse / Stringify

Function	Signature	Description
<code>parse</code>	<code>(str) -> Result<JsonValue, Error></code>	Parses a JSON string into a <code>JsonValue</code>
<code>stringify</code>	<code>(JsonValue) -> str</code>	Serializes a <code>JsonValue</code> to compact JSON text

Function	Signature	Description
<code>stringify</code>	<code>(JsonValue, int) -> str</code>	Serializes with pretty printing (indent = number of spaces)

Type Query

Function	Signature	Description
<code>kind</code>	<code>(JsonValue) -> str</code>	Returns the JSON type: "object", "array", "string", "number", "boolean", or "null"

Object / Array Access

Function	Signature	Description
<code>get</code>	<code>(JsonValue, str) -> Result<JsonValue, Error></code>	Gets a field from an object by key
<code>at</code>	<code>(JsonValue, int) -> Result<JsonValue, Error></code>	Gets an element from an array by index

Value Extraction

Function	Signature	Description
<code>to_str</code>	<code>(JsonValue) -> Result<str, Error></code>	Extracts a string value
<code>to_int</code>	<code>(JsonValue) -> Result<int, Error></code>	Extracts an integer value
<code>to_float</code>	<code>(JsonValue) -> Result<float, Error></code>	Extracts a float value
<code>to_bool</code>	<code>(JsonValue) -> Result<bool, Error></code>	Extracts a boolean value

Collection Info

Function	Signature	Description
<code>length</code>	<code>(JsonValue) -> int</code>	Returns the length of an array or number of keys in an object
<code>keys</code>	<code>(JsonValue) -> Result<List<str>, Error></code>	Returns the keys of an object, or an error if the value is not an object

Memory Management

Function	Signature	Description
<code>json_free</code>	<code>(JsonValue) -> Unit</code>	Frees a JsonValue and all its children

Usage Examples

Parsing and accessing fields

```
from json import parse, get, to_str, to_int, json_free

match parse("{\"name\": \"Alice\", \"age\": 30}"):
    case Ok(data):
        match get(data, "name"):
            case Ok(val):
                match to_str(val):
                    case Ok(name):
                        print(name)    # "Alice"
                    case Err(e):
                        print("error")
            case Err(e):
                print("error")
        json_free(data)
    case Err(e):
        print("parse error: " + e.message)
```

Working with arrays

```
from json import parse, at, to_int, length, json_free

match parse("[10, 20, 30]"):
    case Ok(data):
        print(to_str(length(data)))    # 3
        match at(data, 0):
            case Ok(elem):
                match to_int(elem):
                    case Ok(n):
                        print(to_str(n))    # 10
                    case Err(e):
                        print("error")
            case Err(e):
                print("error")
        json_free(data)
    case Err(e):
        print("parse error")
```

Stringify with pretty printing

```
from json import parse, stringify, json_free

match parse("{\"key\":\"value\", \"count\":42}"):
    case Ok(data):
        print(stringify(data, 2))
        # {
        #   "key": "value",
        #   "count": 42
        # }
        json_free(data)
    case Err(e):
        print("error")
```

Notes

- `to_int` accepts both JSON integers and floats that are whole numbers (e.g., $42.0 \rightarrow 42$)
- `to_float` accepts both JSON floats and integers (e.g., $42 \rightarrow 42.0$)
- `get` and `at` return references to child values within the parsed tree — do not call `json_free` on child values, only on the root value returned by `parse`
- The `kind` function returns "number" for both integers and floats

[English](#) | [日本語](#) | [繁體中文](#)

Math Functions (`math`)

Overview

The `math` package provides mathematical constants and functions. Unlike the `std` package, it is not automatically imported. Use explicit import to access the functions.

```
from math import sqrt, PI, sin
```

Constants

Constant	Type	Description
PI	float	Pi (3.141592653589793)
E	float	Euler' s number (2.718281828459045)
Inf	float	Positive infinity
NaN	float	Not a Number

```
from math import PI, E, Inf, NaN
```

```
circumference = 2.0 * PI * radius
```

Absolute Value

Function	Signature	Description
<code>abs(x)</code>	<code>(int) -> int</code>	Absolute value of integer
<code>abs(x)</code>	<code>(float) -> float</code>	Absolute value of float

```
from math import abs
```

```
abs(-5)      # 5  
abs(-3.14)   # 3.14
```

Rounding

Function	Signature	Description
<code>floor(x)</code>	<code>(float) -> int</code>	Round down to nearest integer
<code>ceil(x)</code>	<code>(float) -> int</code>	Round up to nearest integer

Function	Signature	Description
<code>round(x)</code>	<code>(float) -> int</code>	Round to nearest integer

```
from math import floor, ceil, round
```

```
floor(3.7)    # 3
ceil(3.2)     # 4
round(3.5)    # 4
```

Power and Root

Function	Signature	Description
<code>sqrt(x)</code>	<code>(float) -> float</code>	Square root
<code>pow(x, y)</code>	<code>(float, float) -> float</code>	x raised to the power of y

```
from math import sqrt, pow
```

```
sqrt(9.0)     # 3.0
pow(2.0, 3.0) # 8.0
```

Logarithm

Function	Signature	Description
<code>log(x)</code>	<code>(float) -> float</code>	Natural logarithm (base e)
<code>log2(x)</code>	<code>(float) -> float</code>	Base-2 logarithm
<code>log10(x)</code>	<code>(float) -> float</code>	Base-10 logarithm

Exponential

Function	Signature	Description
<code>exp(x)</code>	<code>(float) -> float</code>	e raised to the power of x

Trigonometric

Function	Signature	Description
<code>sin(x)</code>	<code>(float) -> float</code>	Sine (x in radians)
<code>cos(x)</code>	<code>(float) -> float</code>	Cosine (x in radians)
<code>tan(x)</code>	<code>(float) -> float</code>	Tangent (x in radians)

Inverse Trigonometric

Function	Signature	Description
<code>asin(x)</code>	<code>(float) -> float</code>	Arc sine (result in radians)
<code>acos(x)</code>	<code>(float) -> float</code>	Arc cosine (result in radians)
<code>atan(x)</code>	<code>(float) -> float</code>	Arc tangent (result in radians)

Two-argument Functions

Function	Signature	Description
<code>atan2(y, x)</code>	<code>(float, float) -> float</code>	Arc tangent of y/x (result in radians)
<code>hypot(x, y)</code>	<code>(float, float) -> float</code>	Hypotenuse: $\sqrt{x^2 + y^2}$

Predicates

Function	Signature	Description
<code>is_nan(x)</code>	<code>(float) -> bool</code>	True if x is NaN
<code>is_inf(x)</code>	<code>(float) -> bool</code>	True if x is positive or negative infinity

```
from math import is_nan, is_inf, NaN, Inf
```

```
is_nan(NaN)    # true
is_inf(Inf)    # true
is_nan(1.0)    # false
```

[English](#)

Network (TCP) Reference

Types

Type	Description
<code>TcpListener</code>	Opaque handle for a TCP server socket
<code>TcpStream</code>	Opaque handle for a TCP connection
<code>TlsStream</code>	Opaque handle for a TLS-encrypted TCP connection

All types are opaque pointers managed by ARC (Automatic Reference Counting). They cannot be constructed directly; use `bind()` or `connect()` to obtain `TcpListener`/`TcpStream`, and `tls_connect()` for `TlsStream`. Resources are automatically closed when no longer referenced — explicit `close()` calls are optional but supported for immediate cleanup.

Functions (from net)

These functions require explicit import:

```
from net import bind, listen, accept, connect, listener_port, shutdown, set_timeout, set_receive_timeout
```

Function	Signature	Description
<code>bind</code>	<code>(host: str, port: int) -> Result<TcpListener, Error></code>	Creates a TCP server socket bound to the given address. Returns Error on failure. Use port 0 for dynamic allocation.
<code>listen</code>	<code>(listener: TcpListener, backlog: int) -> Result<Unit, Error></code>	Starts listening for incoming connections. Returns Error on failure.
<code>accept</code>	<code>(listener: TcpListener) -> Result<TcpStream, Error></code>	Accepts a new connection. Waits up to 1 second for a client to connect. Returns Error on timeout or failure.
<code>connect</code>	<code>(host: str, port: int) -> Result<TcpStream, Error></code>	Connects to a remote TCP server. Times out after 5 seconds. Returns Error on timeout or failure.
<code>listener_port</code>	<code>(listener: TcpListener) -> int</code>	Returns the actual port number the listener is bound to. Useful when binding to port 0 (OS-assigned port).
<code>shutdown</code>	<code>(listener: TcpListener) -> Unit</code>	Signals the listener to stop accepting connections. Causes any pending <code>accept()</code> to return within at most 1 second.
<code>tls_connect</code>	<code>(host: str, port: int) -> Result<TlsStream, Error></code>	Connects to a remote server with TLS encryption. Validates the server certificate against the system CA bundle. Returns Error on connection or handshake failure.
<code>set_timeout</code>	<code>(stream: TcpStream TlsStream, ms: int) -> Unit</code>	Sets both receive and send timeout in milliseconds.
<code>set_receive_timeout</code>	<code>(stream: TcpStream TlsStream, ms: int) -> Unit</code>	Sets the receive timeout in milliseconds. <code>receive()</code> returns Error if no data arrives within this time.
<code>set_send_timeout</code>	<code>(stream: TcpStream TlsStream, ms: int) -> Unit</code>	Sets the send timeout in milliseconds.

Built-in Overloaded Functions

These functions are built-in and work with TCP socket types. No import needed.

Function	Signature	Description
<code>send</code>	<code>(stream: TcpStream TlsStream, data: List<u8>) -> Result<int, Error></code>	Sends bytes through a TCP or TLS connection. Returns Ok with the number of bytes sent, or Error on failure.

Function	Signature	Description
receive	(stream: TcpStream TlsStream, max: int) -> Result<List<u8>, Error>	Receives up to max bytes. Returns Ok with an empty list on connection close, or Err on error.
close	(handle: TcpStream TlsStream) -> Unit	Closes a TCP or TLS stream.
close	(handle: TcpListener) -> Unit	Closes a TCP listener.

Usage Example

Echo Server

```

from net import bind, listen, accept, connect
from io import to_bytes, bytes_to_str

# Server
match bind("127.0.0.1", 8080):
    case Ok(server):
        match listen(server, 128):
            case Ok(_):
                match accept(server):
                    case Ok(conn):
                        match receive(conn, 4096):
                            case Ok(data):
                                match send(conn, data):
                                    case Ok(_):
                                        ...
                                    case Err(e):
                                        print(e.message)
                                case Err(e):
                                    print(e.message)
                            close(conn)
                        case Err(e):
                            ...
                    case Err(e):
                        print("listen failed")
            close(server)
        case Err(e):
            print("bind failed")

```

Client

```

match connect("127.0.0.1", 8080):
    case Ok(conn):
        match send(conn, to_bytes("hello")):
            case Ok(_):
                ...
            case Err(e):
                print(e.message)
        match receive(conn, 4096):
            case Ok(resp):

```

```

        match bytes_to_str(resp):
            case Ok(s):
                print(s)
            case Err(e):
                print(e.message)
        case Err(e):
            print(e.message)
    close(conn)
case Err(e):
    print("connect failed")

```

Concurrent Echo Server with async function

```

from net import bind, listen, accept, connect, listener_port
from io import to_bytes, bytes_to_str

```

```

async function echo_server(server: TcpListener) -> str:

```

```

    match accept(server):
        case Ok(conn):
            match receive(conn, 4096):
                case Ok(data):
                    match send(conn, data):
                        case Ok(_):
                            ...
                        case Err(e):
                            ...
                    case Err(e):
                        ...
                close(conn)
            case Err(e):
                ...
    close(server)
    return "done"

```

```

match bind("127.0.0.1", 0):
    case Ok(server):
        match listen(server, 1):
            case Ok(_):
                port = listener_port(server)
                t = echo_server(server)
                # ... client code using port ...
                block_on(t)
            case Err(e):
                ...
        case Err(e):
            ...

```

Timeout Configuration

By default, `receive()` uses a 30-second timeout if no custom timeout is set. Use `set_timeout()`, `set_receive_timeout()`, or `set_send_timeout()` to override the default:

```

from net import connect, set_receive_timeout

```

```

match connect("127.0.0.1", 8080):

```

```

case Ok(conn):
    set_receive_timeout(conn, 5000) # 5-second timeout
    match receive(conn, 4096):
        case Ok(data):
            ...
        case Err(e):
            print("timeout or error")
    close(conn)
case Err(e):
    print("connect failed")

```

Pass 0 to disable the timeout (wait indefinitely).

Error Handling

- All TCP functions except `close()` return `Result<T, Error>` — use `match` with `Ok/Err` to handle failure.
- `receive()` returns `Ok` with an empty `List<u8>` when the connection is closed by the peer, and `Err` on actual errors (timeout, socket error).
- `close()` closes the socket and frees the handle. Using a handle after close is undefined behavior.

Byte Conversion

TCP operations work with `List<u8>`. Use `to_bytes()` and `bytes_to_str()` from `io` to convert between strings and byte lists.

[English](#) | [日本語](#) | [繁體中文](#)

Operator Reference

Precedence Table

Lower numbers indicate higher precedence (evaluated first).

Precedence	Operator	Description	Associativity
0	? !!	Error propagation (postfix)	Left
1	()	Grouping	–
2	+x -x ~x	Unary plus, unary minus, bitwise NOT	Right
3	**	Exponentiation	Right
3.5	as	Type cast	Left
4	* / % //	Multiplication, division, modulo, integer division	Left
5	+ -	Addition, subtraction	Left
6	<< >> >>>	Bit shift	Left
7	&	Bitwise AND	Left
8	^	Bitwise XOR	Left
9	\	Bitwise OR	Left
10	== != < <= > >= in not in	Comparison, membership	Left
11	not	Logical NOT	Right
12	and	Logical AND	Left
13	or	Logical OR	Left

Precedence	Operator	Description	Associativity
13.5	??	Null coalescing	Left

Arithmetic Operators

Operator	Description	Example
+	Addition / string concatenation	1 + 2 -> 3, "a" + "b" -> "ab", "x" + 1 -> "x1"
-	Subtraction	5 - 3 -> 2
*	Multiplication / string repetition	4 * 3 -> 12, "ab" * 3 -> "ababab"
/	Division (always float)	7 / 2 -> 3.5
//	Floor division (toward $-\infty$)	7 // 2 -> 3, -7 // 2 -> -4
%	Modulo	7 % 3 -> 1
**	Exponentiation (always float)	2 ** 10 -> 1024.0
-x	Unary minus	-5, -3.14
+x	Unary plus	+5 (no sign change)

```

a = 10 // 3      # 3 (int)
b = 10 / 3       # 3.3333... (float)
c = 2 ** 8       # 256.0 (float)
s = "foo" + "bar" # "foobar"
t = "val=" + 42   # "val=42"
u = 3.14 + "!"    # "3.14!"

```

When one operand of `+` is `str` and the other is `int`, `float`, or `bool`, the non-`str` operand is automatically converted to its string representation and concatenated.

Comparison Operators

All return `bool`.

Operator	Description
==	Equal
!=	Not equal
<	Less than
<=	Less than or equal
>	Greater than
>=	Greater than or equal

- Can be used with numeric types (`int` / `float`) and `bool`.
- `str` values are compared lexicographically (byte order).
- Record types support `==` and `!=` with auto-generated field-by-field comparison (see [Struct Reference](#)).
- The `in` operator is used for membership checks on sets, lists, and maps (`x in s`).
- The `not in` operator is the negation of `in` (`x not in s`).
- For maps, `in` checks whether the key exists.

```

x = 3 < 5          # true
y = "abc" < "abd"  # true (lexicographic)
s = {1, 2, 3}
z = 2 in s         # true
w = 4 not in s     # true

```

```
xs = [1, 2, 3]
a = 2 in xs      # true (list linear search)
m = {"a": 1}
b = "a" in m     # true (map key lookup)
```

Logical Operators

Operator	Description	Type
and	Logical AND	bool x bool -> bool
or	Logical OR	bool x bool -> bool
not	Logical NOT	bool -> bool

```
a = true and false  # false
b = true or false   # true
c = not true        # false
```

Bitwise Operators

Only available for int type. Applying to float or bool causes a compile error.

Operator	Description	Example
&	Bitwise AND	0b1100 & 0b1010 -> 0b1000
\	Bitwise OR	0b1100 \ 0b1010 -> 0b1110
^	Bitwise XOR	0b1100 ^ 0b1010 -> 0b0110
~	Bitwise NOT (unary)	~0 -> -1
<<	Left shift	1 << 4 -> 16
>>	Arithmetic right shift	16 >> 2 -> 4
>>>	Logical right shift	-1 >>> 1 -> 9223372036854775807

```
flags = 0b0001 | 0b0010  # 3
masked = flags & 0b0011   # 3
shifted = 1 << 8          # 256
```

Error Propagation Operator (? / !!)

The postfix ? operator unwraps a **Result** value. If the value is **Ok(v)**, it evaluates to v. If the value is **Err(e)**, the enclosing function immediately returns **Err(e)**.

The !! operator is an alias for ? with identical semantics. Both can be used interchangeably.

The enclosing function must have a **Result** return type.

```
function safe_divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:
        return Err(Error("division by zero"))
    return Ok(a // b)

function compute(a: int, b: int, c: int) -> Result<int, Error>:
    x = safe_divide(a, b)?      # returns Err early if b == 0
    y = safe_divide(x, c)!!
    return Ok(y + 1)
```

This is equivalent to the following match pattern, but much more concise:

```

function compute(a: int, b: int, c: int) -> Result<int, Error>:
    match safe_divide(a, b):
        case Ok(x):
            match safe_divide(x, c):
                case Ok(y):
                    return Ok(y + 1)
                case Err(e):
                    return Err(e)
        case Err(e):
            return Err(e)

```

when: Conditional Expression

```

x = when:
    condition => true_value
    else => false_value

```

Evaluates conditions from top to bottom and returns the expression from the first truthy arm. All result expressions must have the same type. The `else =>` arm is required, so the expression always produces a value.

```

x = when:
    3 > 2 => 10
    else => 20      # 10

```

```

s = when:
    false => "yes"
    else => "no"  # "no"

```

Nested ternaries flatten into multiple arms

```

y = when:
    true => 2
    false => 1
    else => 3      # 2

```

Range Operator

The `..` operator creates an inclusive integer range.

```

xs = 1 .. 5      # [1, 2, 3, 4, 5]

for i in 1 .. 3:
    print(i)      # 1 2 3

```

The result is a `List<int>` containing all integers from the left operand to the right operand (inclusive).

Null Coalescing Operator (??)

```

x = option_val ?? default_val

```

If `option_val` is `Some(v)`, returns `v`. Otherwise returns `default_val`. The right-hand operand must have the same type as the inner type of the `Option`.

```

a: int? = Some(10)
b: int? = none

print(a ?? 0)    # 10
print(b ?? 0)    # 0

```

Compound Assignment Operators

Shorthand for updating a variable. `x op= y` is equivalent to `x = x op y`.

Operator	Equivalent Expression
<code>x += y</code>	<code>x = x + y</code>
<code>x -= y</code>	<code>x = x - y</code>
<code>x *= y</code>	<code>x = x * y</code>
<code>x /= y</code>	<code>x = x / y</code>
<code>x %= y</code>	<code>x = x % y</code>
<code>x /= y</code>	<code>x = x // y</code>
<code>x **= y</code>	<code>x = x ** y</code>
<code>x &= y</code>	<code>x = x & y</code>
<code>x = y</code>	<code>x = x y</code>
<code>x ^= y</code>	<code>x = x ^ y</code>
<code>x <<= y</code>	<code>x = x << y</code>
<code>x >>= y</code>	<code>x = x >> y</code>

```

x = 10
x += 5    # x = 15
x -= 3    # x = 12
x *= 2    # x = 24
x /= 3    # x = 8
x &= 6    # x = 0

```

Increment / Decrement Operators

Postfix-only, statement-level operators for incrementing or decrementing a variable by 1. These are desugared to `x = x + 1` and `x = x - 1` respectively.

Operator	Equivalent Expression
<code>x++</code>	<code>x = x + 1</code>
<code>x--</code>	<code>x = x - 1</code>

```

count = 0
count++    # count = 1
count++    # count = 2
count--    # count = 1

f = 1.5
f++        # f = 2.5 (int 1 is promoted to float)

```

Note: `++` / `--` can only be used as statements, not as expressions. `@const` variables cannot be incremented/decremented (immutability is enforced).

Type Rules for Operations

Operation	Left Type	Right Type	Result Type
<code>+</code> <code>-</code> <code>*</code>	int	int	int
<code>+</code> <code>-</code> <code>*</code>	float	int / float	float
<code>+</code> <code>-</code> <code>*</code>	int	float	float
<code>/</code>	any numeric	any numeric	float
<code>//</code>	int	int	int
<code>//</code>	float or int (one is float)	-	float
<code>**</code>	any numeric	any numeric	float
<code>%</code>	int	int	int
<code>%</code>	float or int (one is float)	-	float
<code>+</code>	str	str	str
<code>+</code>	str	int / float / bool	str
<code>+</code>	int / float / bool	str	str
<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>	numeric / bool / str	same type	bool
<code>*</code>	str	int	str
<code>in</code>	any	Set / List / Map<K, V>	bool
<code>not in</code>	any	Set / List / Map<K, V>	bool
<code>&</code> <code>\ </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code> <code>>>></code>	int	int	int
<code>and</code> <code>or</code> <code>not</code>	bool	bool	bool

Operator Overloading

You can define operator behavior for user-defined types.

Syntax

```
# Binary operator (2 parameters)
function operator+(a: MyType, b: MyType) -> MyType:
    ...

# Unary operator (1 parameter)
function operator-(a: MyType) -> MyType:
    ...
```

Overloadable Operators

Category	Operators
Arithmetic (binary)	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> <code>**</code> <code>//</code>
Comparison (binary)	<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>
Bitwise (binary)	<code>&</code> <code>\ </code> <code>^</code> <code><<</code> <code>>></code> <code>>>></code>
Logical (binary)	<code>and</code> <code>or</code>
Membership	<code>in</code>
Subscript	<code>[]</code> (read), <code>[]=</code> (write)
Call	<code>()</code>
Cast	<code>as</code>
Unary	<code>-</code> <code>~</code> <code>not</code>
Compound assignment	<code>+=</code> <code>--</code> <code>*=</code> <code>/=</code> <code>%=</code> <code>//=</code> <code>**=</code> <code>&=</code> <code>\ =</code> <code>^=</code> <code><<=</code> <code>>>=</code>

Return Type Constraints

Comparison and logical operators must return `bool`:

Category	Operators	Required Return Type
Comparison	<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>	<code>bool</code>
Logical	<code>and</code> <code>or</code> <code>not</code>	<code>bool</code>
Membership	<code>in</code>	<code>bool</code>
Cast	<code>as</code>	Required (target type)

OK

```
function operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y
```

Error: comparison operator '==' must return 'bool', but returns 'int'

```
function operator==(a: Vec2, b: Vec2) -> int:
    return 42
```

Arithmetic and bitwise operators have no return type constraint.

Distinguishing Binary and Unary

Distinguished by the number of parameters.

Binary -

```
function operator-(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x - b.x, a.y - b.y)
```

Unary -

```
function operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

Compound Assignment Operator Overloading

Compound assignment operators (`+=`, `-=`, etc.) can be independently overloaded. This enables in-place optimization for large data structures.

```
record Matrix:
    data: List
    rows: int
    cols: int

function operator+=(a: Matrix, b: Matrix) -> Matrix:
    for i in range(len(a.data)):
        a.data[i] = a.data[i] + b.data[i]
    return a
```

Resolution Priority When `x += y` is evaluated:

1. If `operator+=` is defined for the types $\rightarrow$ call it directly
2. If `operator+=` is not defined but `operator+` is $\rightarrow$ fall back to `x = x + y`
3. If neither is defined (for non-builtin types) $\rightarrow$ compile error

```
record Vec2:
    x: float
    y: float
```

```
function operator+=(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)
```

```
v = Vec2(1.0, 2.0)
v += Vec2(3.0, 4.0) # calls operator+= directly
# v.x == 4.0, v.y == 6.0
```

Compound assignment operators require exactly 2 parameters and have no return type constraint.

Subscript Operator Overloading

The `[]` (read) and `[]=` (write) operators enable custom subscript behavior for user-defined types. Multi-index access (e.g., `m[row, col]`) is supported.

```
record Grid:
  a: int
  b: int
  c: int
  d: int

# Read: requires 2+ parameters (object + indices)
function operator[](g: Grid, row: int, col: int) -> int:
  if row == 0 and col == 0:
    return g.a
  if row == 0 and col == 1:
    return g.b
  if row == 1 and col == 0:
    return g.c
  return g.d

# Write: requires 3+ parameters (object + indices + value)
function operator[]=(g: Grid, row: int, col: int, value: int):
  ...

g = Grid(1, 2, 3, 4)
print(g[0, 1]) # 2
g[1, 0] = 99
```

User-defined subscript operators are tried first; if no match is found, built-in subscript behavior (for lists, maps, and arrays) is used as a fallback.

Membership Operator Overloading

The `in` operator can be overloaded to define custom membership tests. Must return `bool`.

```
record Range:
  lo: int
  hi: int

function operator in(value: int, r: Range) -> bool:
  return value >= r.lo and value < r.hi

r = Range(1, 10)
print(5 in r) # true
print(15 not in r) # true
```

User-defined `in` operators are tried first; if no match is found, built-in behavior (for sets, maps, and lists) is used as a fallback. `not in` is automatically supported when `in` is defined.

Call Operator Overloading

The `()` operator enables records to behave as callable objects. Requires at least 2 parameters (object + arguments).

```
record Adder:
  base: int

function operator()(a: Adder, x: int) -> int:
  return a.base + x

add5 = Adder(5)
print(add5(10))    # 15
```

When a variable holding a record value is called like a function, the compiler tries `operator()` overloads first. If no match is found, other call resolution strategies (functions, constructors, lambdas) take precedence.

Cast Operator Overloading

The `as` operator can be overloaded to define custom type conversions. Takes exactly 1 parameter (the source value) and must specify a return type (the target type). Dispatch matches by source type and return type.

```
record Celsius:
  value: int

record Fahrenheit:
  value: int

function operator as(c: Celsius) -> Fahrenheit:
  return Fahrenheit(c.value * 9 // 5 + 32)

c = Celsius(100)
f = c as Fahrenheit    # Fahrenheit(212)
```

The target type can be any type the compiler can resolve, including generic types:

```
record Temperature:
  value: int

function operator as(t: Temperature) -> int?:
  return Some(t.value)

t = Temperature(42)
result: int? = t as int?    # Some(42)
```

User-defined `as` operators are tried first; if no match is found, built-in casts (int, float, bool, str, etc.) are used as a fallback.

[English](#) | [日本語](#) | [繁體中文](#)

Package Reference

Overview

Ry uses a package system to organize code. A **package** can be either a single `.ry` file or a directory containing multiple `.ry` files. Use the `from` statement to import packages.

The `std` package (standard library) is automatically imported into every program.

Import Syntax

Import All Definitions

```
from math
```

Imports all functions and types from the package.

Selective Import

```
from math import sqrt
```

Imports only the specified definition.

Multiple Selective Import

```
from math import sqrt, PI
```

Imports multiple definitions separated by commas.

Relative Import

```
from .helper import greet
```

Imports from a module relative to the current file's directory. The `.` prefix restricts resolution to the current directory only (standard library and other search paths are not searched).

Relative Import from Subdirectory

```
from .utils import helper_fn
from .utils.calc import add
```

Imports from a subdirectory relative to the current file's directory.

Relative Import All from Current Directory

```
from . import add, sub
```

Imports specific symbols from the current directory package (all `.ry` files in the directory, excluding `_`-prefixed and `.test.ry` files).

Package Resolution

Packages are resolved using dot-separated notation:

Import Statement	Resolution
<code>from math</code>	<code>math/</code> directory (package) or <code>math.ry</code> file
<code>from utils.math</code>	<code>utils/math/</code> directory or <code>utils/math.ry</code> file
<code>from str</code>	<code>str/</code> directory or <code>str.ry</code> file

Resolution Order

For each search path, the system checks: 1. **Directory** (`{path}/`) — if exists, all `.ry` files in the directory are loaded (package) 2. **File** (`{path}.ry`) —single file (backward compatible)

Directory Packages

When a package resolves to a directory: - All `.ry` files in the directory are automatically loaded - Files starting with `_` are excluded - Test files (`.test.ry`) are excluded - No special entry file (like `__init__.py`) is needed - All functions and types defined in the directory's files are exported

Private Symbols

Definitions whose names start with `_` (underscore) are private to the package and cannot be imported:

- Wildcard imports (`from pkg`) automatically exclude `_`-prefixed symbols
- Named imports (`from pkg import _helper`) produce a compile error

```
# mylib/internal.ry
function _helper() -> int:      # private — not importable
    return 42
function public_api() -> int:  # public — importable
    return _helper()

mypackage/
  calc.ry      # function add(), function sub()
  string.ry    # function concat()

from mypackage      # imports add, sub, concat
from mypackage import add  # imports only add
```

Standard Library (std)

The `std` package is automatically imported into every program. It provides: - Built-in functions (`print`, `length`, `range`, etc.) - String functions (`contains`, `find`, `replace`, etc.) - Type conversion functions (`to_int`, `to_float`, `to_str`) - Collection functions (`map`, `filter`, `sort`, etc.)

Sub-packages

The following sub-packages require explicit import:

Package	Description
<code>math</code>	Mathematical constants and functions
<code>io</code>	File I/O, standard input, and byte conversions
<code>path</code>	File path operations (<code>join</code> , <code>basename</code> , <code>dirname</code> , etc.)

```
from math import sqrt, PI, sin
```

You can also explicitly import specific definitions from standard library packages:

```
from str import contains
```

RY_HOME

The standard library is installed at `$RY_HOME/lib/std/`. The default value of `RY_HOME` is `~/.ry`.

```
export RY_HOME="$HOME/.ry"  # default
```

RY_ENV

The `RY_ENV` environment variable controls the runtime environment mode. You can also use the `--env=<value>` CLI flag.

Value	Alias	.env loading	Lib search
prod	production	Disabled	Project override for repo builds → <code>\$RY_HOME/lib</code> → <code>exe/./lib</code> → <code>exe/lib</code>
dev	development	<code>.env.dev</code> → <code>.env</code>	Same as prod
test	—	<code>.env.test</code> → <code>.env</code>	Same as prod
staging	—	<code>.env.staging</code> → <code>.env</code>	Same as prod
internal	—	<code>.env.internal</code> → <code>.env</code>	Project override for repo builds → <code>exe/./lib</code> → <code>exe/lib</code> (<code>\$RY_HOME</code> skipped)
(unset) (default)	—	<code>.env</code> only	Same as prod

Aliases are automatically resolved to their canonical form. For example, `RY_ENV=production` is normalized to `prod`.

In `prod` mode, `.env` files are not loaded for security —production secrets should be managed via infrastructure-level environment variables (CI/CD, secret managers, etc.).

For other modes, `.env.<env>` is loaded first (if it exists), then `.env`. Environment-specific values take precedence because existing variables are not overwritten.

```
# Short form (recommended)
```

```
RY_ENV=dev ./build/ry app.ry
```

```
# Long form (backward compatible)
```

```
RY_ENV=development ./build/ry app.ry
```

```
# CLI flag
```

```
./build/ry --env=dev test
```

```
# prod mode: .env is NOT loaded
```

```
RY_ENV=prod ./build/ry app.ry
```

```
# Additional isolation when developing Ry itself
```

```
RY_ENV=internal ./build/ry test
```

When a `ry` executable is built inside the Ry source tree, it can use a repo-local `stdlib` override from the project's `package.toml`. This keeps repo builds aligned with the checked-out `lib/std` even if `~/.ry/lib/std` is older. Installed `ry` binaries ignore that override and continue to use `$RY_HOME/lib/std`.

Search Path Priority

1. The directory of the importing file
 2. Repo-local `stdlib` override from the current Ry checkout, when using a repo-built `ry`
 3. `$RY_HOME/lib` (standard library location)
 4. Executable-relative `lib/` directories
 5. Paths specified in the `RY_PATH` environment variable (colon-separated)
-

RY_PATH Environment Variable

Specifying colon-separated directories in RY_PATH adds them to the package search path.

```
export RY_PATH="/usr/local/ry/lib:/home/user/ry-packages"
```

Constraints

Constraint	Details
Allowed location	Top level only (not inside functions or blocks)
Duplicate imports	Automatically skipped (no error)
Circular imports	Compile error
Relative imports	<code>from .</code> and <code>from .pkg</code> resolve only against the current file's directory
Parent directory imports	<code>from ..</code> is not supported
Package names	Only letters, digits, and underscores are allowed (no hyphens)

```
# Error example: Import inside a block
function main():
    from math # Error: imports only allowed at top level

# OK: Importing the same package multiple times does not cause an error
from math
from math # Skipped
```

Creating Package Files

Single File Package

```
# calc.ry
function add(a: int, b: int) -> int:
    return a + b

function sub(a: int, b: int) -> int:
    return a - b

# main.ry
from calc import add, sub

print(add(1, 2)) # 3
print(sub(5, 3)) # 2
```

Directory Package

```
mylib/
  calc.ry
  string.ry

# main.ry
from mylib import add, concat
```

[English](#) | [日本語](#) | [繁體中文](#)

Path Function Reference

File path operations. All functions require explicit import from `path`.

```
from path import join, basename, dirname, extension, resolve, is_absolute
```

Function List

Function	Signature	Description
<code>join</code>	<code>(str, str) -> str</code>	Joins two path segments
<code>join</code>	<code>(str, str, str) -> str</code>	Joins three path segments
<code>join</code>	<code>(str, str, str, str) -> str</code>	Joins four path segments
<code>basename</code>	<code>(str) -> str</code>	Extracts the filename component
<code>dirname</code>	<code>(str) -> str</code>	Extracts the directory component
<code>extension</code>	<code>(str) -> str</code>	Extracts the file extension (including dot)
<code>resolve</code>	<code>(str) -> Result<str, Error></code>	Resolves a path to its absolute canonical form
<code>is_absolute</code>	<code>(str) -> bool</code>	Returns whether a path is absolute

Examples

Joining Paths

```
from path import join

p = join("/tmp", "data", "file.txt")
print(p)  # /tmp/data/file.txt

# Absolute second argument replaces the first
print(join("/tmp", "/usr"))  # /usr
```

Extracting Path Components

```
from path import basename, dirname, extension

p = "/home/user/docs/report.pdf"

print(basename(p))    # report.pdf
print(dirname(p))     # /home/user/docs
print(extension(p))   # .pdf
```

Extension Edge Cases

```
from path import extension

print(extension("archive.tar.gz"))  # .gz
print(extension(".gitignore"))      # (empty string — hidden file with no extension)
print(extension(".config.json"))    # .json
print(extension("Makefile"))        # (empty string)
```

Checking Absolute Paths

```
from path import is_absolute
```

```
print(is_absolute("/usr/local")) # true
print(is_absolute("src/main.ry")) # false
```

Resolving Paths

```
from path import resolve

match resolve("/tmp"):
    case Ok(p):
        print(p) # /private/tmp (on macOS) or /tmp
    case Err(e):
        print(e.message)

match resolve("/nonexistent"):
    case Ok(p):
        print(p)
    case Err(e):
        print(e.message) # cannot resolve path '/nonexistent': No such file or directory
```

[English](#) | [日本語](#) | [繁體中文](#)

Project Management

CLI Overview

```
ry <file.ry> [args...] # Run a Ry script
echo '<code>' | ry # Run code from stdin
ry test [options] [<file> | <dir>] # Run tests
ry init # Initialize a project
ry new <project-name> # Create a new project
ry run [<script-name>] # Run a project script
ry fmt [options] [<file> | <dir>] # Format source files
ry self-update [options] # Update ry itself
```

Global Options

Option	Description
-c	Read and execute code from stdin
-h, --help	Show help
-v, --version	Show version
--env=<env>	Set environment (production development internal). Overrides the RY_ENV environment variable.

Entry Point Execution

When no file argument is given, ry looks for a `package.toml` in the current directory (or parent directories) and runs the file specified by the `entry` field:

```
ry # runs entry file (e.g. src/main.ry)
ry -- arg1 arg2 # runs entry file with arguments
```

If no `package.toml` is found or no `entry` field is set, ry prints help and exits.

Stdin Execution

Use the `-c` flag to read and execute code from stdin:

```
echo 'print("hello")' | ry -c
```

ry init - Project Initialization

Initializes the current directory as a Ry project.

```
ry init
```

Generated Files and Directories

```
my-project/  
  package.toml      # Project configuration file  
  src/  
    main.ry         # Entry point (sample code)
```

Behavior

1. Exits with an error if `package.toml` already exists
 2. Creates the `src/` directory (if it doesn't exist)
 3. Generates `package.toml` (name is set to the current directory name)
 4. Generates `src/main.ry` (skipped if it already exists)
-

ry new - Create a New Project

Creates a new directory and initializes it as a Ry project.

```
ry new my-project
```

Generated Files and Directories

```
my-project/  
  package.toml      # Project configuration file  
  src/  
    main.ry         # Entry point (sample code)
```

Behavior

1. Exits with an error if no project name is given
 2. Exits with an error if the directory already exists
 3. Creates the `<project-name>/` directory
 4. Creates the `src/` directory inside it
 5. Generates `package.toml` (name is set to the given project name)
 6. Generates `src/main.ry`
-

ry run - Run Project Scripts

Executes a script defined in the `[scripts]` section of `package.toml`.

```
ry run           # List all available scripts
ry run build    # Run the "build" script
ry run test     # Run the "test" script
```

Behavior

1. Searches for `package.toml` from the current directory upward
2. Without arguments, lists all available scripts and their commands
3. With a script name, executes the corresponding shell command via `/bin/sh -c`
4. The exit code of the executed command is propagated
5. If the script name is not found, shows an error with a list of available scripts

Notes

- Does not require LLVM initialization (fast startup)
- Commands are executed in the current working directory
- Shell features (pipes, redirects, etc.) are supported since commands run through the shell

ry fmt - Code Formatter

Formats `.ry` source files with consistent 2-space indentation and canonical style.

```
ry fmt           # Format all .ry files in the project (requires package.toml)
ry fmt src/main.ry # Format a single file
ry fmt src/       # Format all .ry files in a directory (recursive)
ry fmt --check    # Check if files are formatted (exit 1 if not)
ry fmt --check src/ # Check specific directory
```

Formatting Rules

- 2-space indentation per block level
- Spaces around binary operators (`a + b`, not `a+b`)
- Space after comma (`f(a, b)`, not `f(a,b)`)
- Blank line between top-level definitions (functions, records, enums)
- Comments are preserved

Behavior

1. Reads the source file, parses it into an AST, and re-emits with canonical formatting
2. Writes the formatted result back to the file (in-place)
3. With `--check`, only reports unformatted files and exits with code 1 if any are found (useful for CI)
4. Skips `.git/`, `build/`, and `node_modules/` directories during recursive formatting

Notes

- Does not require LLVM initialization (fast startup)
- Compound assignment operators (`+=`, `-=`, etc.) are represented in their desugared form (`x = x + expr`) after formatting, because the parser desugars them during parsing
- Hex (`0xFF`) and binary (`0b1010`) number literals are converted to decimal notation

ry test - Run Tests

Discovers and runs test files (`*.test.ry`). See [Testing](#) for full test syntax documentation.


```

ry test                                # Auto-discover and run all *.test.ry files
ry test tests/spec                    # Run all tests under a directory
ry test test_file.ry                  # Run a specific test file
ry test -p                            # Run tests in parallel
ry test -w                            # Watch mode: re-run on file change
ry test --coverage                     # Collect line coverage information

```

Options

Option	Description
-p, --parallel	Run tests in parallel
-w, --watch	Watch for changes and re-run
--coverage, --cov	Collect coverage information
-h, --help	Show help

Behavior

1. Without arguments, searches for `package.toml` to find the project root and recursively discovers `*.test.ry` files (skipping `.git`, `build`, `node_modules`)
2. Exit code is 0 if all tests passed, 1 if any failed
3. `--coverage` with `--parallel` falls back to sequential execution

ry self-update - Self Update

Updates ry itself to the latest version. Downloads a binary from GitHub Releases and replaces the current executable.

```

ry self-update                        # Update to the latest stable version
ry self-update --nightly              # Update to the latest nightly pre-release
ry self-update v0.0.1                 # Update to a specified version

```

Behavior

1. Displays the current version
2. Resolves the target version based on arguments
 - No arguments: Latest stable release from GitHub (`/releases/latest`)
 - `--nightly`: Latest pre-release
 - Version specified: Release with the specified tag
3. If the current version is the same, exits with "Already up to date."
4. Downloads the binary and replaces the current executable

Security

Release archives are verified in two steps:

1. **Authenticity**: The `checksums.txt.sig` file is verified against the embedded Ed25519 public key.
 - If the signature file is **missing**, the update is aborted unless `RY_SKIP_SIGNATURE=1` is set.
 - If the signature file is **present but invalid**, the update is aborted regardless of `RY_SKIP_SIGNATURE`.
2. **Integrity**: The archive's SHA-256 hash is compared against `checksums.txt`.

To bypass the failure caused by a missing signature file (not recommended), set `RY_SKIP_SIGNATURE=1`. This does **not** bypass verification when a signature file is present but invalid.

Notes

- Requires `curl` and `tar` commands
- If replacing the binary fails due to insufficient permissions, a message suggesting `sudo` is displayed (`sudo` is not invoked automatically)
- Downloads are performed to a temporary directory first; however, if the cross-filesystem `cp` fallback is interrupted, the destination binary may be left in a partial state

package.toml Configuration File

Describes project metadata and path settings in TOML format.

```
[project]
name = "my-project"
version = "0.1.0"
entry = "src/main.ry"

[paths]
src = "src"

[scripts]
build = "cmake --preset default && cmake --build build"
test = "./build/ry_tests"
clean = "rm -rf build"
```

[project] Section

Key	Description
<code>name</code>	Project name (directory name at initialization)
<code>version</code>	Version string
<code>entry</code>	Source file serving as the entry point

[paths] Section

Key	Description
<code>src</code>	Source code directory

[scripts] Section

Defines named scripts that can be executed with `ry run <name>`. Each key is a script name and the value is a shell command string.

Key	Description
<code><name></code>	Shell command to execute (run with <code>ry run <name></code>)

TOML Subset Specification

`package.toml` supports the following TOML subset.

- Section headers: `[section]`

- Key-value pairs: `key = "value"` (string values only)
- Comments: From `#` to end of line
- Blank lines are ignored

[English](#) | [日本語](#) | [繁體中文](#)

Regular Expression Reference

Regex Literal Syntax

Regex literals use the `/pattern/` syntax and produce a `Regex` type value:

```
from regex import is_match, split, replace

# Regex literals enable type-based overloading
"hello".is_match(/[a-z]+)/          # true
"a1b2c".split(/[0-9]/)              # ["a", "b", "c"]
"abc123".replace(/[0-9]+/, "X")    # "abcX"
```

Regex literals can be stored in variables:

```
pat = /[a-z]+/
"hello".is_match(pat)  # true
```

The `/` inside a regex literal can be escaped with `\`:

```
"a/b".is_match(/a\/b/)  # true
```

Division vs Regex

The lexer uses context to distinguish regex literals from division:

- After value-producing tokens (identifiers, numbers, string literals, `)` or `]`), `/` is parsed as division
- After operators, keywords, or delimiters that expect an expression (`(`, `[`, `,`, `=`), `/` starts a regex literal

```
x = 10 / 2          # division: 5
y = is_match("a", /a/) # regex literal
```

Function List

Regex Literal Functions (text-first, UFCS-compatible)

These functions take a `Regex` type pattern and use text-first argument order for UFCS:

Function	Signature	Description
<code>is_match</code>	<code>(str, Regex) -> bool</code>	Returns whether the entire text matches the pattern
<code>search</code>	<code>(str, Regex) -> int</code>	Returns the start position of the first match (-1 if not found)
<code>replace</code>	<code>(str, Regex, str) -> str</code>	Replaces all matches with a replacement string
<code>split</code>	<code>(str, Regex) -> List<str></code>	Splits text by pattern matches
<code>find_all</code>	<code>(str, Regex) -> List<str></code>	Returns all non-overlapping matches

```
from regex import is_match, search, replace, split, find_all
```

```
# Direct call
print(is_match("hello", /[a-z]+/))  # true
```

```
# UFCS (text.function(pattern))
print("abc123".search(/[0-9]+)/)      # 3
print("abc123".replace(/[0-9]+/, "X")) # abcX
parts = "hello world".split(/\s+/)
nums = "a1b2c3".find_all(/[0-9]/)
```

Legacy Functions (text-first)

The original `regex_*` functions remain available for backward compatibility. They take pattern strings (not regex literals) with text-first argument order, consistent with the regex literal API:

Function	Signature	Description
<code>regex_match</code>	<code>(text: str, pattern: str) -> bool</code>	Returns whether the entire text matches the pattern
<code>regex_search</code>	<code>(text: str, pattern: str) -> int</code>	Returns the start position of the first match (-1 if not found)
<code>regex_replace</code>	<code>(text: str, pattern: str, replacement: str) -> str</code>	Replaces all matches with a replacement string
<code>regex_split</code>	<code>(text: str, pattern: str) -> List<str></code>	Splits text by pattern matches
<code>regex_find_all</code>	<code>(text: str, pattern: str) -> List<str></code>	Returns all non-overlapping matches

```
print(regex_match("hello", "[a-z]+")) # true
pos = regex_search("abc123", "[0-9]+") # 3
```

Supported Pattern Syntax

Syntax	Description	Example
<code>abc</code>	Literal characters	<code>"hello"</code>
<code>.</code>	Any character (except newline)	<code>"a.c"</code> matches <code>"abc"</code> , <code>"aXc"</code>
<code> </code>	Alternation	<code>"cat dog"</code> matches <code>"cat"</code> or <code>"dog"</code>
<code>*</code>	Zero or more	<code>"a*"</code> matches <code>"", "a", "aaa"</code>
<code>+</code>	One or more	<code>"a+"</code> matches <code>"a", "aaa"</code>
<code>?</code>	Zero or one	<code>"a?"</code> matches <code>""</code> or <code>"a"</code>
<code>{n}</code>	Exactly n times	<code>"a{3}"</code> matches <code>"aaa"</code>
<code>{n,m}</code>	Between n and m times	<code>"a{2,4}"</code> matches <code>"aa"</code> to <code>"aaaa"</code>
<code>{n,}</code>	At least n times	<code>"a{2,}"</code> matches <code>"aa", "aaa", ...</code>
<code>*?</code>	Zero or more (non-greedy)	<code>".*?"</code> matches shortest
<code>+?</code>	One or more (non-greedy)	<code>".+?"</code> matches shortest
<code>??</code>	Zero or one (non-greedy)	<code>"a??"</code> prefers zero
<code>{n,m}?</code>	Range (non-greedy)	<code>"a{2,4}?"</code> prefers n times
<code>(...)</code>	Group	<code>"(ab)+"</code> matches <code>"abab"</code>
<code>[abc]</code>	Character class	<code>"[aeiou]"</code> matches vowels
<code>[a-z]</code>	Character range	<code>"[a-z]+"</code> matches lowercase words
<code>[^...]</code>	Negated character class	<code>"[^0-9]"</code> matches non-digits
<code>^</code>	Start of string anchor	<code>"^hello"</code>

Syntax	Description	Example
\$	End of string anchor	"world\$"
\d	Digit [0-9]	"\d+" matches numbers
\D	Non-digit [^0-9]	
\w	Word character [a-zA-Z0-9_]	"\w+" matches identifiers
\W	Non-word character	
\s	Whitespace	"\s+" matches spaces/tabs
\S	Non-whitespace	
\b	Word boundary	"\bword\b" matches whole word
\B	Non-word boundary	"\Bword" matches inside a word
(?i)	Case-insensitive flag	"(?i)hello" matches "HELLO"
\.	Escaped special character	"\." matches literal .

Usage Examples

Range Quantifiers

```
print(regex_match("123-4567", "\\d{3}-\\d{4}")) # true
print(regex_match("aaa", "a{2,4}"))           # true
print(regex_match("ababab", "(ab){2,}"))      # true
```

Non-Greedy (Lazy) Match

```
# Greedy: matches longest
g = regex_replace("\"a\" and \"b\"", "\".*\"", "X")
print(g) # X
```

```
# Non-greedy: matches shortest
l = regex_replace("\"a\" and \"b\"", "\".*?\"", "X")
print(l) # X and X
```

```
# Find individual HTML-like tags
tags = regex_find_all("<a> <bb> <ccc>", "<.*?>")
print(length(tags)) # 3
```

Note: Non-greedy matching controls the overall match length. Without support for extracting parenthesized groups, mixed greedy/lazy patterns may behave differently from PCRE-style engines.

Word Boundary

```
# Match whole words only
pos = regex_search("hello world", "\\bworld\\b")
print(pos) # 6

# Find all words
words = regex_find_all("hello world foo", "\\b\\w+\\b")
print(length(words)) # 3
```

Case-Insensitive Matching

```
# (?i) at the start of pattern enables case-insensitive matching
print(regex_match("HELLO", "(?i)hello")) # true
```

```
print(regex_match("Hello", "(?i)hello")) # true
```

Note: `(?i)` must appear at the beginning of the pattern and applies to the entire pattern. Partial case-insensitive matching (e.g., `(?i:sub)pattern`) is not supported.

[English](#) | [日本語](#) | [繁體中文](#)

Record Reference

Overview

Structs are value types allocated on the stack. They are defined with the `record` keyword. Structs can have `invariant` clauses for Design by Contract. See [Design by Contract](#).

Naming convention: Struct names must use PascalCase (e.g., `Point`, `Rectangle`). Field names must use `snake_case`. The compiler enforces these conventions.

Definition Syntax

```
record TypeName:
    field_name: type
    field_name: type
```

Example

```
record Point:
    x: int
    y: int

record Rectangle:
    width: float
    height: float
```

Constructor

Arguments are passed in the order of field definitions. Named arguments are not supported.

```
p = Point(10, 20)
r = Rectangle(3.0, 4.5)
```

Field Access

Fields are read using dot notation.

```
p = Point(10, 20)
print(p.x)    # 10
print(p.y)    # 20
```

Field Assignment

Variable Declaration	Field Assignment
Mutable (no <code>@const</code>)	Allowed
<code>@const</code>	Compile error

```
p = Point(10, 20)
p.x = 100      # OK: mutable variable

@const
q = Point(10, 20)
q.x = 100      # Error: fields of @const variables cannot be modified
```

Usage as Function Parameters and Return Values

```
function distance(p: Point) -> float:
    return (p.x * p.x + p.y * p.y) as float

function make_point(x: int, y: int) -> Point:
    return Point(x, y)
```

Nested Structs

Structs can be used as fields of other structs.

```
record Point:
    x: int
    y: int

record Circle:
    center: Point
    radius: float

c = Circle(Point(0, 0), 1.0)
print(c.center.x)    # 0
```

Comparison (== / !=)

Record types automatically support `==` and `!=` operators. Comparison is performed field-by-field (structural equality).

```
record Point:
    x: int
    y: int

p1 = Point(10, 20)
p2 = Point(10, 20)
p3 = Point(30, 40)

print(p1 == p2)    # true
print(p1 != p3)    # true
```

- All fields are compared in order. For `==`, all fields must be equal. For `!=`, at least one field must differ.

- Nested records are compared recursively.
- If a user-defined `operator==` or `operator!=` is provided, it takes precedence over the auto-generated version.

Record Subtyping (Inheritance)

Records support single inheritance using the `<` syntax. A child record inherits all fields from its parent.

Syntax

```
record ChildName < ParentName:
  child_field: type
```

Example

```
record HttpError < Error:
  status: int
  url: str
```

Field Inheritance

- The child record inherits all parent fields at the beginning of its layout.
- The constructor takes parent fields first, then child-specific fields.

```
err = HttpError("not found", 404, 404, "/api")
print(err.message)  # "not found" (inherited from Error)
print(err.status)   # 404 (own field)
```

Subtype Coercion

A child value can be passed where the parent type is expected. The child is automatically sliced to extract the parent-prefix fields (value-type slicing).

```
function handle(e: Error) -> str:
  return e.message

err = HttpError("fail", 500, 500, "/api")
handle(err)  # OK — HttpError coerced to Error
```

Deep Inheritance

Records can form inheritance chains. Each level inherits all ancestor fields.

```
record DetailedHttpError < HttpError:
  detail: str

# Constructor: Error fields + HttpError fields + own fields
derr = DetailedHttpError("fail", 500, 500, "/x", "server crash")
handle(derr)  # OK — coerced to Error (grandparent)
```

Rules

Rule	Details
Single inheritance only	<code>record A < B</code> : — one parent only
Deep inheritance	<code>record C < B</code> : where <code>record B < A</code> : — allowed

Rule	Details
Name collision	Child field with same name as parent field → compile error
Auto == / to_str	Includes all inherited fields
Invariant inheritance	Parent invariant : clauses are checked when constructing or modifying child records
Subtype coercion	Applies to: function args, return, Err() , field assignment, ? operator
Generic bounds	<T: RecordName > constrains type parameter to subtypes of the record
@const	Applies to all fields including inherited

Constraints and Errors

Constraint	Details
Duplicate field names	Compile error
Field assignment on @const variables	Compile error

```
# Error example: Duplicate field names
record Bad:
  x: int
  x: int  # Error
```

Enumerations (enum)

Overview

Enumerations are a set of named constants. By default, they are represented as sequential i64 integers (0, 1, 2, ...). Explicit integer values can also be assigned.

Definition Syntax

```
enum TypeName:
  VariantName
  VariantName
  ...
```

Example

```
enum Color:
  Red
  Green
  Blue
```

Variant Access

Variants are accessed using the :: operator.

```
c = Color::Red
print(c)  # Red
```

Comparison

Since enum values are integers, they can be compared directly with `==` / `!=`.

```
print(Color::Red == Color::Red)    # true
print(Color::Red != Color::Green)  # true
```

Usage in if Statements

```
c = Color::Green
when:
  c == Color::Red:
    print("red")
  c == Color::Green:
    print("green")
  else:
    print("blue")
```

Function Parameters

Use the enum name as the type name.

```
function is_red(c: Color) -> bool:
  return c == Color::Red

print(is_red(Color::Red))    # true
print(is_red(Color::Green))  # false
```

print

`print()` outputs the variant name.

```
c = Color::Blue
print(c)    # Blue
```

Explicit Values

Simple enum variants can be assigned explicit integer values for use cases like HTTP status codes or bitmask patterns.

```
enum HttpStatus:
  Ok = 200
  NotFound = 404
  InternalError = 500

s = HttpStatus::NotFound
print(s)                # NotFound
print(s == HttpStatus::NotFound)  # true

enum Permission:
  Read = 1
  Write = 2
  Execute = 4
```

Rules: - Only simple enums (no ADT variants with associated data) support explicit values. - Values must be integer literals (negative values are allowed). - If any variant has an explicit value, all variants must have explicit values (no mixing auto and manual). - Duplicate values are a compile error. - `print()` displays the variant name, not the integer value.

Named Fields in ADT Variants

ADT variant fields can optionally include names for documentation purposes. Named fields make definitions self-describing without changing construction or pattern matching semantics.

```
enum Shape:
  Circle(radius: float)
  Rect(width: float, height: float)
  Point
```

- Construction is always positional: `Shape::Circle(3.14)`, not `Shape::Circle(radius: 3.14)`.
- Pattern matching binds user-chosen variable names: `case Shape::Circle(r):.`
- Field names must be `snake_case`. Mixing named and unnamed fields within a single variant is not allowed.
- Unnamed syntax (`Circle(float)`) remains valid.

Constraints and Errors

Constraint	Details
Variant access requires <code>EnumName::VariantName</code>	The <code>::</code> operator is required
Variant values	Auto-assigned (0, 1, 2, ...) by default, or explicitly specified with <code>= value</code>
Comparison uses integer comparison	<code>==</code> , <code>!=</code> can be used
Named field names	Must be <code>snake_case</code> ; no duplicates within a variant; no mixing named/unnamed

[English](#) | [日本語](#) | [繁體中文](#)

Testing

Ry has a built-in RSpec-style test syntax. Test files are executed using the `ry test` subcommand.

Running Tests

```
ry test           # Auto-discover and run all *.test.ry files in the project
ry test tests/spec # Run all *.test.ry files under a directory (recursive)
ry test test_file.ry # Run a specific test file
ry test -p         # Run all tests in parallel (-p or --parallel)
ry test -p tests/  # Run tests in a directory in parallel
ry test -w         # Watch mode: re-run tests on file change (-w or --watch)
ry test -w -p      # Watch mode with parallel execution
ry test -w tests/  # Watch a specific directory
ry test --coverage # Run all tests with line coverage summary
ry test --cov      # Short alias for --coverage
ry test --outline  # Print describe/it structure without running tests
```

The exit code is 0 if all tests passed, 1 if any test failed.

Auto-Discovery Mode

When `ry test` is run without arguments, it:

1. Searches for `package.toml` to find the project root

2. Recursively discovers all `*.test.ry` files under the project root (`.git`, `build`, `node_modules` are skipped)
3. Runs each file and aggregates results

Syntax

describe / it

```
describe("description", ():
  it("test case name", ():
    # test body
    expect(actual_value).to_eq(expected_value)
  )
)
```

- `describe` and `it` take a description string and a **lambda argument** `()`: as the second parameter
- `it` blocks and other statements (e.g., variable declarations) can be written inside a `describe` block
- Each `it` block is an independent test case
- `describe` / `expect` are only available with `ry test` (compile error with normal `ry` execution)

Trailing Block Syntax

Any function call (except `describe/it/mock`) can use trailing block syntax. A colon after `()` causes the indented block to be passed as a no-argument lambda in the last argument position:

These are equivalent:

```
foo("arg"):
  bar()
```

```
foo("arg", ():
  bar()
)
```

expect / Matchers

Matcher	Description	Supported Types
<code>to_eq(expected)</code>	Equality comparison	int, float, bool, str
<code>to_not_eq(expected)</code>	Asserts not equal	int, float, bool, str
<code>to_be_true()</code>	Asserts true	bool
<code>to_be_false()</code>	Asserts false	bool
<code>to_be_none()</code>	Asserts None	Option
<code>to_be_some()</code>	Asserts Option is Some	Option
<code>to_be_ok()</code>	Asserts Result is Ok	Result
<code>to_be_err()</code>	Asserts Result is Err	Result
<code>to_contain(val)</code>	Asserts container includes value	List, Set, Map, str
<code>to_not_contain(val)</code>	Asserts container does not include value	List, Set, Map, str
<code>to_be_greater_than(v)</code>	Asserts actual > v	int, float
<code>to_be_less_than(v)</code>	Asserts actual < v	int, float
<code>to_be_greater_than_or_eq(v)</code>	Asserts actual >= v	int, float
<code>to_be_less_than_or_eq(v)</code>	Asserts actual <= v	int, float
<code>to_have_length(n)</code>	Asserts length equals n	List, Set, Map, str
<code>to_be_empty()</code>	Asserts length is 0	List, Set, Map, str
<code>to_start_with(prefix)</code>	Asserts string starts with prefix	str
<code>to_end_with(suffix)</code>	Asserts string ends with suffix	str

Matcher	Description	Supported Types
---------	-------------	-----------------

fail

Immediately marks the current test as failed.

```
it("should not reach here", ():
  fail("unexpected error")
)
```

- `fail()` — marks the test as failed with a generic message
- `fail(msg)` — marks the test as failed with a custom message
- Execution continues after `fail()` (does not abort the test)
- Only available in `ry test` mode

Output Format

Calculator

```
+ adds numbers
+ subtracts
- fails test (red)
  line 10: expected 3, got 2
```

2 passed, 1 failed

- `+` indicates pass (green), `-` indicates failure (red)
- On failure, the line number and expected/actual values are displayed

Example

```
describe("Arithmetic", ():
  it("adds integers", ():
    expect(1 + 2).to_eq(3)

  )
  it("compares strings", ():
    expect("hello").to_eq("hello")

  )
  it("checks booleans", ():
    expect(3 > 1).to_be_true()

  )
)
describe("Booleans", ():
  it("false check", ():
    expect(1 > 2).to_be_false()

  )
)
```

Mocking

`mock(fn_name, replacement)`

Replaces a function with a mock implementation for the current `it` block. The mock is automatically cleared when the `it` block ends.

```
function fetch_data() -> str:
  return "real data"

describe("mocking", ():
  it("replaces function", ():
    mock(fetch_data, () => "fake")
    expect(fetch_data()).to_eq("fake")
  )
  it("auto-restores", ():
    expect(fetch_data()).to_eq("real data")
  )
)
```

- The first argument is the function name (identifier, not a string)
- The second argument is a replacement lambda
- The replacement must have the same parameter types and return type as the original function
- `require` and `ensure` contracts on the original function are still enforced when the mock is called
- Mocks are automatically restored at the end of each `it` block

`verify(fn_name)`

Returns the number of times a mocked function was called (as `int`).

```
describe("verify", ():
  it("counts calls", ():
    mock(fetch_data, () => "fake")
    fetch_data()
    fetch_data()
    expect(verify(fetch_data)).to_eq(2)
  )
)
```

Limitations

- Overloaded functions cannot be mocked
- Capture-based closures cannot be used as replacements (use plain lambdas)
- `@native function` functions cannot be mocked

Parameterized Tests (@each)

`@each` runs the same test with multiple sets of parameters. Attach it to an `it` block with a list of tuples:

```
@each([
  (1, 2, 3),
  (0, 0, 0),
  (-1, 1, 0)
])
it("adds {0} + {1} = {2}", (a: int, b: int, expected: int):
```

```
    expect(a + b).to_eq(expected)
)
```

- The list must contain tuples whose arity matches the lambda parameter count
- Placeholders {0}, {1}, ... in the description are replaced with the parameter values
- Each tuple generates an independent test case
- Supported parameter types: `int`, `float`, `bool`, `str`

Property-Based Tests (@property)

@property generates random inputs and runs the test multiple times:

```
@property(count=100)
it("addition is commutative", (a: int, b: int):
    expect(a + b).to_eq(b + a)
)
```

- count=N specifies the number of random trials (default: 100)
- On failure, the counterexample (failing inputs) is printed
- The test stops at the first failure
- Supported parameter types: `int` ([-1000, 1000]), `float` ([-1000.0, 1000.0]), `bool`, `str` (random ASCII, 0-20 chars)

Test Coverage

Run tests with the `--coverage` (or `--cov`) flag to measure line coverage:

```
ry test --coverage           # All tests with coverage summary
ry test --cov tests/spec/math.test.ry # Single file
ry test --coverage tests/spec/    # Directory
```

Output

Test Coverage Summary:

```
tests/spec/math.test.ry    100.0% (74/74 lines)
tests/spec/strings.test.ry  92.3%  (24/26 lines)
-----
Total                      95.1%  (98/100 lines)
```

- Only user code is reported; standard library files are excluded
- `--coverage` with `--parallel` falls back to sequential execution

Test Outline

Use `--outline` to display the `describe/it` structure of test files without executing any test bodies:

```
ry test --outline tests/spec/mock.test.ry
```

Output:

```
describe mock
  it replaces function
  it auto-restores after it block
  it with arguments
```

```
describe verify
  it counts calls
  it zero calls
```

- Works with individual files, directories, and `-p` (all test files)
- `@each` parameterized tests show the format template with an `(@each)` suffix
- `@property` tests show the label with a `(@property)` suffix

Limitations

- Nesting of `describe` is not supported
- `before_each` / `after_each` are not supported

[English](#)

Thread Reference

The `thread` package provides OS-level threading primitives for CPU-bound parallel workloads and fine-grained concurrency control.

Types

Type	Description
<code>Thread</code>	Opaque handle for an OS thread
<code>Lock</code>	Opaque handle for a mutex (mutual exclusion lock)
<code>RWLock</code>	Opaque handle for a read-write lock
<code>Semaphore</code>	Opaque handle for a counting semaphore
<code>Barrier</code>	Opaque handle for a thread barrier
<code>AtomicInt</code>	Opaque handle for an atomic 64-bit integer
<code>AtomicBool</code>	Opaque handle for an atomic boolean

All types are opaque pointers managed by ARC (Automatic Reference Counting). Use the corresponding `*_new()` functions to create instances. Resources are automatically cleaned up when no longer referenced — explicit `*_free()` calls are optional but supported for immediate cleanup.

Import

```
from thread import thread_spawn, thread_join, lock_new, lock_acquire, lock_release
```

Thread

Function	Signature	Description
<code>thread_spawn</code>	<code>(body: function() -> Unit) -> Thread</code>	Creates and starts a new OS thread executing <code>body</code> . Captured variables are copied by value.
<code>thread_join</code>	<code>(thread: Thread) -> Result<Unit, Error></code>	Waits for the thread to finish. Returns <code>Err</code> if the thread raised an error. The thread handle is consumed after join.

Example

```
from thread import thread_spawn, thread_join, atomic_int_new, atomic_int_load, atomic_int_add

counter = atomic_int_new(0)
t = thread_spawn():
    atomic_int_add(counter, 1)
)
thread_join(t)
print(atomic_int_load(counter)) # 1
```

Note: Captured variables are copied into the thread's environment. For primitive types (int, float, bool, str), this produces an independent copy. For opaque handle types (Lock, AtomicInt, etc.), the pointer is copied, sharing the underlying resource — which is the intended behavior for synchronization primitives.

Lock (Mutex)

Function	Signature	Description
lock_new	() -> Lock	Creates a new mutex.
lock_acquire	(lock: Lock) -> Result<Unit, Error>	Acquires the lock. Blocks until available.
lock_release	(lock: Lock) -> Result<Unit, Error>	Releases the lock.
lock_free	(lock: Lock) -> Unit	Immediately frees the lock. Optional — ARC handles cleanup automatically.

RWLock (Read-Write Lock)

Function	Signature	Description
rwlock_new	() -> RWLock	Creates a new read-write lock.
rwlock_read_lock	(rwlock: RWLock) -> Result<Unit, Error>	Acquires a shared read lock. Multiple readers allowed.
rwlock_write_lock	(rwlock: RWLock) -> Result<Unit, Error>	Acquires an exclusive write lock. Blocks until all readers and writers release.
rwlock_unlock	(rwlock: RWLock) -> Result<Unit, Error>	Releases the lock (shared or exclusive).
rwlock_free	(rwlock: RWLock) -> Unit	Immediately frees the read-write lock. Optional — ARC handles cleanup automatically.

Semaphore

Function	Signature	Description
semaphore_new	(count: int) -> Result<Semaphore, Error>	Creates a semaphore with the given initial count. Returns Err if count is negative.
semaphore_acquire	(sem: Semaphore) -> Result<Unit, Error>	Decrements the semaphore. Blocks if count is zero.

Function	Signature	Description
<code>semaphore_release</code>	<code>(sem: Semaphore) -> Result<Unit, Error></code>	Increments the semaphore and wakes a waiting thread.
<code>semaphore_free</code>	<code>(sem: Semaphore) -> Unit</code>	Immediately frees the semaphore. Optional — ARC handles cleanup automatically.

Barrier

Function	Signature	Description
<code>barrier_new</code>	<code>(count: int) -> Result<Barrier, Error></code>	Creates a barrier that synchronizes <code>count</code> threads. Returns <code>Err</code> if count is not positive.
<code>barrier_wait</code>	<code>(barrier: Barrier) -> Result<Unit, Error></code>	Blocks until all <code>count</code> threads have called <code>barrier_wait</code> .
<code>barrier_free</code>	<code>(barrier: Barrier) -> Unit</code>	Immediately frees the barrier. Optional — ARC handles cleanup automatically.

AtomicInt

Function	Signature	Description
<code>atomic_int_new</code>	<code>(value: int) -> AtomicInt</code>	Creates an atomic integer with the given initial value.
<code>atomic_int_load</code>	<code>(atomic: AtomicInt) -> int</code>	Atomically reads the value.
<code>atomic_int_store</code>	<code>(atomic: AtomicInt, value: int) -> Unit</code>	Atomically writes the value.
<code>atomic_int_add</code>	<code>(atomic: AtomicInt, delta: int) -> int</code>	Atomically adds <code>delta</code> and returns the previous value.
<code>atomic_int_sub</code>	<code>(atomic: AtomicInt, delta: int) -> int</code>	Atomically subtracts <code>delta</code> and returns the previous value.
<code>atomic_int_cas</code>	<code>(atomic: AtomicInt, expected: int, desired: int) -> bool</code>	Compare-and-swap: if current value equals <code>expected</code> , sets to <code>desired</code> and returns <code>true</code> ; otherwise returns <code>false</code> .
<code>atomic_int_free</code>	<code>(atomic: AtomicInt) -> Unit</code>	Immediately frees the atomic integer. Optional — ARC handles cleanup automatically.

AtomicBool

Function	Signature	Description
<code>atomic_bool_new</code>	<code>(value: bool) -> AtomicBool</code>	Creates an atomic boolean with the given initial value.
<code>atomic_bool_load</code>	<code>(atomic: AtomicBool) -> bool</code>	Atomically reads the value.
<code>atomic_bool_store</code>	<code>(atomic: AtomicBool, value: bool) -> Unit</code>	Atomically writes the value.
<code>atomic_bool_free</code>	<code>(atomic: AtomicBool) -> Unit</code>	Immediately frees the atomic boolean. Optional — ARC handles cleanup automatically.

Comparison with `async/await`

Feature	<code>async/await</code>	<code>thread</code> package
Execution model	Work-stealing thread pool	Dedicated OS threads
Best for	I/O-bound tasks, many lightweight tasks	CPU-bound tasks, fine-grained control
Synchronization	Automatic via <code>await</code>	Manual via <code>Lock</code> , <code>Semaphore</code> , etc.
Overhead	Low (task scheduling)	Higher (OS thread creation)

[English](#) | [日本語](#) | [繁體中文](#)

Type Reference

Type List

Type	Internal Representation	Literal Examples	Description
<code>int</code>	<code>i64</code>	<code>42</code> , <code>-7</code> , <code>0xFF</code> , <code>0b1010</code> , <code>100_000</code>	64-bit signed integer
<code>u8</code>	<code>i8</code>	(no dedicated literal)	Unsigned 8-bit integer (0-255). Used with type annotation <code>b</code> : <code>u8 = 42</code>
<code>float</code>	<code>f64</code>	<code>3.14</code> , <code>0.5</code> , <code>.5</code> , <code>3.14_159</code>	64-bit floating-point number
<code>bool</code>	<code>i1</code>	<code>true</code> , <code>false</code>	Boolean value
<code>str</code>	<code>ptr</code>	<code>"hello"</code> , <code>""</code> , <code>"a\nb"</code>	String (immutable byte sequence on the heap)
<code>Unit</code>	<code>void</code>	(no return value)	Return type for functions with no return value. Must be specified explicitly with <code>-> Unit</code>
<code>Option<T></code>	<code>{ i1, T }</code>	<code>Some(42)</code> , <code>None</code>	A type that may or may not contain a value
<code>(T1, T2, ...)</code>	LLVM <code>StructType</code> (literal)	<code>(1, 3.14)</code>	Tuple type
<code>List<T></code>	<code>ptr</code> (heap)	<code>[1, 2, 3]</code>	Dynamic array
<code>Map<K, V></code>	<code>ptr</code> (heap)	<code>{"a": 1}</code>	Hash map
<code>Set<T></code>	<code>ptr</code> (heap)	<code>{1, 2, 3}</code>	Set with no duplicates
<code>function(T1, T2) -> R</code>	<code>ptr</code> (function pointer)	<code>(x: int) => x * 2</code>	Function type
User-defined type	LLVM <code>StructType</code> (named)	<code>record Point: ...</code>	Struct defined with the <code>record</code> keyword
<code>enum</code>	<code>i64</code> / tagged union	<code>Color::Red</code> , <code>Shape::Circle(3.14)</code>	Enumeration defined with the <code>enum</code> keyword (supports associated data)
<code>Error</code>	<code>{ ptr, i64 }</code>	<code>Error("msg")</code> , <code>Error("msg", 404)</code>	Built-in error type
<code>any</code>	<code>{ i64, [8 x i8] }</code>	<code>x: any = 42</code>	Tagged union that can hold any primitive value
<code>T1 \ T2</code>	<code>{ i64, [N x i8] }</code>	<code>int \ str</code>	Union type (holds one of multiple types)

Type	Internal Representation	Literal Examples	Description
Int literal	i64	42, 0 \ 1	Int literal type (value constraint)
String literal	ptr	"N" \ "S"	String literal type (value constraint)
Range	i64	1..12, -10..10	Range type (inclusive integer range constraint)
i8	i8	x: i8 = 42, x = 42i8	8-bit signed integer (low-level, no implicit conversion)
i16	i16	x: i16 = 100, x = 100i16	16-bit signed integer (low-level, no implicit conversion)
i32	i32	x: i32 = 42, x = 42i32	32-bit signed integer (low-level, no implicit conversion)
i64	i64	x: i64 = 100, x = 100i64	64-bit signed integer (low-level, no implicit conversion)
u8	i8	x: u8 = 200, x = 200u8	8-bit unsigned integer (low-level, no implicit conversion)
u16	i16	x: u16 = 60000, x = 60000u16	16-bit unsigned integer (low-level, no implicit conversion)
u32	i32	x: u32 = 3000000000, x = 100u32	32-bit unsigned integer (low-level, no implicit conversion)
u64	i64	x: u64 = 100, x = 100u64	64-bit unsigned integer (low-level, no implicit conversion)
f32	float	x: f32 = 3.14, x = 3.14f32	32-bit floating-point (low-level, no implicit conversion)
weak T	ptr (header)	weak s	Weak reference to an ARC-managed value (does not prevent deallocation)
Regex	ptr	/[a-z]+/, /\d{3}/	Regular expression pattern (created via regex literal syntax)
Result<T, E>	{ i1, T/E }	Ok(42), Err(Error("fail"))	A type representing success (Ok) or failure (Err)
Task<T>	ptr	(returned by async functions)	Asynchronous task handle (used with await and block_on)
Iterator<T>	ptr	(created by <code>iter()</code>)	Lazy iterator for sequential element access
T[N]	[N x T]	buf: i32[8]	Fixed-length contiguous array of low-level type T with N elements (stack-allocated)

Type Annotation Syntax

You can explicitly specify the type when declaring a variable. The annotation can be omitted when the type is inferable.

```
x: int = 42
b: u8 = 255
f: float = 3.14
s: str = "hello"
b: bool = true
opt: Option<int> = Some(10)
t: (int, float) = (1, 3.14)
xs: List<int> = [1, 2, 3]
m: Map<str, int> = {"a": 1}
s: Set<int> = {1, 2, 3}
fn_val: function(int) -> int = (x: int) => x * 2
rx: Regex = /[0-9]+/
u: int | str = 42
a: any = 42
```

Available Type Names

Type Name	Notes
int	Built-in scalar type
u8	Built-in scalar type (unsigned 0-255)
float	Built-in scalar type
bool	Built-in scalar type
str	Built-in string type
Unit	Return type of functions that return no value
Option<T>	Generic type (T is any type)
(T1, T2, ...)	Tuple type (arbitrary number and combination of element types)
List<T>	Generic dynamic array type
Map<K, V>	Generic hash map type
Set<T>	Generic set type
function(T1, ...) -> R	Function type
Error	Built-in error type (<code>message: str, code: int</code>)
any	Built-in type that can hold any primitive value (<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code>) or <code>Unit</code> . Supports implicit conversion: concrete values are automatically wrapped when assigned to <code>any</code> , and <code>any</code> values are automatically unwrapped (with runtime type check) when assigned to a concrete type. <code>any(int) → float</code> auto-promotion is supported. See any Type for details
T1 \ T2 \ ...	Union type (one of multiple types separated by <code>\ </code>)
i8	Low-level 8-bit signed integer (no implicit conversion)
i16	Low-level 16-bit signed integer (no implicit conversion)
i32	Low-level 32-bit signed integer (no implicit conversion)
i64	Low-level 64-bit signed integer (no implicit conversion)

Type Name	Notes
u8	Low-level 8-bit unsigned integer (no implicit conversion)
u16	Low-level 16-bit unsigned integer (no implicit conversion)
u32	Low-level 32-bit unsigned integer (no implicit conversion)
u64	Low-level 64-bit unsigned integer (no implicit conversion)
f32	Low-level 32-bit floating-point (no implicit conversion)
T[N]	Fixed-length array of low-level type T with N elements. Stack-allocated, contiguous memory. Supports index read/write and <code>length()</code>
User-defined type name	Type declared with the <code>record</code> or <code>enum</code> keyword

Type Aliases

The `type` keyword creates a new name for an existing type. The alias is fully interchangeable with the original type.

```
type Meters = float
type StringList = List<str>
```

```
d: Meters = 3.14
names: StringList = ["Alice", "Bob"]
```

Naming convention: Type alias names must use PascalCase (e.g., `Meters`, `StringList`). The compiler enforces this convention.

Type aliases also work with function types, literal types, and range types:

```
type Callback = function(int, int) -> int

add: Callback = function(a: int, b: int) => a + b
print(add(3, 4))    # 7

type Month = 1..12
type Direction = "N" | "S" | "E" | "W"
type Digit = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

m: Month = 6
d: Direction = "N"
n: Digit = 5
```

Literal Types

A literal type restricts a variable to specific constant values. The compiler checks these constraints at compile time for constant values, and emits runtime checks for dynamic values.

Int Literal Type

```
x: 42 = 42           # single literal type
y: 0 | 1 = 0         # union of int literals
```

```

z: 0 | 1 = 0
z = 1          # OK
# z = 2        # compile error (constant) or runtime error (dynamic)

```

String Literal Type

```

dir: "N" | "S" | "E" | "W" = "N"
# @const bad: "N" | "S" = "X"    # compile error

```

Constraint Checking

- **Compile time:** If the assigned value is a constant (`ConstantInt` or string literal), the constraint is checked at compile time and a compile error is raised on violation.
- **Runtime:** If the value is dynamic (e.g., from a function call), the constraint is checked at runtime and the program exits with an error on violation.

Range Type

A range type constrains an integer variable to a contiguous range of values (inclusive on both ends).

```

month: 1..12 = 6          # OK
# @const bad: 1..12 = 0    # compile error: out of range
# @const bad: 1..12 = 13   # compile error: out of range

t: -10..10 = -5          # negative ranges are supported

```

With Mutable Variables (Runtime Check)

```

x: 1..12 = 6
x = 12          # OK
# x = dynamic_value() # runtime check: exits if out of range

```

In Function Parameters

```

function set_month(m: 1..12) -> int:
    return m

set_month(6)          # OK
# set_month(13)        # compile error (constant argument)

```

none Keyword and Option Type Shorthand

The `none` keyword represents the absence of a value for Option types, equivalent to `None`.

The `T?` syntax is a shorthand for `Option<T>`.

```

x: int? = 42          # equivalent to Option<int>
y: int? = none        # equivalent to None

function find(xs: List<int>, val: int) -> int?:
    for x in xs:
        if x == val:
            return Some(x)
    return none

```

Weak References (`weak T`)

A **weak** reference is a non-owning reference to an ARC-managed value. Unlike strong references, weak references do not increment the strong reference count. When the last strong reference is released, the referenced object is deallocated — and any surviving weak references automatically become `None`.

Weak references are the user-facing mechanism for breaking reference cycles.

Creating a Weak Reference

Use the `weak` keyword in both type annotation and expression position:

```
s = "hello"
w: weak str = weak s
```

The type `weak T` is a new type constructor where `T` must be an ARC-managed type (currently `str`, `List<T>`, `Map<K, V>`, `Set<T>`).

Accessing a Weak Reference (Upgrade)

Accessing a weak variable automatically performs an **upgrade** —an atomic check-and-increment of the strong reference count. The result is always `Option<T>`:

- `Some(value)` if the referent is still alive (strong count > 0)
- `None` if the referent has been deallocated (strong count == 0)

```
s = "alive"
w: weak str = weak s
match w:
  case Some(v):
    print(v)           # "alive"
  case None:
    print("deallocated")
```

The coalesce operator (`??`) also works with weak references:

```
w: weak str = weak s
val = w ?? "default"
```

Reassignment

Weak references can be reassigned. The old weak reference is released and the new one is retained:

```
a = "first"
b = "second"
w: weak str = weak a
w = weak b
```

Thread Safety

The upgrade operation uses a compare-and-swap (CAS) loop internally, making it safe to use across threads. This is essential since the strong reference may be released concurrently.

Scope Cleanup

Weak references are automatically released when they go out of scope. If both strong and weak reference counts reach zero, the ARC header is freed.

F-String (String Interpolation)

String interpolation with the `f"..."` syntax. Expressions inside `{}` are evaluated and converted to strings.

```
name = "world"
print(f"Hello {name}")      # Hello world

a = 1
b = 2
print(f"{a} + {b} = {a + b}")  # 1 + 2 = 3
```

Supported Types in Interpolation

Any expression that evaluates to `int`, `float`, `bool`, `str`, a record type, a tuple, or a collection type (`List`, `Map`, `Set`) can be used inside `{}`.

```
xs = [1, 2, 3]
print(f"items: {xs}")      # items: [1, 2, 3]

t = (1, "hello")
print(f"tuple: {t}")      # tuple: (1, hello)
```

Escape Sequences

Sequence	Output
<code>{{</code>	<code>{</code> (literal brace)
<code>}}</code>	<code>}</code> (literal brace)
<code>\n \r \t \\ \"</code>	Same as regular strings

```
print(f"{{braces}}")      # {braces}
```

Type Casting (as)

Explicit type conversion using the `as` keyword.

```
x = 42 as float      # 42.0
y = 3.14 as int      # 3
z = 1 as bool        # true
s = 42 as str        # "42"
b = 255 as u8        # u8 value 255
```

Supported Casts

From	To	Behavior
<code>int</code>	<code>float</code>	<code>SIToFP</code>
<code>float</code>	<code>int</code>	Truncation (<code>FPToSI</code>)
<code>int</code>	<code>bool</code>	<code>0 -> false</code> , non-zero -> <code>true</code>
<code>bool</code>	<code>int</code>	<code>false -> 0</code> , <code>true -> 1</code>
<code>int / float / bool</code>	<code>str</code>	String representation
<code>int</code>	<code>u8</code>	Truncation (lower 8 bits)
<code>u8</code>	<code>int</code>	Zero extension

`int` | `i8` / `i16` / `i32` / `i64` | Truncation (or identity for `i64`) |
`i8` / `i16` / `i32` / `i64` | `int` | Sign extension (`SExt`) |

```

int | u8 / u16 / u32 / u64 | Truncation (or identity for u64) |
u8 / u16 / u32 / u64 | int | Zero extension (ZExt) |
signed | signed (wider) | Sign extension (SExt) |
signed | signed (narrower) | Truncation |
unsigned | unsigned/signed (wider) | Zero extension (ZExt) |
unsigned | unsigned/signed (narrower) | Truncation |
signed / unsigned int | float | SIToFP / UIToFP then f64 |
float | signed / unsigned int | FPToSI / FPToUI |
float | f32 | FPTrunc |
f32 | float | FPEExt |
signed int | f32 | SIToFP |
unsigned int | f32 | UIToFP |
f32 | signed / unsigned int | FPToSI / FPToUI |

```

The target type of `as` supports the full type syntax, including generic types:

```

x = value as Option<int>
y = data as Map<str, int>

```

Any `as` cast (including with generics) must be a built-in cast or have a matching user-defined operator `as`, otherwise it is a compile error. Use `to_int()` / `to_float()` for string-to-number conversions.

Enum with Associated Data (ADT)

Enum variants can carry associated data by specifying types in parentheses after the variant name. Variants without parentheses remain simple tags.

```

enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point

```

Named Fields

Variants can optionally use named fields for documentation clarity. Named fields make variant definitions self-describing but do not change runtime behavior — construction and pattern matching remain positional.

```

enum Shape:
    Circle(radius: float)
    Rectangle(width: float, height: float)
    Point

```

Rules: - Field names must be `snake_case`. - Within a single variant, all fields must be either named or unnamed (no mixing). - Duplicate field names within a variant are a compile error.

Constructor

Use the `EnumName::Variant(value)` syntax to construct a variant with data. Arguments are always positional, even when fields are named.

```

c = Shape::Circle(3.14)
r = Shape::Rectangle(4.0, 5.0)
p = Shape::Point

```

Pattern Matching with Binding

Use `case EnumName::Variant(binding):` to extract the associated data. Bindings use user-chosen variable names, not field names.

```

match c:
    case Shape::Circle(r):
        print(r)           # 3.14
    case Shape::Rectangle(w, h):
        print(w)
        print(h)
    case Shape::Point:
        print("point")

```

Internal Representation

An ADT enum is stored as a tagged union: { i64 tag, [N x i8] data } where N is sized to fit the largest variant's payload.

Generic Enum

An enum can have type parameters using angle-bracket syntax <T>. This allows the same enum shape to hold different payload types.

```

enum MyOption<T>:
    MySome(T)
    MyNone

```

Usage

Instantiate by providing a concrete type argument. The type argument is required when the compiler cannot infer it.

```

a = MyOption<int>::MySome(42)
b = MyOption<int>::MyNone

```

```

match a:
    case MyOption::MySome(v):
        print(v)           # 42
    case MyOption::MyNone:
        print("none")

```

Error Type

A built-in type for error handling. **Error** has two fields: **message** (str) and **code** (int).

```

e = Error("something went wrong")           # code defaults to 0
e2 = Error("not found", 404)                # explicit code

print(e.message)    # something went wrong
print(e2.code)      # 404
print(e2)            # Error: not found (code: 404)

```

Error Handling with Result

Functions that can fail return **Result<V, E>**:

```

function divide(a: int, b: int) -> Result<int, Error>:
    if b == 0:

```

```

        return Err(Error("division by zero"))
    return Ok(a // b)

match divide(10, 2):
    case Ok(v):
        print(v)                # 5
    case Err(e):
        print(e.message)

```

When the return value is not meaningful, use `Result<Unit, Error>`:

```

function save(path: str, data: str) -> Result<Unit, Error>:
    return Ok(0 as u8)          # Unit placeholder

match save("/tmp/test.txt", "hello"):
    case Ok(_):
        print("saved")
    case Err(e):
        print(e.message)

```

Result Type

`Result<V, E>` is a built-in parameterized type with two constructors:

- `Ok(value)` — success variant
- `Err(error)` — error variant

It is used with `match` for exhaustive error handling. Both `Ok` and `Err` cases must be covered (or use `_` wildcard).

Test matchers: - `expect(x).to_be_ok()` — asserts the result is `Ok` - `expect(x).to_be_err()` — asserts the result is `Err`

Internal Representation

Error is represented as `{ ptr message, i64 code }`. `Result<V, E>` is represented as `{ i1 isOk, V okValue, E errValue }`.

Union Type

You can declare a variable that may hold one of multiple types using `|`.

```

x: int | str = 42
x = "hello"      # Reassignment is allowed (any type in the union)
print(x)         # hello

```

Usage in Function Parameters and Return Types

```

function show(x: int | str) -> int:
    print(x)
    return 0

function get_val(flag: bool) -> int | str:
    if flag:
        return 42
    return "hello"

```

Internal Representation

A union type is represented as `{ i64 tag, [N x i8] data }`. The `tag` indicates the index of each component type (sorted alphabetically), and `data` is a byte array sized to the largest component type.

Constraints

- Assigning a type not included in the union causes a compile error
- `int | str` and `str | int` are the same type (normalized)
- When printing a union value with `print()`, the value is displayed using the appropriate type based on the runtime tag

any Type

The `any` type is a built-in dynamic type that can hold any primitive value. It follows Python's approach of allowing flexible typing — when you don't need static type guarantees, `any` lets you write code that works with multiple types without explicit generics or union types.

Supported Types

`any` can hold the following types:

Type	Tag	Description
<code>int</code>	0	64-bit signed integer
<code>float</code>	1	64-bit floating-point number
<code>bool</code>	2	Boolean value
<code>str</code>	3	String
<code>Unit</code>	4	Unit value (for functions with no return value)

`any` **cannot** hold collection types (`List`, `Map`, `Set`), resource types (`TcpListener`, `TcpStream`, etc.), function pointers, or user-defined types (`record`, `enum`).

Internal Representation

`any` is implemented as a tagged union:

```
{ i64 tag, [8 x i8] data }    // 16 bytes total
```

The `tag` field identifies the stored type, and the `data` field holds the value (up to 8 bytes).

Wrapping and Unwrapping

Concrete values are automatically **wrapped** when assigned to `any`, and `any` values are automatically **unwrapped** when assigned to a concrete type.

```
# Wrapping: concrete → any
x: any = 42           # int is wrapped into any
x = "hello"          # reassignment with a different type is allowed

# Unwrapping: any → concrete
function get_value() -> any:
    return 42
n: int = get_value()  # any(int) is unwrapped to int

# int → float auto-promotion during unwrap
f: float = get_value() # any(int) is unwrapped and promoted to float
```

If the runtime type does not match the target type (e.g., unwrapping `any(str)` into an `int` variable), a **runtime error** occurs.

Reassignment

An `any` variable can be reassigned to a value of any supported type:

```
x: any = 42
x = 3.14      # OK: now holds float
x = "hello"   # OK: now holds str
x = true      # OK: now holds bool
```

Arithmetic Operations

When both operands are `any`, arithmetic operations dispatch at runtime based on the actual types:

Operation	Types	Result
+	int + int	int
+	float + float	float
+	int + float	float
+	str + str	str (concatenation)
-	numeric	int or float
*	numeric	int or float
*	str * int / int * str	str (repetition)
/	numeric	float (always)
//	int // int	int
//	with float	float
%	numeric	int or float
**	numeric	float (always)
unary -	int	int
unary -	float	float

When one operand is `any` and the other is a concrete type, the concrete value is automatically wrapped before the operation.

```
x: any = 10
y: any = x + 20    # 20 is auto-wrapped; result is any(int) = 30
```

Incompatible type combinations (e.g., `str - int`) cause a **runtime error**.

Comparison Operations

Operation	Behavior
<code>==, !=</code>	Works for same types; int/float mixing is allowed
<code><, <=, >, >=</code>	Numeric (int/float mixing allowed) and string (lexicographic)

```
x: any = 3
y: any = 3.0
print(x == y)    # true (int/float comparison)
```

Type mismatches in comparison (e.g., `int < str`) cause a **runtime error**.

String Conversion

any values support `print()` and f-string interpolation:

```
x: any = 42
print(x)           # 42
print(f"value: {x}") # value: 42
```

Conversion rules: `int` $\rightarrow$ decimal string, `float` $\rightarrow$ %g format, `bool` $\rightarrow$ "true"/"false", `str` $\rightarrow$ as-is, `Unit` $\rightarrow$ "Unit".

Passing any to Typed Functions

An `any` value can be passed to a function with concrete parameter types. The value is automatically unwrapped with a runtime type check:

```
function add_one(x: int) -> int:
    return x + 1

v: any = 42
result = add_one(v) # any(int) is unwrapped to int; result is 43
```

Type Rules (Type Conversion in Operations)

Operation	Left	Right	Result Type	Notes
<code>+</code>	<code>int</code>	<code>int</code>	<code>int</code>	Low-level type: native-width unsigned operations, no implicit promotion
<code>*</code>	<code>u8</code>	<code>u8</code>	<code>u8</code>	
<code>+</code>	<code>float</code> or <code>int</code>	<code>float</code> or <code>int</code> (one is <code>float</code>)	<code>float</code>	Implicit float promotion
<code>/</code>	any numeric	any numeric	<code>float</code>	Always float
<code>//</code>	any numeric	any numeric	<code>int</code> or <code>float</code>	Floor division (toward $-\infty$); <code>int</code> for <code>int</code> operands, <code>float</code> if either is <code>float</code>
<code>**</code>	any numeric	any numeric	<code>float</code>	Uses <code>libm pow</code>
<code>%</code>	<code>int</code>	<code>int</code>	<code>int</code>	
<code>%</code>	<code>float</code> or <code>int</code>	<code>float</code> or <code>int</code> (one is <code>float</code>)	<code>float</code>	
<code>+</code>	<code>str</code>	<code>str</code>	<code>str</code>	String concatenation
<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>	<code>str</code>	<code>str</code>	<code>bool</code>	Lexicographic comparison
<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>	numeric or <code>bool</code>	numeric or <code>bool</code>	<code>bool</code>	Whether the element is in the set
<code>in</code>	any	Set	<code>bool</code>	
<code>&</code> <code> </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code>	<code>int</code>	<code>int</code>	<code>int</code>	Error for <code>float</code>
<code>+</code> <code>-</code> <code>*</code>	<code>i32</code>	<code>i32</code>	<code>i32</code>	Low-level types: no implicit conversion, same type required

Operation	Left	Right	Result Type	Notes
/ //	i32	i32	i32	Signed integer division (SDiv)
/ //	u32	u32	u32	Unsigned integer division (UDiv)
%	i32	i32	i32	Signed remainder (SRem)
%	u32	u32	u32	Unsigned remainder (URem)
+ - * /	f32	f32	f32	
== !=	i32/u32	i32/u32	bool	Sign-agnostic equality
< <= > >=	i32	i32	bool	Signed comparison (ICMP_SLT etc.)
< <= > >=	u32	u32	bool	Unsigned comparison (ICMP_ULT etc.)
>>	i32	i32	i32	Arithmetic right shift (sign-preserving)
>>	u32	u32	u32	Logical right shift (zero-fill)
**	low-level	any	error	Power operator not supported for low-level types
mixed	low-level	different	error	Mixing low-level and high-level types is a compile error

Escape Sequences (in str Literals)

Sequence	Meaning
\n	Newline
\r	Carriage return
\t	Tab
\\	Backslash
\"	Double quote
\0	Null character

Type Safety Constraints

- **Implicit widening conversions** – Safe widening conversions are supported in function calls: `u8 → int`, `u8 → float`, `int → float`. For binary operators, mixing `int` and `float` triggers float promotion. `u8` is a low-level type that operates at native width with unsigned semantics; mixing `u8` with `int` in binary operators is a compile error. Narrowing conversions (e.g., `float → int`) are not allowed implicitly. Narrowing conversion from an `int` literal to `u8` is only allowed with a type annotation `b`: `u8 = 42`.
- **Variable types are fixed at declaration** – A variable declared as `int` cannot be reassigned a `float` value.
- **Bitwise operations are for int only** – Applying bitwise operations to `float` or `bool` causes a compile error.

- **Non-bool types can be used in conditions** – if conditions accept `int` (0 = false, non-zero = true) and other types besides `bool`.
- **Numeric literal separators** – Underscores can be used as visual separators in numeric literals: `100_000`, `0xFF_FF`, `0b1010_0101`, `3.14_159`. Underscores must appear between digits (no leading, trailing, or consecutive underscores).
- **Numeric literal suffixes** – Low-level types can be specified via literal suffixes: `42i32`, `255u8`, `3.14f32`, `.5f32`, `0xFFu8`, `0b1010u8`. An integer literal with a float suffix (`42f32`) produces a float value. A float literal with an integer suffix (`3.14i32`) is a compile error. Out-of-range values (e.g., `256u8`, `129i8`) are also compile errors.
- **Low-level numeric types (`i8`, `i16`, `i32`, `i64`, `u8`, `u16`, `u32`, `u64`, `f32`) have no implicit conversions** – Mixing low-level types with each other or with high-level types (`int`, `float`) causes a compile error. Use explicit `as` casts. The `/` operator on low-level integers performs integer division (like Rust), not float division. Signed types use `SDiv/SRem`, unsigned types use `UDiv/URem`.
- **Signed vs unsigned** – Signed types (`i8`, `i16`, `i32`, `i64`) use signed comparison (`ICMP_SLT` etc.) and arithmetic right shift (`AShr`). Unsigned types (`u8`, `u16`, `u32`, `u64`) use unsigned comparison (`ICMP_ULT` etc.) and logical right shift (`LShr`). The `>>>` operator always performs logical shift regardless of signedness.
- **int arithmetic overflow is a runtime error** – Arithmetic (`+`, `-`, `*`, unary `-`) on the high-level `int` type raises a runtime error on overflow, similar to Swift’s default behavior. This prevents silent data corruption from two’s complement wrapping. Constant expressions that overflow are caught at compile time.
- **Low-level integer overflow wraps around** – Arithmetic on low-level integer types uses Ry-defined two’s complement wrapping on overflow for signed types and modular arithmetic for unsigned types. For example, `2147483647i32 + 1i32` wraps to `-2147483648`. For explicit overflow control, use `checked_add/sub/mul` (returns `Result<T, Error>`), `saturating_add/sub/mul` (clamps to type bounds), or `wrapping_add/sub/mul` (self-documenting wrapping). See [Function Reference](#).